1. FILOSOFÍA LEAN

Lean es una filosofía de gestión originada en la industria automotriz japonesa, vinculada con el sistema de producción de Toyota, cuyo **objetivo es maximizar el valor generado al cliente con el mínimo consumo de recursos**. Para ello, busca identificar y reducir los esfuerzos que no generan valor, concentrando los recursos en los aspectos esenciales del producto.

Para lograr este objetivo, Lean propone una **visión integrada de la cultura y la estrategia de la organización**, orientada a entregar productos de alta calidad, a bajo costo y en el momento oportuno. Esto implica producir exactamente lo que el cliente necesita, cuando y donde lo necesita, utilizando los recursos indispensables. **Es el cliente quien determina qué aporta valor al producto o servicio.**

Además, Lean utiliza el método **Just in Time (JIT)**, que consiste en producir y entregar cada resultado del trabajo únicamente cuando se necesita y en la cantidad necesaria, tanto entre las distintas fases del desarrollo como en la entrega del producto final al cliente. De esta manera, se evita realizar trabajo anticipadamente y acumular productos o artefactos que todavía no serán utilizados. En el desarrollo de software, esto se refleja en la idea de **«analizar cuando se necesite, no antes»**.

7 PRINCIPIOS LEAN

Los principios de Lean surgieron en el ámbito industrial y fueron adaptados al desarrollo de software, debido a las particularidades de este producto intangible.

1- ELIMINAR DESPERDICIOS

Es uno de los principios fundamentales de Lean. Busca lograr un **proceso eficiente que maximice la generación de valor**, identificando las actividades que no aportan valor directamente para eliminarlas o minimizarlas según corresponda.

Se distinguen dos tipos de desperdicios:

- **Desperdicio necesario (Tipo 1):** Actividades que no generan valor directamente, pero que resultan necesarias para el proceso. Se busca minimizarlas.
- **Desperdicio puro (Tipo 2):** Actividades innecesarias que no aportan valor. Se busca eliminarlas.

No toda actividad que consume recursos sin generar directamente una nueva funcionalidad debe eliminarse. Por ejemplo, el **Testing bien realizado contribuye a la calidad del producto**, mientras que las pruebas improvisadas o ineficientes pueden generar trabajo innecesario y retrabajo. Por eso, Lean propone evaluar el valor y la necesidad de cada actividad antes de eliminarla.

La identificación y reducción constante de desperdicios permite **mejorar continuamente el proceso**, objetivo que Lean comparte con Agile. Ambos enfoques buscan evitar el trabajo innecesario y concentrarse en generar valor para el cliente. Esto se refleja en la simplicidad, la entrega frecuente de software funcionando y la importancia de completar funcionalidades útiles en lugar de acumular trabajo parcialmente terminado.

2- AMPLIFICAR EL APRENDIZAJE

Busca fomentar el **aprendizaje colectivo mediante el intercambio de experiencias y conocimientos** entre los integrantes del equipo. Para ello, se promueve la transformación del conocimiento implícito (individual) en explícito (compartido y accesible), evitando que la organización dependa exclusivamente de lo que saben determinadas personas.

Se relaciona con Agile por la colaboración entre personas técnicas y del negocio, la retroalimentación frecuente y la adaptación continua a partir de lo aprendido.

3- EMBEBER LA INTEGRIDAD CONCEPTUAL

Consiste en **construir un producto de calidad cuyos componentes y funcionalidades se integren de manera coherente y consistente**, formando un sistema que satisfaga las necesidades y expectativas del cliente. Para ello, la calidad debe

incorporarse durante todo el desarrollo, mediante decisiones de diseño que favorezcan la simplicidad, la mantenibilidad y la excelencia técnica. **La calidad no se negocia ni se deja únicamente para el final.**

4- DIFERIR COMPROMISOS HASTA EL ULTIMO MOMENTO RESPONSABLE

Consiste en **postergar las decisiones importantes, especialmente aquellas difíciles de revertir, hasta disponer de suficiente información para tomarlas correctamente**, pero sin esperar tanto que resulte demasiado tarde. Esto permite reducir la incertidumbre y evitar decisiones prematuras que podrían generar retrabajo.

En Agile, este principio se refleja en el refinamiento progresivo del Product Backlog, donde los requerimientos se detallan a medida que se obtiene mayor información sobre las necesidades del cliente, en lugar de intentar definir todo anticipadamente. De esta manera, se mantiene la flexibilidad para incorporar cambios y tomar decisiones informadas.

5- DAR PODER AL EQUIPO

Consiste en **otorgar autonomía, responsabilidad y capacidad de decisión al equipo**, favoreciendo que sus integrantes puedan organizar su trabajo y resolver problemas sin depender constantemente de decisiones de sus superiores.

Para ello, se necesitan equipos multifuncionales, capacitados y motivados, junto con líderes que confíen en sus integrantes, deleguen decisiones y fomenten su participación.

Este principio se relaciona con Agile, que promueve equipos autoorganizados capaces de tomar decisiones técnicas y de trabajo. Sin embargo, llevarlo a la práctica puede ser difícil, ya que requiere un compromiso real de los integrantes, capacitación y una cultura organizacional que permita ejercer esa autonomía.

6- VER EL TODO

Consiste en **comprender y optimizar el producto y el proceso de desarrollo como un conjunto**, en lugar de enfocarse únicamente en mejorar sus partes de manera aislada. Para ello, los objetivos de cada área o equipo deben estar alineados con los objetivos generales de la organización y con el valor que se busca entregar al cliente.

Esto es importante porque **mejorar una actividad individual no necesariamente mejora el resultado global**.

7- ENTREGAR LO ANTES POSIBLE

Busca **entregar al cliente software funcionando y que aporte valor de manera temprana y frecuente**, antes de que las necesidades del negocio cambien o el producto pierda relevancia.

Para lograrlo, Lean propone acortar los ciclos de desarrollo y entregar incrementos pequeños y útiles, lo que permite obtener retroalimentación temprana, adaptarse a los cambios y evitar desarrollar funcionalidades que podrían dejar de ser necesarias.

Este principio se relaciona con Agile, que también promueve las entregas frecuentes de software funcionando para satisfacer continuamente las necesidades del cliente.

RELACIÓN LEAN-AGIL

Lean es anterior al Manifiesto Ágil y ambos enfoques comparten fundamentos como la **orientación al cliente, la generación de valor, la mejora continua, el aprendizaje y la autonomía de los equipos**.

Sin embargo, tienen enfoques diferentes: **Lean se concentra en maximizar el valor y eliminar desperdicios mediante la mejora continua de los procesos**, mientras que **Agile enfatiza la adaptación al cambio, la colaboración con el cliente y la entrega frecuente de software funcionando**.

Ambos son compatibles y pueden complementarse: Agile permite responder a las necesidades cambiantes del cliente, mientras que Lean ayuda a realizar ese trabajo de manera más eficiente.

LOS 7+1 DESPERDICIOS DE LEAN

A continuación, se presentan los desperdicios de Lean adaptados al desarrollo de software y, entre corchetes, sus equivalentes en la industria manufacturera.

1- TRABAJO PARCIALMENTE TERMINADO [Inventario]

Representa el **trabajo que se inicia, pero no se termina**, por ejemplo: historias de usuario, casos de uso, correcciones de defectos, etc. Este trabajo acumula esfuerzo sin generar todavía valor entregado y puede requerir tiempo adicional cuando se retoma. Por eso, se prioriza completar las unidades de trabajo antes que acumular tareas iniciadas.

Aunque es posible medir el avance parcial, una funcionalidad solo se considera terminada cuando cumple los criterios establecidos. Un porcentaje de avance no equivale necesariamente a valor entregado al cliente.

2- CARACTERÍSTICAS EXTRA [Sobreproducción]

Consiste en desarrollar **funcionalidades que el cliente no necesita o que no aportan valor**. También puede relacionarse con la especificación excesivamente anticipada (*Up Front Specification*), ya que los requerimientos definidos cuando todavía existe mucha incertidumbre pueden volverse obsoletos, incompletos o incorrectos, generando trabajo innecesario o retrabajo.

3- PROCESO EXTRA [Proceso extra]

Consiste en realizar **actividades o pasos dentro del proceso de desarrollo que no aportan valor y que podrían evitarse**. Esto puede ocurrir cuando los procesos incorporan tareas burocráticas o controles innecesarios. Por eso, Lean propone revisar continuamente las actividades para identificar cuáles pueden simplificarse o eliminarse sin comprometer la calidad.

4- CAMBIO DE TAREAS Y TRANSFERENCIA DE CONOCIMIENTO [Transporte]

Se relaciona con el traspaso de trabajo, tareas y conocimientos entre personas o equipos, cuando estas transferencias generan esfuerzos adicionales, interrupciones o pérdida de información. Por ejemplo, si un analista entrega requerimientos incompletos a un desarrollador, este deberá invertir tiempo en solicitar aclaraciones. **El desperdicio no es compartir conocimiento, sino las ineficiencias que pueden producirse durante su transferencia.**

También puede aparecer el desperdicio de **búsqueda de información**, especialmente cuando la información necesaria no está disponible o resulta difícil de localizar. Una adecuada Gestión de Configuración de Software (SCM) contribuye a reducir este problema al facilitar la organización, disponibilidad y trazabilidad de la información.

5- ESPERAS O DEMORAS [Esperas]

Representa el **tiempo durante el cual una persona o unidad de trabajo permanece detenida porque depende de otra actividad, persona o aprobación**. Este desperdicio puede reducirse mediante equipos multidisciplinarios, una mejor coordinación y la eliminación de dependencias innecesarias.

6- CAMBIO DE CONTEXTOS (SETEO) [Movimiento]

Ocurre cuando **una persona debe cambiar continuamente de contexto de trabajo**, por ejemplo, alternando entre diferentes proyectos o funcionalidades. Como el desarrollo de software es una actividad intelectual, estos cambios requieren tiempo para recuperar la concentración y comprender nuevamente el estado del trabajo, generando pérdidas de productividad.

Para reducir este desperdicio, Kanban propone **limitar el trabajo en progreso (WIP)**, favoreciendo que se termine el trabajo iniciado antes de comenzar nuevas tareas.

7- DEFECTOS [Defectos]

Son problemas introducidos en los productos de trabajo que **requieren correcciones y generan retrabajo**, especialmente cuando se detectan en etapas posteriores a aquellas donde se originaron. Por ejemplo, una funcionalidad que llega a Testing con defectos debe regresar a desarrollo para su corrección, aumentando los tiempos y costos.

El **Aseguramiento de Calidad (QA)** busca prevenir y detectar tempranamente estos problemas a lo largo del desarrollo.

7+1- TALENTO NO UTILIZADO [Talento no utilizado]

Consiste en **desaprovechar los conocimientos, habilidades, experiencia o creatividad de los integrantes del equipo**. Por ejemplo, asignar tareas que no aprovechan sus capacidades o no escuchar sus propuestas de mejora. Esto constituye un desperdicio porque se pierden oportunidades de resolver problemas, mejorar los procesos y generar valor. Por eso, Lean busca fomentar la participación del equipo y aprovechar las capacidades de sus integrantes.

2. KANBAN

Kanban es un **método para definir, gestionar y mejorar servicios que realizan trabajo del conocimiento**, como el desarrollo de software. Su objetivo es mejorar continuamente el flujo de trabajo para entregar valor al cliente de manera eficiente y predecible. No es un proceso de desarrollo de software ni una metodología completa de administración de proyectos.

Se basa en el principio **«empezar con lo que haces ahora»**, es decir, comprender el proceso existente y mejorarlo mediante cambios graduales, respetando inicialmente los roles y responsabilidades actuales. Esto permite reducir la resistencia al cambio y favorecer la evolución continua del proceso.

Su origen se relaciona con el sistema Just in Time de Toyota. La palabra japonesa *kan-ban* significa «tarjeta de señalización», utilizada para indicar cuándo se necesita producir o reponer algo. Kanban, al igual que Lean, tiene sus orígenes en la industria manufacturera y fue adaptado para gestionar el trabajo del conocimiento, incluido el desarrollo de software.

RELACIÓN KANBAN-LEAN

Kanban aprovecha los fundamentos de Lean y los aplica mediante prácticas concretas:

1. **Definir el valor desde la perspectiva del cliente:** Tanto Lean como Kanban orientan el trabajo hacia aquello que el cliente considera valioso. Lean establece este fundamento, mientras que Kanban ayuda a aplicarlo mediante la gestión y mejora del flujo de trabajo
2. **Identificar y eliminar desperdicios:** Mediante la visualización del trabajo, Kanban permite detectar actividades innecesarias, esperas y retrabajos para mejorar la eficiencia del proceso. Aplica directamente el principio Lean de **eliminar desperdicios**.
3. **Limitar el trabajo en progreso (WIP):** Permite reducir desperdicios como el trabajo parcialmente terminado y los cambios de contexto, favoreciendo la finalización de tareas antes de iniciar nuevas. Se relaciona con el principio Lean de **eliminar desperdicios** y **con Just in Time**.
4. **Identificar y reducir las barreras en el flujo:** Busca detectar cuellos de botella, bloqueos y dependencias que dificultan el avance del trabajo. Se relaciona con **ver el todo** y **entregar lo antes posible**, ya que se pretende optimizar el flujo completo y no solamente las actividades individuales.
5. **Dar poder al equipo y promover la mejora continua:** Fomenta que los integrantes participen en la identificación de problemas y en la propuesta de soluciones. Se relaciona con los principios Lean de **dar poder al equipo** y **amplificar el aprendizaje**, promoviendo una cultura de mejora continua.

VALORES DE KANBAN

1. RESPETO:
Valorar, entender y mostrar consideración por las personas. Es la base sobre la cual reposan el resto de los valores.

2. TRANSPARENCIA:
La creencia de que compartir información abiertamente, de forma clara y directa, mejora el flujo de valor de negocio.

3. EQUILIBRIO:
El entendimiento de que los diferentes aspectos, puntos de vista y capacidades deben ser equilibrados para conseguir efectividad. Algunos aspectos, como demanda y capacidad, causarán colapso si no se encuentran equilibrados por un periodo prolongado.

4. COLABORACIÓN:
Trabajar juntos. El Método Kanban fue formulado para mejorar la manera en que las personas trabajan juntas, por ello, la colaboración está en su corazón.

5. FOCO EN EL CLIENTE:
Los clientes y el valor que estos reciben es el foco natural en Kanban, cada sistema kanban fluye a un punto de valor realizable, este es cuando los clientes reciben un elemento solicitado o servicio. Los clientes pueden ser internos o externos a la organización como un todo.

6. FLUJO:
Representa el avance del trabajo a través de las distintas etapas del proceso, desde su inicio hasta la entrega de valor al cliente. Kanban busca visualizar y gestionar este flujo para que el trabajo avance de manera continua, estable y predecible, reduciendo esperas, bloqueos y cuellos de botella.

7. LIDERAZGO:
La habilidad de inspirar a otros a la acción a través del ejemplo, de las palabras y la reflexión. Muchas organizaciones tienen diferentes grados de jerarquía estructural, pero en Kanban, el liderazgo es necesario a todos los niveles para alcanzar la entrega de valor y la mejora.

8. ENTENDIMIENTO:
Esto abarca conocimiento tanto individual como de la organización para ir hacia adelante. Kanban es un método de mejora, por lo que conocer el punto de inicio es la base de todo.

9. ACUERDO:
El compromiso de avanzar juntos hacia los objetivos, respetando y donde sea posible, acomodando las diferencias de opinión o aproximaciones. Esto no es gestión por consenso sino un compromiso dinámico para mejorar.

PRINCIPIOS FUNDACIONALES

Hay seis principios fundacionales en Kanban que pueden ser divididos en dos grupos: los principios de gestión de cambios y los principios de entrega o despliegue de servicios.

PRINCIPIOS DE GESTIÓN DE CAMBIOS

Los principios de gestión del cambio orientan **cómo introducir mejoras en una organización**, teniendo en cuenta su forma actual de trabajar y a las personas que participan en ella. Kanban propone cambios graduales, respetando inicialmente los roles y responsabilidades existentes para reducir la resistencia y favorecer la mejora continua.

1. **Empezar con lo que estás haciendo ahora**:
    - Entender los procesos actuales tal y como están siendo realizados en la actualidad.
    - Respetar los roles actuales, las responsabilidades de cada persona y los puestos de trabajo.
2. Acordar en buscar la **mejora a través del cambio evolutivo**.
3. **Fomentar el liderazgo en cada nivel de la organización**, desde las contribuciones individuales de cada persona hasta las posiciones más senior de la organización.

PRINCIPIOS DE DESPLIEGUE DE SERVICIOS

Las organizaciones independientemente de su tamaño, son ecosistemas de servicios interdependientes. Kanban reconoce esto y define principios aplicables a todos los servicios:

4. **Entender las necesidades y expectativas de tus clientes y enfocarte en ellas.**
5. **Gestionar el trabajo**: dejar que la gente se autoorganice alrededor de las tareas.
6. **Revisar periódicamente la red de servicios y sus políticas:** las organizaciones están formadas por servicios y equipos que pueden depender unos de otros. Kanban propone evaluar periódicamente cómo funcionan e interactúan esos servicios, revisando sus políticas e identificando oportunidades de mejora para obtener mejores resultados para el cliente y el negocio.

PRACTICAS GENERALES DE KANBAN

1- VISUALIZACION DEL TRABAJO

Kanban utiliza un **tablero visual para representar las etapas del proceso y mostrar cómo avanza el trabajo**. Las columnas representan los estados o actividades, mientras que las tarjetas (*kan-bans*) representan las unidades de trabajo, como historias de usuario, funcionalidades, defectos o solicitudes de cambio.

Se distinguen dos tipos de columnas:

- **Producción:** Contienen elementos sobre los que se está trabajando activamente.
- **Acumulación:** Contienen elementos que terminaron una actividad y esperan que la siguiente etapa tenga capacidad para recibirlos.

Esta visualización permite que todos los integrantes conozcan el estado del trabajo, identifiquen bloqueos y acumulaciones, y tomen decisiones de manera más rápida y colaborativa.

2- LIMITAR EL WiP (Work in Progress)

El WIP representa el **trabajo que se encuentra en progreso**. Limitarlo consiste en establecer una cantidad máxima de elementos que pueden encontrarse simultáneamente en una etapa o parte del proceso, evitando la sobrecarga y la acumulación excesiva de trabajo.

Estos límites favorecen un **sistema Pull (arrastre)**: una etapa toma trabajo de la anterior cuando dispone de capacidad, respetando las políticas establecidas. Esto se diferencia de un sistema Push (empuje), donde el trabajo se envía a la siguiente etapa, aunque esta no tenga capacidad para recibirlo.

Limitar el WIP permite reducir los cambios de contexto, las esperas y el trabajo parcialmente terminado. Kanban prioriza **terminar el trabajo iniciado antes de comenzar nuevas tareas**, mejorando el flujo y los tiempos de entrega.

3- GESTIONAR EL FLUJO DE TRABAJO

Consiste en **lograr que los elementos de trabajo avancen por las distintas etapas del proceso de manera continua, estable y predecible**, evitando acumulaciones, bloqueos y demoras.

Kanban se concentra en optimizar el flujo completo, desde que ingresa una solicitud hasta que se entrega valor al cliente, en lugar de buscar que cada persona permanezca ocupada constantemente.

A diferencia de Scrum, **Kanban no exige trabajar mediante iteraciones de duración fija**. El proceso puede funcionar mediante un flujo continuo de elementos de trabajo y aplicarse dentro de proyectos o servicios existentes.

Para gestionar el flujo, se analizan los cuellos de botella, tiempos de espera y capacidades de las distintas etapas, introduciendo mejoras donde sean necesarias.

Al analizar el tablero Kanban, se recomienda **observar el trabajo de derecha a izquierda**, comenzando por los elementos más próximos a finalizar. Esto permite identificar qué está impidiendo su avance y priorizar su finalización antes de comenzar nuevas tareas, siguiendo la idea de **«dejar de empezar y empezar a terminar»**.

4- HACER EXPLÍCITAS LAS POLÍTICAS

Consiste en **definir y hacer visibles las reglas y acuerdos que determinan cómo se gestiona el trabajo**. Estas políticas deben ser simples, claras, conocidas por el equipo y modificables cuando sea necesario para mejorar el proceso.

Pueden establecer límites WIP, criterios de entrada y salida de cada etapa (como DoR y DoD), condiciones para incorporar trabajo mediante Pull, reglas de reposición y tratamiento de las distintas clases de servicio.

Estas políticas pueden representarse directamente en el tablero Kanban, por ejemplo, junto a las columnas correspondientes, para facilitar su comprensión y cumplimiento.

5- ESTABLECER CICLOS DE RETROALIMENTACIÓN

El circuito de retroalimentación es una parte esencial para cualquier proceso controlado que **nos ayuda a realizar cambios evolutivos**. Se debe definir con qué frecuencia se realizan las reuniones y en qué contexto se presentan ya que es importante para el resultado.

Hay que determinar el periodo adecuado para las reuniones ya que si se realizan revisiones demasiadas frecuentes pueden obligar a cambiar cosas que no se vieron con los cambios anteriores, pero si tardan demasiado, ante un problema existirá un bajo rendimiento durante mucho tiempo.

Kanban utiliza distintas reuniones o **cadencias de retroalimentación**, cuya frecuencia se adapta a las necesidades del equipo. Algunos ejemplos son:

- **Team Kanban Meeting (diaria):** revisar el flujo de trabajo, identificar bloqueos y decidir qué elementos deben avanzar.
- **Team Retrospective (quincenal o mensual):** reflexionar sobre el funcionamiento del equipo e identificar oportunidades de mejora.
- **Team Replenishment Meeting (semanal o según necesidad):** seleccionar los próximos elementos de trabajo que ingresarán al sistema.

6- EVOLUCIONAR EXPERIMENTALMENTE

Kanban propone una cultura de mejora continua mediante **cambios graduales y colaborativos sobre el proceso existente**. Para ello, el equipo identifica problemas, propone mejoras y realiza pequeños experimentos para evaluar sus resultados.

Si un cambio produce una mejora, se mantiene; si no funciona, se revisa o se vuelve al estado anterior. Por eso, se busca experimentar en condiciones donde sea seguro equivocarse, aprendiendo de los resultados para mejorar continuamente el proceso.

APLICAR KANBAN

1. **Comprender el proceso actual:** Identificar cómo trabaja la organización y qué etapas componen su proceso.
2. **Definir los tipos y unidades de trabajo:** Identificar los elementos que se gestionarán, como requerimientos, historias de usuario, defectos, mantenimiento o solicitudes de cambio. Según la demanda, el equipo puede distribuir su capacidad entre los distintos tipos.
3. **Modelar el proceso mediante un tablero:** Representar las etapas del trabajo utilizando columnas de producción y acumulación, y tarjetas para los elementos de trabajo.
4. **Definir límites WIP:** Establecer la cantidad máxima de elementos permitidos en cada etapa o agrupación de columnas, para evitar sobrecargas y mejorar el flujo.
5. **Establecer las clases de servicio y las políticas explícitas:** Acordar cómo se prioriza y trata cada trabajo, los criterios para avanzar, las condiciones de calidad y las reglas de reposición.
6. **Gestionar y mejorar el flujo:** Identificar bloqueos y cuellos de botella, ajustar las capacidades y políticas cuando corresponda y realizar mejoras graduales basadas en los resultados obtenidos.

CLASES DE SERVICIO DE KANBAN

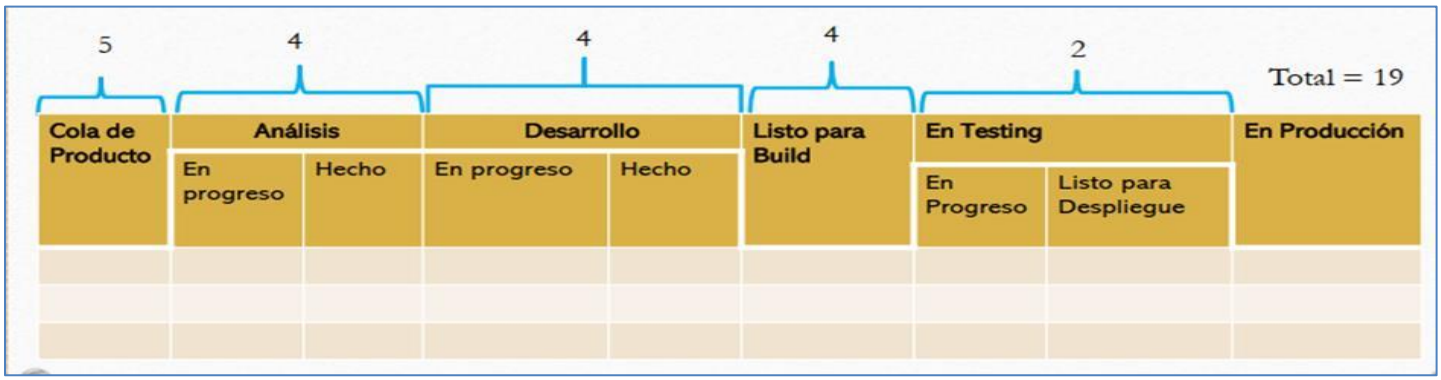
Las clases de servicio permiten **establecer cómo se prioriza y gestiona cada elemento de trabajo** según su urgencia, riesgo e impacto para el negocio. Se distinguen cuatro clases:

- **Expreso:** Trabajos urgentes que requieren atención inmediata. Pueden tener políticas excepcionales, como interrumpir otras tareas o superar temporalmente el límite WIP.
- **Fecha fija:** Trabajos que deben finalizarse antes de una fecha determinada. Su prioridad aumenta cuando existe riesgo de incumplirla.
- **Estándar:** Trabajos habituales que se gestionan según las políticas y prioridades acordadas. Puede utilizarse FIFO (primero en entrar, primero en salir).
- **Intangible:** Trabajos cuyos beneficios no son inmediatos o fácilmente visibles, pero que contribuyen al mantenimiento o mejora futura del sistema.

No deben confundirse con los tipos de trabajo: una historia de usuario, un defecto o una tarea de mantenimiento pueden pertenecer a distintas clases de servicio según sus circunstancias.

Relación FIFO–Pull: Pull determina **cuándo** se puede incorporar un nuevo trabajo, según la capacidad disponible; FIFO determina **cuál** se selecciona primero, siguiendo el orden de ingreso a la cola. FIFO es una política posible, no obligatoria, y puede modificarse para atender trabajos urgentes o con fecha fija.

EJEMPLO TABLERO KANBAN:



Cada columna representa una etapa o estado del proceso, y los números superiores indican la cantidad máxima de elementos permitidos en los sectores correspondientes (WiP). En este ejemplo, la suma de los límites es 19.

Algunas etapas se dividen en **En progreso** (columna de producción) y **Hecho** (columna de acumulación). Un elemento solo puede pasar a la siguiente etapa cuando existe capacidad disponible y cumple las políticas establecidas.

Los límites representan capacidades máximas, **no la cantidad actual de tareas**. Como el ejemplo no contiene tarjetas, no permite identificar cuellos de botella reales, pero sí visualizar cómo se estructuraría el sistema.

Por ejemplo, el límite WIP de 4 para Análisis comprende conjuntamente los elementos de «En progreso» y «Hecho», no cuatro elementos permitidos en cada subcolumna.

MÉTRICAS DE KANBAN

Las métricas más representativas de Kanban son (Se desarrollan en la sección métricas del resumen):

- Cycle Time.
- Lead Time.
- Touch Time.
- Eficiencia del Ciclo de Proceso.

RELACIÓN ENTRE LEAN, KANBAN Y SCRUM

Lean es una filosofía de gestión orientada a maximizar el valor y eliminar desperdicios. **Kanban es un método de gestión y mejora continua** que aplica muchos de estos fundamentos mediante la visualización y optimización del flujo de trabajo.

Por su parte, **Scrum es un framework ágil** que organiza el trabajo mediante Sprints, responsabilidades y eventos definidos, mientras que Kanban no exige iteraciones de duración fija ni prescribe una estructura completa de roles.

Los tres enfoques son compatibles: un equipo puede utilizar Scrum para organizar su trabajo e incorporar prácticas Kanban, como limitar el WIP y visualizar el flujo, para mejorarlo.

3. MÉTRICAS

Una **métrica de software es una medida cuantitativa que permite obtener información sobre el estado, avance o desempeño de un producto, proyecto o proceso**. Su finalidad es representar aspectos de la realidad para evaluar resultados, detectar desviaciones y tomar decisiones fundamentadas.

Para medir una característica se debe definir qué se observará y cómo se cuantificará. Por ejemplo, clasificar defectos como leves, graves o críticos permite distinguir su severidad; contar cuántos defectos corresponden a cada categoría permite obtener una métrica cuantitativa.

Algunas características, como la usabilidad, son difíciles de medir directamente. Por eso pueden utilizarse **indicadores indirectos** relacionados con aquello que se quiere evaluar. Por ejemplo, la cantidad de veces que se utiliza una funcionalidad aporta información sobre su uso, aunque por sí sola no demuestra que sea fácil de utilizar.

CRITERIOS Y CONSIDERACIONES PARA LA MEDICIÓN

Todas las métricas tienen un costo asociado, porque requieren recursos para definirlas, obtener los datos, analizarlos e interpretar sus resultados. Por eso debe evaluarse su costo-beneficio: una métrica solo se justifica si proporciona información útil.

También debe considerarse la **precisión necesaria**. No siempre resulta conveniente invertir más recursos para obtener una medición extremadamente precisa si esa precisión adicional no mejora las decisiones.

La incorporación de métricas debe ser gradual, evitando sobrecargar al equipo con mediciones costosas que después no se utilizan. **No se debe trabajar para las métricas; las métricas deben ayudar a mejorar el trabajo.** Además, **las métricas no deben utilizarse para juzgar, premiar o castigar a las personas**, porque esto puede fomentar la manipulación de los resultados y perjudicar la confiabilidad de la información.

¿PARA QUÉ MEDIR?

- **Monitorear y controlar:** Conocer cómo avanza el trabajo, comparar los resultados con lo planificado y detectar desviaciones.
- **Evaluar:** Comprender el estado, desempeño y calidad de un producto, proyecto o proceso.
- **Estimar y predecir:** Utilizar mediciones históricas como información para anticipar esfuerzo, costos, plazos o resultados futuros.
- **Mejorar:** Identificar problemas y tomar decisiones para optimizar el proceso, el proyecto o el producto.

Métrica y estimación no son lo mismo: una métrica describe o cuantifica una situación observada, mientras que una estimación intenta predecir un resultado futuro. Las mediciones históricas pueden utilizarse como base para realizar estimaciones.

SELECCIÓN DE MÉTRICAS: «MANTÉNGALO SIMPLE»

No conviene medir todo lo que sea técnicamente posible. Una métrica debe responder a una necesidad concreta y aportar información práctica.

Antes de incorporarla, conviene preguntarse:

1. ¿Nos proporciona información que todavía no tenemos?
2. ¿Esa información resulta útil para tomar decisiones?
3. ¿Permite conocer lo que realmente queremos evaluar?

La medición debe requerir el menor esfuerzo razonable y realizarse con una frecuencia adecuada. **Medir sin analizar ni utilizar los resultados constituye un costo sin beneficio.**

MÉTRICAS EN EL ENFOQUE TRADICIONAL

En los enfoques tradicionales basados en procesos definidos, las métricas se utilizan para **monitorear y controlar los proyectos**, analizar sus desviaciones y mejorar los procesos organizacionales.

Se consideran tres dominios de medición: **proceso, proyecto y producto**.

1. MÉTRICAS DE PROCESO

Son métricas de carácter estratégico, orientadas a **evaluar y mejorar los procesos de software de la organización a largo plazo**.

Se construyen a partir de información de múltiples proyectos, cuyos resultados pueden consolidarse y despersonalizarse para obtener indicadores generales del funcionamiento del proceso organizacional.

En este enfoque, el ingeniero de procesos cumple un papel importante en el análisis y la difusión de esos indicadores. Su utilización permite identificar tendencias, problemas recurrentes y oportunidades de mejora de los procesos.

Ejemplos: desviación promedio de estimaciones entre proyectos, defectos por severidad en los distintos productos, tiempos promedio de planificación y propagación de errores entre fases.

2. MÉTRICAS DE PROYECTO

Se enfocan en **evaluar y controlar el desarrollo de un proyecto específico**, considerando principalmente recursos, esfuerzo, costos, tiempo y cumplimiento de la planificación.

Son utilizadas por el líder de proyecto y los involucrados para comparar lo planificado con lo realizado, identificar riesgos, corregir desviaciones y ajustar el trabajo.

Ejemplos: Costos estimados frente a reales, esfuerzo por tarea, fechas de entrega previstas frente a reales, duración de actividades y cantidad de cambios en los requerimientos.

Relación con las métricas de proceso: La información obtenida de distintos proyectos puede consolidarse para evaluar cómo funciona el proceso de desarrollo de toda la organización. Una misma clase de medición puede utilizarse en ambos dominios, pero con distintos objetivos.

3. MÉTRICAS DE PRODUCTO

Se enfocan en **las características de los productos de trabajo y del software construido**, como su tamaño, estructura, calidad y defectos.

Son utilizadas principalmente por el equipo de desarrollo y Testing para evaluar el producto, verificar su adecuación a los requerimientos e identificar problemas técnicos. Estas métricas proporcionan información sobre la calidad del producto, pero ninguna medición aislada permite garantizar que el software esté libre de defectos.

Ejemplos: líneas de código, cantidad de clases o métodos, cantidad de casos de uso por complejidad, defectos por severidad, cobertura de pruebas y densidad de defectos.

MÉTRICAS BÁSICAS PARA UN PROYECTO DE SOFTWARE

Las cuatro métricas básicas se relacionan principalmente con los siguientes dominios:

- **Tamaño → Métrica de producto:** mide la dimensión del software que se está construyendo.
- **Esfuerzo → Métrica de proyecto:** mide la cantidad de trabajo necesaria para realizar las actividades del proyecto.
- **Calendario → Métrica de proyecto:** mide los tiempos y plazos previstos y reales de las actividades y entregas.
- **Defectos → Métrica de producto:** mide la cantidad y características de los defectos encontrados en el software.

Estas asociaciones representan su **dominio principal de medición**. Los datos obtenidos en distintos proyectos también pueden consolidarse para generar métricas de proceso a nivel organizacional.

RELACIÓN CON EL MONITOREO Y CONTROL

En la gestión tradicional, las métricas se definen durante la planificación, estableciendo qué se medirá, cómo se obtendrán los datos y quiénes serán responsables. Su análisis permite comparar los resultados reales con lo planificado y tomar medidas correctivas.

También ayudan a **equilibrar las variables de la triple restricción: alcance, tiempo y costo**, considerando las necesidades del cliente, la calidad requerida y otras restricciones, como la tecnología, las personas y el proceso utilizado.

En la gestión tradicional, **el alcance no es completamente fijo**, ya que los requerimientos pueden modificarse. Sin embargo, constituye una referencia fundamental para planificar los costos y tiempos del proyecto. Las métricas permiten monitorear y controlar estas variables, buscando mantener el equilibrio sin comprometer la calidad.

Las métricas relevantes varían según el rol. Por ejemplo, un desarrollador puede necesitar información sobre esfuerzo y defectos de sus componentes; el equipo, sobre tareas completadas, tamaño y cobertura; y la dirección, sobre costos, plazos y desviaciones generales.

MÉTRICAS EN AMBIENTES ÁGILES

En Agile se adopta una **filosofía minimalista de medición: medir lo necesario y nada más**.

Este enfoque se relaciona con dos principios ágiles: satisfacer al cliente mediante entregas tempranas y continuas de software valioso, y considerar al **software funcionando como la principal medida de progreso**.

Por eso, las métricas ponen especial énfasis en los incrementos de producto que realmente se completan y aportan valor, en lugar de considerar suficiente el avance de tareas o documentación.

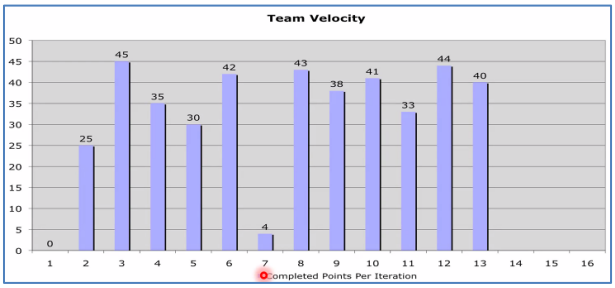
Las mediciones deben servir al equipo para comprender su funcionamiento y mejorar. En particular, los **Story Points y la velocidad dependen de los criterios de estimación de cada equipo**, por lo que no deben utilizarse para comparar directamente la productividad de equipos diferentes.

1. VELOCIDAD (VELOCITY)

La **velocidad mide la cantidad de Story Points correspondientes a historias de usuario completadas y aceptadas en un Sprint**. Se calcula al finalizar el Sprint, considerando únicamente las historias terminadas y aceptadas por el Product Owner. **El trabajo parcialmente terminado no se incluye.**

La velocidad permite observar cuánto trabajo completa el equipo por iteración y utilizar esa información histórica para orientar la planificación de los próximos Sprints.

Se representa habitualmente mediante un gráfico de barras, donde cada barra indica los puntos completados en un Sprint.



La gráfica permite observar cómo varía la velocidad entre iteraciones. Una tendencia relativamente estable favorece la planificación y se relaciona con la búsqueda de un ritmo de trabajo sostenible.

En el gráfico de la cátedra, por ejemplo, la iteración 7 presenta una disminución muy marcada. Este resultado merece investigarse, pero **una caída aislada no demuestra por sí sola una pérdida de productividad o un problema de desempeño del equipo**.

2. CAPACIDAD (CAPACITY)

La **capacidad representa cuánto trabajo puede asumir un equipo durante un Sprint**, considerando los recursos y el tiempo disponibles. Se utiliza en la **Sprint Planning** para decidir cuánto trabajo es razonable incorporar al Sprint.

Puede expresarse de dos maneras:

- **Horas de trabajo disponibles:** se utilizan especialmente cuando el equipo recién comienza o todavía no tiene una velocidad estable. Se considera el tiempo efectivamente disponible para trabajar durante el Sprint, descontando ausencias y otras limitaciones
- **Story Points:** cuando el equipo ya tiene una velocidad histórica relativamente estable, esta sirve como referencia para estimar cuántos puntos de historia podrá completar en el siguiente Sprint

No debe establecerse una conversión fija entre Story Points y horas: los puntos representan una estimación relativa del trabajo, mientras que las horas representan tiempo disponible.

Diferencia con velocidad: la capacidad se determina antes del Sprint para planificar; la velocidad se conoce después, según el trabajo efectivamente completado y aceptado.

3. RUNNING TESTED FEATURES (RTF)

La métrica **Running Tested Features mide la cantidad de funcionalidades desarrolladas, probadas y que se encuentran funcionando** en cada momento del proyecto.

Permite observar el crecimiento del producto mediante funcionalidades efectivamente operativas, representadas generalmente en un gráfico de líneas a lo largo de las iteraciones.

Se espera que la cantidad de funcionalidades funcionando tienda a aumentar a medida que avanza el desarrollo.

- **Pendiente creciente:** aumenta la cantidad de funcionalidades probadas y funcionando.
- **Pendiente nula:** no se incorporan nuevas funcionalidades que cumplan esas condiciones.
- **Pendiente negativa:** disminuye la cantidad de funcionalidades consideradas probadas y funcionando; por ejemplo, porque alguna dejó de funcionar o dejó de cumplir los criterios establecidos. Debe investigarse la causa.

Limitación: RTF cuenta funcionalidades, pero no considera directamente su tamaño o complejidad. Dos equipos pueden tener el mismo número de funcionalidades funcionando y haber realizado trabajos de magnitud muy diferente.

Por esta razón, velocidad y RTF ofrecen perspectivas complementarias sobre el progreso del producto.

Las métricas de defectos también siguen siendo necesarias en los enfoques ágiles, aunque la selección de mediciones debe mantenerse orientada a la utilidad y al valor aportado.

MÉTRICAS EN LEAN/KANBAN

En Lean/Kanban, el foco de medición se concentra en **el desempeño del flujo de trabajo y la calidad del servicio entregado al cliente**.

A diferencia de Scrum, que organiza el trabajo mediante Sprints, Kanban puede gestionar servicios con una demanda continua, sin requerir proyectos o iteraciones de duración fija.

Por eso, interesa conocer cuánto tarda un elemento desde que se solicita hasta que se entrega, cuánto tiempo pasa en procesamiento y cuánto permanece esperando.

1. LEAD TIME (TIEMPO DE ENTREGA)

Es el **tiempo transcurrido desde que el cliente solicita un elemento de trabajo hasta que este es entregado**.

Representa la **perspectiva del cliente**, porque indica cuánto debe esperar para recibir lo solicitado. Incluye el tiempo previo al comienzo del trabajo, el procesamiento y las esperas posteriores hasta la entrega.

2. CYCLE TIME (TIEMPO DE CICLO)

Es el **tiempo transcurrido desde que el equipo comienza a trabajar sobre un elemento hasta que finaliza el proceso considerado**.

Representa la **perspectiva interna del equipo**, porque permite evaluar cuánto tarda el sistema en completar el trabajo una vez iniciado. A diferencia del Lead Time, excluye la espera previa al inicio del trabajo, por ejemplo, el período durante el cual la solicitud permanece en el Backlog. Sin embargo, **sí incluye las esperas y bloqueos que ocurren después de comenzar el trabajo**, hasta su finalización.

3. TOUCH TIME (TIEMPO DE TRABAJO EFECTIVO)

Es el **tiempo durante el cual el equipo trabaja efectivamente sobre un elemento**, excluyendo los períodos de espera, bloqueo o permanencia en colas.

Se relaciona con las columnas de producción del tablero Kanban, a diferencia de las columnas de acumulación. Sin embargo, un elemento ubicado en una columna de trabajo activo también puede estar temporalmente bloqueado, por lo que no todo ese tiempo necesariamente constituye Touch Time.

En condiciones equivalentes de medición, se cumple: **Touch Time ≤ Cycle Time ≤ Lead Time**. Esta relación muestra que el tiempo de trabajo efectivo no puede superar el tiempo total transcurrido desde que se comenzó a trabajar, ni este superar el tiempo total desde la solicitud.

4. EFICIENCIA DEL CICLO DE PROCESO

Mide **qué proporción del tiempo total transcurrido se destinó efectivamente a trabajar sobre el elemento**, permitiendo identificar pérdidas de tiempo por esperas, bloqueos y acumulaciones. La fórmula es:

$$\text{Eficiencia del ciclo de proceso (\%)} = \frac{\text{Touch Time}}{\text{Elapsed Time}} \times 100$$

Elapsed Time representa el tiempo total transcurrido en el intervalo considerado para calcular la eficiencia. En las notas de clase se lo equipara con Lead Time para el caso trabajado.

Cuanto mayor sea la proporción de Touch Time respecto del tiempo transcurrido, menor será el peso relativo de las esperas y bloqueos.

Una eficiencia elevada indica un mejor aprovechamiento temporal del flujo medido, **pero no garantiza por sí sola la calidad ni el valor del producto entregado**.

OTRAS MÉTRICAS DE FLUJO

- **Throughput (tasa de entrega):** mide la **cantidad de elementos de trabajo completados por unidad de tiempo**, por ejemplo, 15 historias de usuario terminadas por semana. Permite conocer el ritmo de entrega del sistema.
- **WIP actual (trabajo en progreso):** mide la **cantidad de elementos de trabajo que se encuentran dentro del sistema, o de una parte de él, en un momento determinado**, incluyendo los que están siendo procesados y los que esperan avanzar. Permite identificar acumulaciones y sobrecarga de trabajo.

El **WIP actual no debe confundirse con el límite WIP**: el primero indica cuánto trabajo hay en progreso, mientras que el segundo establece la cantidad máxima permitida.

MÉTRICAS ORIENTADAS AL SERVICIO

Las métricas de Kanban permiten conocer el desempeño real del servicio y utilizar esa información para gestionar las expectativas del cliente. Se distinguen cuatro conceptos:

- **Expectativa del nivel de servicio:** rendimiento o plazo que el cliente espera recibir.
- **Capacidad del nivel de servicio:** rendimiento que el sistema puede ofrecer según su comportamiento real.
- **Acuerdo de nivel de servicio (SLA):** compromiso establecido entre el proveedor y el cliente sobre el nivel de servicio que se brindará.
- **Umbral de adecuación del servicio:** límite por debajo del cual el servicio deja de resultar aceptable para el cliente.

Por ejemplo, en un servicio de soporte, el cliente puede esperar una resolución en dos días, mientras que las mediciones muestran que normalmente se requieren tres o cuatro. Con esta información se puede negociar un acuerdo realista y establecer qué demora se considera inaceptable.

El **Lead Time** es especialmente útil para estos análisis, porque representa directamente el tiempo que espera el cliente desde que solicita el trabajo hasta que lo recibe.

MÉTRICAS PARA EL PRODUCTO DE SOFTWARE

Las métricas de producto pueden aplicarse independientemente del enfoque de gestión, aunque la presentación las desarrolla especialmente en el contexto tradicional. Se agrupan principalmente en dos categorías:

1. MÉTRICAS DE TAMAÑO

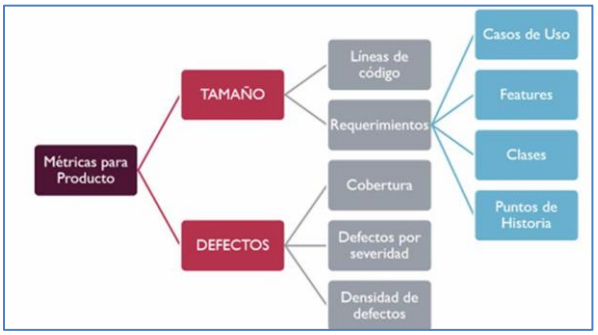
Permiten representar la dimensión del producto mediante unidades como:

- Líneas de código.
- Cantidad de requerimientos, casos de uso o funcionalidades.
- Cantidad de clases.
- Puntos de historia, cuando se utilizan como unidad relativa de estimación.

La elección depende del contexto. **Las líneas de código no deben utilizarse como indicador aislado de productividad o calidad**, porque una mayor cantidad de código no necesariamente representa más valor ni un mejor producto.

2. MÉTRICAS DE DEFECTOS

- **Cobertura:** Mide qué proporción de los elementos definidos fue cubierta por las pruebas, según el criterio utilizado (requerimientos, funcionalidades, código, etc.). Permite evaluar el alcance de las pruebas, pero **no indica directamente cuántos defectos existen ni garantiza su ausencia**.
- **Defectos por severidad:** cantidad de defectos clasificados según su impacto, por ejemplo, críticos, mayores o menores.
- **Densidad de defectos:** cantidad de defectos en relación con una unidad de tamaño del producto, como líneas de código, funcionalidades o historias de usuario, según la definición de la medición.



COMPARACIÓN DE MÉTRICAS EN LOS TRES ENFOQUES

Enfoque	Foco principal	Métricas representativas
Tradicional	Control del proyecto, desempeño de procesos y características del producto.	Tamaño, esfuerzo, tiempo, costos y defectos.
Agile/Scrum	Software funcionando, progreso del incremento y capacidad del equipo.	Velocidad, capacidad y Running Tested Features.
Lean/Kanban	Eficiencia del flujo de trabajo y servicio entregado al cliente.	Lead Time, Cycle Time, Touch Time y eficiencia del ciclo de proceso.

La diferencia fundamental está en el propósito de la medición, no en que cada enfoque tenga prohibido utilizar métricas de los demás. En todos los enfoques se mantiene el mismo criterio: **medir para comprender, tomar decisiones y mejorar, no para juzgar personas**.
En gestión tradicional se enfatiza la comparación entre lo planificado y lo realizado. En Agile se prioriza conocer cuánto producto funcionando se completa. En Lean/Kanban se busca comprender cómo fluye el trabajo y cuánto tarda en llegar al cliente.

4. SCM EN AMBIENTES ÁGILES Y PRÁCTICAS CONTINUAS

GESTIÓN DE CONFIGURACIÓN EN AMBIENTES ÁGILES

La **Gestión de Configuración de Software (SCM)** busca mantener la integridad y trazabilidad del producto a lo largo de su evolución, controlando sus versiones, componentes y modificaciones.

En los ambientes ágiles, SCM mantiene estos objetivos, pero **se orienta a facilitar el trabajo del equipo y responder a los cambios**, en lugar de centrarse principalmente en controles formales sobre los desarrolladores. Esto se relaciona con los principios ágiles de adaptación al cambio, colaboración, entrega frecuente y responsabilidad compartida.

CARACTERÍSTICAS

1. **Respuesta al cambio:** Permite incorporar modificaciones en los requerimientos y mantener la integridad del producto.
2. **Automatización:** Se automatizan actividades cuando resulta conveniente, especialmente la integración, construcción y verificación del software.
3. **Retroalimentación continua:** Facilita la detección temprana de problemas relacionados con versiones, integración y estabilidad.
4. **Integración con el desarrollo:** Las actividades de SCM se incorporan al trabajo cotidiano del equipo, en lugar de tratarse como tareas completamente separadas.

5. **Responsabilidad compartida:** Todo el equipo participa en el mantenimiento de la configuración del producto, por lo que necesita conocer las prácticas y herramientas utilizadas.
6. **Adopción de prácticas continuas:** SCM proporciona una base para integrar, verificar y entregar cambios frecuentes de manera controlada.

En comparación con enfoques tradicionales, los equipos ágiles suelen simplificar los mecanismos formales de aprobación y control de cambios. **Esto no significa eliminar toda auditoría o control**, sino adaptarlos al contexto, manteniendo la trazabilidad y los criterios de calidad necesarios.

PRÁCTICAS CONTINUAS

Las prácticas continuas buscan **reducir el tiempo entre la modificación del software, su verificación y su disponibilidad para los usuarios**, mediante integraciones frecuentes, automatización y retroalimentación temprana.

Se distinguen tres prácticas relacionadas: **Continuous Integration, Continuous Delivery y Continuous Deployment**.

1. CONTINUOUS INTEGRATION (CI) — INTEGRACIÓN CONTINUA

Es una práctica de desarrollo que consiste en **integrar frecuentemente los cambios de código en un repositorio compartido**, verificando cada integración mediante procesos automatizados.

Generalmente, cada integración activa un *pipeline* que realiza actividades como:

- Construir o compilar el software (*build*).
- Ejecutar pruebas automatizadas, especialmente unitarias y de integración.
- Informar rápidamente si se detectan errores.

Su objetivo es detectar y corregir tempranamente los problemas de integración, evitando que se acumulen cambios incompatibles que luego resulten difíciles de combinar.

Por ejemplo, si dos desarrolladores modifican componentes relacionados, la integración frecuente permite identificar rápidamente si sus cambios provocan fallas al trabajar juntos.

CI requiere mantener el código integrado y verificable, pero **no garantiza por sí sola que el producto esté listo para desplegarse en producción**. Puede apoyarse en prácticas como TDD, aunque estas no son obligatorias para realizar integración continua.

2. CONTINUOUS DELIVERY — ENTREGA CONTINUA

Es una práctica que amplía la integración continua y busca **mantener el software en condiciones de ser desplegado en producción cuando el negocio lo decida**.

Para ello, el producto atraviesa un proceso de construcción y verificación que puede incluir pruebas automatizadas de integración, regresión y aceptación, además de comprobaciones en ambientes similares al productivo.

El objetivo es disponer de una **versión comprobada y desplegable**, reduciendo el esfuerzo, el tiempo y los riesgos asociados a una liberación.

La característica fundamental es que **el despliegue a producción requiere una decisión o autorización**, que puede provenir del responsable de negocio, el Product Owner u otra autoridad definida por la organización.

Por lo tanto, Continuous Delivery **no significa desplegar automáticamente cada modificación**. Significa estar preparado para hacerlo de manera rápida y confiable cuando se tome esa decisión.

3. CONTINUOUS DEPLOYMENT — DESPLIEGUE CONTINUO

Es una práctica que lleva la automatización un paso más allá: **los cambios que superan satisfactoriamente todas las verificaciones del pipeline se despliegan automáticamente en producción**, sin requerir una autorización manual para cada despliegue.

Se apoya en CI y Continuous Delivery, incorporando la automatización de la publicación en el ambiente productivo. Para aplicarla se necesitan controles automatizados confiables, porque las modificaciones pueden llegar directamente a los usuarios.

Su principal beneficio es **reducir el tiempo entre desarrollar un cambio y ponerlo a disposición del usuario**, favoreciendo entregas pequeñas y frecuentes y una retroalimentación más rápida.

RELACIÓN ENTRE LAS TRES PRÁCTICAS

Práctica	Objetivo principal	Relación con producción
Continuous Integration	Integrar y verificar frecuentemente los cambios.	No implica desplegar en producción.
Continuous Delivery	Mantener el producto verificado y listo para desplegarse.	Se despliega cuando existe una decisión o autorización.
Continuous Deployment	Automatizar también la publicación de los cambios verificados.	Se despliega automáticamente al superar los controles establecidos.

Las tres prácticas se relacionan progresivamente: **CI permite integrar y verificar los cambios; Delivery mantiene una versión lista para producción; Deployment automatiza también su publicación**.

La diferencia fundamental no es únicamente la cantidad de pruebas automatizadas, sino **hasta qué etapa llega la automatización y cómo se decide la puesta en producción**.

Estas prácticas contribuyen a reducir el retrabajo y los riesgos de integración, obtener retroalimentación temprana y facilitar entregas frecuentes de software.

ESTRATEGIAS DE DESPLIEGUE

Las estrategias de despliegue establecen **cómo introducir una nueva versión en producción**, buscando disminuir el impacto de posibles fallas y facilitar la recuperación de una versión estable. Son estrategias que pueden utilizarse dentro de procesos de entrega o despliegue automatizados.

CANARY DEPLOYMENT

Consiste en **publicar inicialmente la nueva versión para una pequeña parte de los usuarios o del tráfico**, mientras que el resto continúa utilizando la versión anterior. Se observa su comportamiento en condiciones reales y, si los resultados son satisfactorios, se amplía progresivamente su disponibilidad.

Si aparecen problemas, puede detenerse la expansión o revertirse el cambio. **Su principal ventaja es limitar inicialmente el alcance de una falla**, evitando exponer a todos los usuarios al mismo tiempo.

BLUE/GREEN DEPLOYMENT

Consiste en mantener **dos ambientes de producción equivalentes**, que ejecutan versiones diferentes del software:

- **Blue:** ambiente activo, donde los usuarios utilizan la versión actual.
- **Green:** ambiente equivalente donde se despliega la nueva versión sin afectar inicialmente a los usuarios de Blue.

Una vez que se comprueba que Green funciona correctamente, **se redirige el tráfico de los usuarios desde Blue hacia Green**. Si aparecen problemas, puede revertirse el cambio volviendo a Blue, siempre que el ambiente anterior permanezca disponible y sea compatible.

Su principal ventaja es facilitar el cambio entre versiones y una posible reversión, reduciendo las interrupciones asociadas al despliegue.

DIFERENCIA PRINCIPAL

Canary se concentra en **exponer gradualmente a los usuarios a una nueva versión** y evaluar su comportamiento real.

Blue/Green se concentra en **mantener dos ambientes equivalentes y cambiar el tráfico entre versiones**, facilitando la sustitución y la reversión del despliegue.

Ambas estrategias buscan disminuir los riesgos de las actualizaciones, aunque no garantizan por sí solas la ausencia de fallas o interrupciones.

5. CONCEPTOS GENERALES SOBRE CALIDAD

La **calidad de software es el grado en que un producto satisface los requerimientos explícitos y las necesidades y expectativas implícitas de sus interesados**. Incluye el cumplimiento de los requisitos funcionales, de rendimiento, de los estándares establecidos y de otras características esperadas de un producto profesional.

La calidad tiene también un **componente subjetivo**, porque su percepción depende de las personas que evalúan el producto, sus necesidades, las circunstancias, el contexto y el momento en que se realiza la evaluación.

Por ejemplo, un sistema puede cumplir todos los requerimientos funcionales especificados y, aun así, generar insatisfacción porque resulta difícil de utilizar o su rendimiento es insuficiente.

Por eso, para evaluar la calidad no alcanza con comprobar si el software hace lo solicitado: también deben considerarse las expectativas de quienes lo utilizarán y las condiciones en las que deberá funcionar.

IMPORTANCIA DE TRABAJAR PARA Y CON CALIDAD

Durante el desarrollo de software pueden presentarse problemas como:

- Atrasos en las entregas y costos excedidos.
- Incumplimiento de compromisos.
- Requerimientos poco claros o incompletos.
- Software que no cumple con lo esperado.
- Trabajo fuera de horario y retrabajo.
- Problemas para localizar componentes o conocer sus versiones cuando no existe una adecuada SCM.
- **Fenómeno del 90-90:** dificultad para conocer el avance real, cuando un trabajo parece estar casi terminado, pero todavía requiere un esfuerzo considerable.

Estos problemas pueden afectar tanto al producto como a la gestión del proyecto. Por eso, **obtener un producto de calidad no depende únicamente de sus características técnicas, sino también de cómo se organiza, controla y ejecuta el trabajo que permite construirlo**.

El proyecto es el medio mediante el cual se construye el producto. Por eso, si su planificación, ejecución y control son deficientes, pueden producirse retrabajos, incumplimientos y dificultades que comprometan la calidad del software entregado.

En consecuencia, **no basta con evaluar la calidad del producto final: también es necesario gestionar adecuadamente los procesos y el proyecto que permiten construirlo**.

VENTAJAS DE TRABAJAR CON CALIDAD

1. **Prevención de defectos y reducción del retrabajo:** Detectar y evitar problemas tempranamente disminuye el esfuerzo necesario para corregirlos.
2. **Reducción de costos:** Menos defectos y correcciones pueden disminuir los costos totales del desarrollo y mantenimiento.
3. **Mayor satisfacción y retención de clientes:** Aumenta la posibilidad de cumplir sus necesidades y expectativas.
4. **Mejora continua:** Permite identificar problemas recurrentes y optimizar la forma de trabajar.
5. **Mayor competitividad:** Trabajar con estándares y procesos de calidad puede favorecer la confianza de los clientes y el acceso a determinados mercados.

COSTOS Y DIFICULTADES (DESVENTAJAS) DE TRABAJAR CON CALIDAD

Trabajar con calidad **requiere invertir recursos** en capacitación, definición de procesos, planificación, controles, revisiones y mediciones. Además, pueden existir dificultades para incorporar nuevas prácticas, especialmente cuando implican cambios culturales o generan resistencia en los equipos.

Por eso, **las actividades de calidad deben guardar una relación razonable entre su costo y sus beneficios**. Incorporar controles excesivos o innecesarios puede generar burocracia sin mejorar el producto. La idea no es evitar los costos de calidad, sino realizar una inversión adecuada para prevenir problemas cuyo costo de corrección podría ser mayor.

CALIDAD EN EL DESARROLLO DE SOFTWARE

Para considerar que un producto de software tiene calidad deben contemplarse las necesidades y expectativas de distintos interesados:

- **Cliente:** espera que el producto cumpla los objetivos acordados.
- **Usuario:** espera que el sistema resulte útil, confiable y adecuado para sus actividades.
- **Gerencia:** necesita que el desarrollo contribuya a los objetivos de la organización.
- **Equipo desarrollo y mantenimiento:** requiere que el software pueda comprenderse, modificarse y mantenerse.
- **Otros interesados:** pueden establecer necesidades particulares, como requisitos de seguridad, operación o integración.

Por lo tanto, **la calidad debe considerarse durante todo el desarrollo**, desde la definición de los requerimientos hasta la construcción, evaluación y mantenimiento del producto.

PRINCIPIOS DE CALIDAD

1. **La calidad debe estar embebida:** no puede incorporarse únicamente al final mediante pruebas o correcciones. Debe considerarse desde los requerimientos y durante todo el desarrollo.
2. **Es responsabilidad de todos:** cada integrante del proyecto contribuye a la calidad mediante su trabajo, no solamente el equipo de QA o Testing.
3. **Las personas son fundamentales:** el desarrollo de software depende intensivamente de las capacidades humanas. La capacitación, los conocimientos y la motivación resultan esenciales.
4. **Se necesita compromiso gerencial:** la dirección debe apoyar las iniciativas de calidad y proporcionar recursos, aunque las mejoras pueden comenzar desde equipos individuales.
5. **Se debe liderar con el ejemplo:** quienes conducen los equipos deben demostrar mediante sus acciones el compromiso con las prácticas de calidad.
6. **No se puede controlar lo que no se mide:** las métricas permiten conocer resultados, identificar problemas y tomar decisiones para mejorar.
7. **Simplicidad:** conviene comenzar con prácticas básicas, útiles y sostenibles, evitando incorporar complejidad innecesaria.
8. **El aseguramiento de calidad debe planificarse:** deben establecerse anticipadamente los objetivos, criterios, actividades y responsabilidades relacionados con la calidad.
9. **Aumentar las pruebas no garantiza aumentar la calidad:** Testing permite evaluar el producto y descubrir defectos, pero no reemplaza la prevención y las buenas prácticas aplicadas durante el desarrollo.
10. **La calidad debe ser razonable para el negocio:** las prácticas adoptadas deben responder a las necesidades del producto y justificar sus costos.

Estos principios se relacionan con una idea central: **prevenir problemas es preferible a corregirlos tardíamente**, porque los defectos pueden generar retrabajo, mayores costos y retrasos.

CALIDAD PARA QUIÉN — CINCO VISIONES DE CALIDAD

La calidad puede analizarse desde diferentes perspectivas, según qué se considere más importante para evaluar un producto.

1. Visión del usuario: la calidad depende de la capacidad del producto para satisfacer las necesidades y expectativas de quienes lo utilizan. Es importante mantener una comunicación frecuente para comprender qué consideran valioso.

2. Visión del producto: la calidad se relaciona con las características y atributos del software, como funcionalidad, fiabilidad, rendimiento o mantenibilidad, que pueden evaluarse mediante criterios definidos.

3. Visión de manufactura o proceso: la calidad se asocia al cumplimiento de los procesos, procedimientos y estándares establecidos para construir el producto. Se busca que el trabajo se realice de forma controlada y consistente.

4. Visión basada en el valor: considera la relación entre calidad, costo y beneficios obtenidos. El producto debe proporcionar un valor adecuado en relación con los recursos necesarios para producirlo y adquirirlo.

5. Visión trascendental: entiende la calidad como una forma de excelencia que se reconoce, aunque resulte difícil definirla o medirla completamente. Puede asociarse con ideales que orientan la mejora continua.

Estas perspectivas son complementarias. Un software puede cumplir los procedimientos establecidos y presentar buenas características técnicas, pero no satisfacer al usuario. Por eso, **ninguna visión por sí sola permite evaluar completamente la calidad del producto**.

CALIDAD DEL PRODUCTO, DEL PROCESO Y DEL PROYECTO

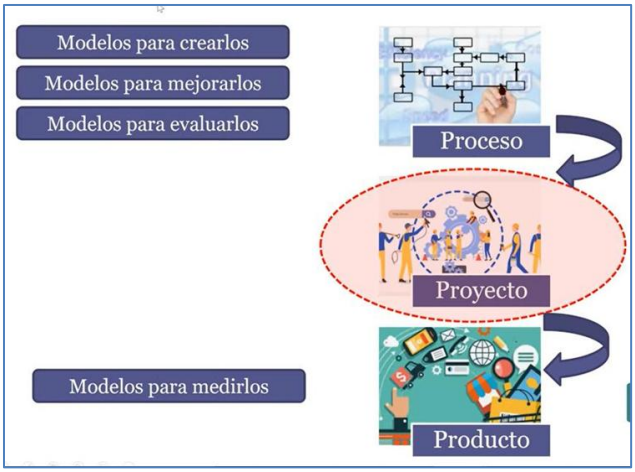
Aunque están relacionadas, es importante distinguirlas:

- **Calidad del producto:** se refiere a las características del software construido y a su adecuación a los requerimientos y necesidades.
- **Calidad del proceso:** se relaciona con la forma en que se desarrolla y mantiene el software, incluyendo actividades, procedimientos, métodos y estándares.
- **Calidad de la gestión del proyecto:** se vincula con la planificación, los recursos, los compromisos, los plazos y el control del trabajo necesario para obtener el producto.

Un proceso es la forma definida de realizar el trabajo; el proyecto aplica ese proceso en un contexto concreto; y el producto es el resultado que se obtiene.

Por eso, aunque el objetivo final es obtener software de calidad, también es necesario trabajar sobre el proceso que permite construirlo.

La cátedra destaca al **proceso como un factor fundamental sobre el que la organización puede intervenir para mejorar la calidad**, porque puede definirlo, evaluarlo y modificarlo. Sin embargo, no existe un único proceso adecuado para todas las organizaciones y proyectos.



Las organizaciones pueden utilizar diferentes tipos de modelos:

- **Modelos para definir procesos:** sirven como referencia para establecer cómo debe realizarse el trabajo.
- **Modelos para mejorar procesos:** permiten identificar oportunidades de mejora e introducir cambios.
- **Modelos para evaluar procesos:** permiten comparar el funcionamiento real con criterios o modelos de referencia.
- **Modelos para medir productos:** ayudan a evaluar las características y atributos del software construido.

Los procesos se aplican mediante proyectos concretos, que generan productos de trabajo. Durante esa ejecución deben incorporarse actividades de evaluación y aseguramiento de calidad.

GESTIÓN/ADMINISTRACIÓN DE LA CALIDAD DEL SOFTWARE

La **gestión o administración de la calidad** busca asegurar que el software alcance los niveles de calidad requeridos mediante la definición y aplicación de procesos, estándares y controles adecuados.

Actúa en distintos niveles:

- **Organización:** establece políticas, procesos y estándares generales de calidad.
- **Proceso:** verifica y mejora la forma en que se ejecutan las actividades de desarrollo.
- **Proyecto:** define y aplica un plan de calidad adecuado a sus características y necesidades.

La presentación oficial distingue tres actividades principales de la administración de calidad:

1. ASEGURAMIENTO DE CALIDAD (QA)

Consiste en **establecer los procesos, estándares y procedimientos que deben orientar el desarrollo hacia la obtención de productos de calidad**. Tiene una orientación principalmente preventiva: busca reducir la posibilidad de introducir defectos y asegurar que existan prácticas adecuadas durante el desarrollo.

El **Grupo de Aseguramiento de Calidad (GAC)** debe contar con suficiente independencia respecto de la gestión del proyecto para evaluar objetivamente el cumplimiento de los criterios establecidos. Esto no significa que la calidad sea responsabilidad exclusiva de ese grupo.

2. PLANIFICACIÓN DE LA CALIDAD

Consiste en **seleccionar los procedimientos y estándares aplicables a un proyecto y establecer cómo se evaluará la calidad del software**. Su principal resultado es el **Plan de Calidad**, que identifica los atributos de calidad más importantes, los criterios para evaluarlos y las responsabilidades correspondientes.

Puede incluir información general del proyecto, características del producto, procesos aplicables, fechas y responsabilidades, objetivos de calidad y riesgos relevantes.

3. CONTROL DE CALIDAD (QC)

Consiste en **evaluar el cumplimiento de los procedimientos, estándares y criterios de calidad establecidos**. Puede orientarse tanto al proceso como al producto:

- **Control del proceso:** comprueba que el trabajo se realice según los procedimientos y estándares acordados.
- **Control del producto:** evalúa los productos de trabajo para detectar defectos o comprobar características de calidad.

Para realizar estas evaluaciones se utilizan actividades como revisiones, auditorías, pruebas y mediciones, según el objeto y propósito del control.

Relación entre las tres actividades: el aseguramiento establece el marco de trabajo, la planificación lo adapta al proyecto y el control comprueba su cumplimiento.

La administración de calidad también debe promover una **cultura de calidad**, donde todos los integrantes asuman responsabilidad por su trabajo y participen en la mejora continua.

CALIDAD DEL PRODUCTO: PROGRAMADA, REALIZADA Y NECESARIA

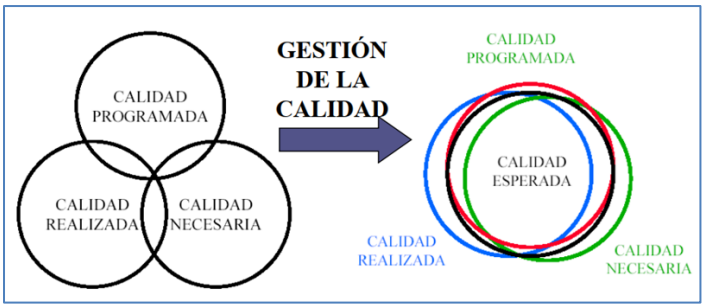
La gestión de calidad del producto requiere buscar un equilibrio entre tres dimensiones:

- **Calidad programada:** características y requerimientos del producto que fueron planificados o especificados.
- **Calidad realizada:** características que efectivamente fueron implementadas en el producto.
- **Calidad necesaria:** características que el producto necesita para satisfacer los requerimientos y expectativas reales de sus interesados.

El objetivo es lograr la **mayor coincidencia posible entre lo programado, lo realizado y lo necesario**, obteniendo un producto que responda a las expectativas de calidad.

Las diferencias entre estas dimensiones pueden representar problemas:

- Si algo es necesario, pero no fue programado o realizado, puede generar insatisfacción.
- Si algo fue programado, pero no realizado, existe un incumplimiento respecto de lo previsto.
- Si se realiza algo que no resulta necesario ni aporta valor, puede constituir desperdicio.



El diagrama representa cómo la gestión de calidad busca aproximar estas tres dimensiones para reducir los desajustes, el desperdicio y la insatisfacción del cliente.

MODELOS DE CALIDAD DEL PRODUCTO

La calidad de un producto de software depende de sus requerimientos y de las necesidades de sus usuarios. Por eso, **no existe un modelo único que permita certificar un nivel absoluto de calidad para cualquier software**.

Sin embargo, existen modelos de referencia que permiten identificar, organizar y evaluar atributos de calidad del producto, especialmente aquellos relacionados con los requerimientos no funcionales.

1. FAMILIA ISO/IEC 25000 — MODELOS DE CALIDAD

La familia de normas **ISO/IEC 25000** proporciona modelos y criterios para evaluar diferentes aspectos de la calidad del software. Se distinguen tres perspectivas:

- **Calidad del producto:** características propias del software, como adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad.
- **Calidad en uso:** calidad experimentada por los usuarios cuando utilizan el producto en un contexto determinado.
- **Calidad de datos:** evalúa si los datos utilizados o gestionados por el sistema son correctos, completos, consistentes y adecuados para su utilización.

La norma **ISO/IEC 25010** se relaciona con los modelos de calidad del producto y calidad en uso, mientras que **ISO/IEC 25012** aborda la calidad de los datos. Estos modelos ayudan a definir qué atributos evaluar y con qué criterios, pero no garantizan por sí mismos que un producto tenga calidad.

2. MODELO DE BARBACCI / SEI

Este modelo plantea la necesidad de **buscar un equilibrio entre diferentes atributos de calidad**, como rendimiento, confiabilidad y seguridad. Optimizar un atributo de manera aislada puede perjudicar otros. Por ejemplo, incorporar controles de seguridad adicionales puede afectar el tiempo de respuesta del sistema. Por eso, se distingue entre:

- **Óptimo local:** Optimizar un único atributo de calidad.
- **Óptimo global:** Buscar la mejor combinación posible entre varios atributos, considerando las necesidades del producto.

La idea fundamental es que **la calidad debe evaluarse considerando el conjunto de atributos relevantes**, no solamente el desempeño de uno de ellos.

3. MODELO DE CALIDAD DE SOFTWARE DE MCCALL

El modelo McCall organiza los factores de calidad del software en tres grandes perspectivas:

- **Operación del producto:** Cómo funciona el software durante su utilización. Incluye corrección, fiabilidad, usabilidad, integridad y eficiencia.
- **Revisión del producto:** Facilidad para modificar, mantener y probar el software. Incluye mantenibilidad, flexibilidad y facilidad de prueba.
- **Transición del producto:** Facilidad para adaptar o trasladar el software a otros entornos y aprovechar sus componentes. Incluye portabilidad, reutilización e interoperabilidad.

Este modelo permite analizar la calidad no solo durante el funcionamiento del producto, sino también considerando su mantenimiento y evolución futura.

CALIDAD DEL PROCESO DE SOFTWARE

La **calidad del proceso** se refiere a cómo se organiza y ejecuta el trabajo necesario para desarrollar y mantener el software. Aunque el objetivo final es obtener un producto de calidad, **mejorar el proceso permite aumentar las posibilidades de conseguirlo**, reduciendo problemas recurrentes, errores y retrabajo.

No existe un proceso único adecuado para todas las organizaciones y proyectos. Cada organización debe definir y adaptar su forma de trabajar según sus necesidades y contexto.

IMPORTANCIA DEL PROCESO

Se destaca que **el proceso es el principal factor sobre el que la organización puede intervenir sistemáticamente para mejorar la calidad**. Esto se debe a que existen otros factores que influyen sobre el producto, como las personas, las tecnologías disponibles, las necesidades del cliente y las condiciones del negocio, cuyo comportamiento no puede controlarse completamente.

Por eso, se busca establecer procesos que permitan trabajar de manera consistente, evaluar los resultados e introducir mejoras. Sin embargo, **tener un proceso definido y de calidad no garantiza automáticamente un producto de calidad**: también es necesario comprobar que el producto satisfaga sus requerimientos y las necesidades del cliente.

DEFINICIÓN DE PROCESOS

Un **proceso de software es un conjunto organizado de actividades, métodos y prácticas utilizadas para desarrollar o mantener software y sus productos asociados**. Para trabajar de manera controlada, es necesario definir y comunicar explícitamente el proceso a todos los involucrados. Esta definición debe establecer:

- **Qué** actividades y tareas se realizarán.
- **Quiénes** participarán y cuáles serán sus responsabilidades.
- **Cuándo** deben ejecutarse las actividades y en qué orden.
- **Cómo** deben realizarse, incluyendo los procedimientos, métodos y herramientas correspondientes.

El proceso debe ser conocido por el equipo y utilizarse como referencia para desarrollar los proyectos.

ACTIVIDADES TÉCNICAS Y TRANSVERSALES

Un proceso de desarrollo incluye actividades técnicas, como Ingeniería de Requerimientos, Análisis y Diseño, Implementación, Pruebas y Despliegue.

Además, se incorporan **disciplinas transversales** que acompañan el desarrollo durante sus distintas etapas:

- **Planificación y seguimiento del proyecto:** Organizar y controlar el trabajo, los recursos y los compromisos.
- **Gestión de Configuración de Software (SCM):** Controlar y mantener la integridad de los productos de trabajo y sus versiones.
- **Aseguramiento de Calidad (QA):** Evaluar el cumplimiento de los procesos y estándares establecidos e identificar oportunidades de mejora.

Estas disciplinas son necesarias independientemente del enfoque de gestión adoptado, aunque su implementación puede variar.

ADAPTACIÓN Y MEJORA DEL PROCESO

El proceso organizacional constituye una referencia que debe **adaptarse a las características de cada proyecto**, respetando las actividades y controles de calidad que resulten necesarios.

La adaptación no significa eliminar arbitrariamente actividades esenciales, como las pruebas, sino ajustar la forma de realizarlas según el contexto. Además, un proceso no debe mantenerse sin cambios solamente porque fue definido previamente. La organización necesita evaluar su funcionamiento y mejorarlo continuamente.

Los procesos basados en calidad siguen un ciclo de evaluación y mejora:

1. **Definir el proceso** que se utilizará.
2. **Desarrollar el producto** aplicando ese proceso.
3. **Evaluar la calidad del producto** y los resultados obtenidos.
4. **Si la calidad es satisfactoria:** Estandarizar el proceso para aprovecharlo en futuros proyectos.
5. **Si la calidad no es satisfactoria:** Mejorar el proceso y volver a aplicarlo, evaluando nuevamente sus resultados.

De esta manera, **los resultados de los productos permiten retroalimentar y mejorar los procesos utilizados para construirlos**.

ESTÁNDARES DE CALIDAD DE SOFTWARE

Los **estándares establecen criterios y reglas de referencia** que permiten organizar el trabajo y evaluar su cumplimiento. Pueden ser internacionales, nacionales, organizacionales o particulares de un proyecto.

Se distinguen dos categorías principales:

- **Estándares de producto:** Establecen las características que deben cumplir determinados productos de trabajo. Por ejemplo, convenciones de programación, estructura de la documentación o formatos de especificación.
- **Estándares de proceso:** Establecen cómo deben ejecutarse determinadas actividades del desarrollo. Por ejemplo, procedimientos para realizar revisiones técnicas, administrar configuraciones o validar requerimientos.

Los estándares permiten trabajar con criterios comunes y facilitan la evaluación de la calidad. Sin embargo, **sus prácticas deben ser adecuadas al contexto de cada proyecto**, evitando aplicar procedimientos innecesarios o excesivos únicamente porque están establecidos. Por eso, deben revisarse y adaptarse cuando corresponda, respetando los requisitos obligatorios.

PROCESOS DEFINIDOS Y PROCESOS EMPÍRICOS

Ambos enfoques buscan desarrollar productos de calidad, pero presentan diferencias en la manera de organizar, controlar y mejorar el trabajo.

PROCESOS DEFINIDOS

Se basan en **establecer previamente las actividades, procedimientos y criterios que orientarán el desarrollo**.

Se busca lograr resultados consistentes mediante **procesos conocidos, documentados y controlados**. La mejora continua se realiza evaluando el cumplimiento y desempeño del proceso e introduciendo cambios cuando corresponda.

Este enfoque es especialmente relevante para los modelos de mejora y evaluación de procesos, como CMMI.

PROCESOS EMPÍRICOS

Se basan en **aprender de la experiencia y ajustar el trabajo mediante inspección y adaptación frecuentes**.

En lugar de intentar definir anticipadamente todos los detalles, se utilizan **ciclos de retroalimentación para realizar la inspección y adaptación**. Para ello, se desarrollan **incrementos** de producto que permiten evaluar los resultados, se realizan **revisiones** para adaptar el producto y **retrospectivas** para mejorar la forma de trabajar del equipo.

Esto no significa que los procesos empíricos carezcan de reglas, estándares o actividades de calidad. La principal diferencia está en la forma de gestionar la incertidumbre y adaptar el proceso.

RELACIÓN CON LA CALIDAD

Los enfoques definidos enfatizan la conformidad con procesos establecidos y su mejora sistemática. Los empíricos ponen mayor énfasis en la inspección frecuente, la adaptación y la responsabilidad del equipo.

La cátedra también plantea una diferencia en la utilización de auditorías: los enfoques definidos suelen recurrir a evaluaciones formales de cumplimiento, mientras que los ágiles utilizan mecanismos internos de inspección y mejora, como las retrospectivas.

Sin embargo, **una retrospectiva no equivale a una auditoría independiente**, y un equipo ágil también puede incorporar auditorías cuando sean necesarias. Ambos enfoques pueden combinar prácticas para obtener mejores resultados.

ACTIVIDADES RELACIONADAS CON EL PPQA

SQA (Software Quality Assurance) se refiere al aseguramiento de calidad del software en general. **PPQA (Process and Product Quality Assurance)** es un área de proceso de CMMI orientada a evaluar objetivamente que los procesos y productos de trabajo cumplan los procedimientos y estándares establecidos, y a comunicar y realizar el seguimiento de las desviaciones detectadas.

Como vimos anteriormente, la administración de calidad comprende **Aseguramiento, Planificación y Control de Calidad**.

El aseguramiento establece los estándares y procedimientos; la planificación determina cuáles se utilizarán en un proyecto y cómo se evaluará la calidad; y el control verifica su cumplimiento.

OBJETIVOS DEL ASEGURAMIENTO DE CALIDAD (SQA/PPQA)

El Aseguramiento de Calidad de Software busca:

1. **Realizar controles apropiados** del software y de su proceso de desarrollo.
2. **Verificar el cumplimiento** de los estándares y procedimientos establecidos para el producto y el proceso.
3. **Informar las desviaciones y defectos detectados** a los responsables correspondientes, para que puedan tomar medidas correctivas.

Su finalidad es brindar confianza en que el desarrollo se realiza bajo criterios de calidad y que los problemas identificados serán tratados adecuadamente.

FUNCIONES DEL ASEGURAMIENTO DE CALIDAD

1. **Establecer prácticas de calidad:** Desarrollar y poner a disposición herramientas, técnicas, métodos y estándares adecuados para realizar evaluaciones y revisiones.
2. **Evaluar la planificación del proyecto:** Comprobar que los planes, procedimientos y controles previstos sean apropiados.
3. **Evaluar los requerimientos y el diseño:** Revisar la calidad de estas actividades y su cumplimiento de los criterios establecidos.
4. **Evaluar las prácticas de programación:** comprobar que el desarrollo se ajuste a las prácticas y estándares acordados.
5. **Evaluar la integración y las pruebas:** verificar que estas actividades se realicen conforme a los procedimientos definidos.
6. **Evaluar la planificación y el control de proyectos:** comprobar cómo se gestionan y supervisan las actividades.
7. **Adaptar los procedimientos de QA:** adecuar las prácticas de calidad a las características de cada proyecto.

Estas evaluaciones permiten identificar incumplimientos, registrar los hallazgos y promover acciones correctivas o mejoras sobre el proceso.

GRUPO DE ASEGURAMIENTO DE CALIDAD (GAC)

El **Grupo de Aseguramiento de Calidad (GAC)** es responsable de realizar o coordinar actividades de evaluación y seguimiento del cumplimiento de los criterios de calidad.

Para desarrollar estas tareas de manera objetiva, necesita contar con **independencia respecto de la gestión del proyecto que evalúa**. La presentación oficial establece que:

- El GAC **no debería depender directamente del gerente de proyectos**, para preservar su independencia.
- **No debería existir más de una posición jerárquica entre la gerencia de primer nivel y el GAC**, facilitando la comunicación de los problemas detectados.
- Cuando sea posible, debería reportar a una autoridad realmente comprometida con la calidad del software

Esta independencia permite que el grupo informe desviaciones y problemas sin quedar condicionado por los intereses particulares de un proyecto.

Sin embargo, **la existencia del GAC no elimina la responsabilidad de calidad del resto del equipo**, ya que esta debe ser compartida por todos los involucrados.

ASEGURAMIENTO Y CONTROL DE CALIDAD EN LA PRÁCTICA

Para asegurar la calidad del proceso se deben definir estándares, monitorear su cumplimiento, evaluar los resultados y reportar los problemas detectados.

Existen dos mecanismos importantes de control:

- **Revisiones de calidad:** Examinar productos de trabajo, procesos o documentación para identificar defectos, desviaciones y oportunidades de mejora. Incluyen inspecciones para remover defectos, revisiones de progreso y revisiones de cumplimiento de calidad.
- **Evaluaciones automáticas y mediciones:** Utilizar herramientas e indicadores para comprobar características del software o el cumplimiento de determinados criterios.

Estas actividades se complementan con otras técnicas de aseguramiento y control, como las auditorías y el Testing, que desarrollaremos en sus secciones correspondientes.

El objetivo no es realizar controles por obligación, sino utilizar sus resultados para prevenir problemas, corregir desviaciones y mejorar continuamente la calidad del proceso y del producto.

6. PRINCIPALES MODELOS DE CALIDAD Y MEJORA DE PROCESOS

MODELOS PARA LA MEJORA DE PROCESOS

La **mejora continua de procesos** consiste en comprender cómo trabaja actualmente una organización e introducir cambios para aumentar la calidad de sus productos, reducir costos, disminuir tiempos de desarrollo y mejorar su desempeño.

Para ello, se utilizan modelos que cumplen funciones diferentes:

- **Modelos de mejora:** Orientan cómo organizar y llevar adelante las iniciativas de mejora de procesos.
- **Modelos de referencia y evaluación:** Establecen objetivos y buenas prácticas contra los cuales pueden evaluarse los procesos de una organización, permitiendo identificar fortalezas y oportunidades de mejora.

El propósito es desarrollar **proyectos de mejora cuyo resultado es un proceso organizacional mejorado**, que luego pueda aplicarse y adaptarse a los proyectos de desarrollo de software.

Es importante distinguir que estos modelos no necesariamente indican paso a paso cómo debe trabajar una organización. En particular, los modelos de referencia, como CMMI, establecen los objetivos y prácticas que deben considerarse, pero permiten elegir cómo implementarlos según el contexto.

MODELO IDEAL – MODELO DE MEJORA

IDEAL (Initiating, Diagnosing, Establishing, Acting, Learning) es un modelo cíclico que sirve como guía para **organizar e implementar proyectos de mejora de procesos** dentro de una organización.

Su finalidad es partir de una situación actual, identificar qué se necesita mejorar, implementar cambios y aprovechar lo aprendido para continuar mejorando.

Se estructura en cinco fases:

1. INICIACIÓN (INITIATING)

Se identifican los problemas o necesidades que motivan la mejora y se establecen sus objetivos y contexto. Es fundamental conseguir el **patrocinio o apoyo de la dirección (sponsor)**, porque los proyectos de mejora requieren recursos y compromiso organizacional. Sin este respaldo, pueden perder prioridad frente a las actividades operativas habituales. También se establece la infraestructura necesaria para conducir la iniciativa.

2. DIAGNÓSTICO (DIAGNOSING)

Se analiza y caracteriza la situación actual de los procesos de la organización. Se identifican fortalezas, debilidades y diferencias respecto de la situación que se pretende alcanzar. Esta comparación permite realizar un **análisis de brecha**, identificando qué aspectos necesitan mejorarse. A partir de los resultados se elaboran recomendaciones para orientar las acciones posteriores.

3. ESTABLECIMIENTO (ESTABLISHING)

Se determinan las prioridades, estrategias y acciones concretas que permitirán alcanzar los objetivos de mejora. Se elabora un **plan de mejora** que establece actividades, responsables, recursos y resultados esperados, y se organizan los equipos que llevarán adelante las acciones.

4. ACCIÓN (ACTING)

Se implementan los cambios definidos en el plan. Es conveniente comenzar mediante **proyectos piloto**, que permiten comprobar la efectividad de las mejoras antes de extenderlas a toda la organización.

Se evalúan los resultados de la implementación y, cuando corresponde, se planifica y ejecuta el despliegue de los cambios en otros proyectos o sectores.

5. APRENDIZAJE (LEARNING)

Se analizan los resultados de la iniciativa y se documentan las lecciones aprendidas. Se determina qué mejoras funcionaron, cuáles presentaron dificultades y qué debería modificarse para que los siguientes ciclos sean más efectivos.

También puede revisarse el propio enfoque organizacional de mejora.

IDEAL es cíclico: el aprendizaje obtenido permite iniciar nuevas acciones y mantener la mejora continua. IDEAL **no establece qué nivel de madurez tiene una organización**. Proporciona una estructura para conducir un proyecto de mejora de procesos.

MODELO SPICE – MODELO DE EVALUACIÓN Y REFERENCIA

SPICE, vinculado históricamente con la norma **ISO/IEC 15504**, es un marco para la evaluación de procesos de software que permite conocer su capacidad e identificar oportunidades de mejora.

Su enfoque consiste en **evaluar procesos y determinar en qué medida alcanzan sus objetivos y están gestionados**, utilizando los resultados como base para orientar las mejoras.

SPICE distingue seis niveles de capacidad:

- **Nivel 0 — Incompleto:** el proceso no se realiza o no logra adecuadamente su propósito.
- **Nivel 1 — Realizado:** el proceso se ejecuta y alcanza su propósito.
- **Nivel 2 — Gestionado:** el proceso se planifica, monitorea y controla, y sus productos de trabajo se gestionan.
- **Nivel 3 — Establecido:** el proceso se ejecuta siguiendo una definición establecida y adaptada.
- **Nivel 4 — Predecible:** el proceso opera dentro de límites establecidos y sus resultados pueden gestionarse cuantitativamente.
- **Nivel 5 — En optimización:** el proceso se mejora continuamente para responder a los objetivos del negocio.

Estos niveles describen la capacidad de los procesos evaluados, no necesariamente un único nivel de madurez global de toda la organización.

RELACIÓN ENTRE SPICE E IDEAL

Ambos pueden contribuir a la mejora de procesos, pero cumplen funciones diferentes:

- **SPICE:** Ayuda a evaluar la capacidad de los procesos e identificar qué debe mejorarse.
- **IDEAL:** Ayuda a organizar el proyecto de mejora, desde su inicio hasta la implementación y el aprendizaje.

Por lo tanto, los resultados de una evaluación de procesos pueden utilizarse como entrada para planificar acciones de mejora mediante IDEAL.

MODELO CMMI – MODELO DE REFERENCIA

CMMI (Capability Maturity Model Integration) es un modelo de referencia para la evaluación y mejora de procesos, basado en un conjunto integrado de buenas prácticas.

Su objetivo es **orientar a las organizaciones para mejorar sus procesos y aumentar la capacidad de alcanzar sus objetivos**, buscando resultados más consistentes y predecibles.

CARACTERÍSTICAS PRINCIPALES

- **Es un modelo de referencia, no una norma ISO.**
- Recopila buenas prácticas relacionadas con la gestión, el desarrollo y la mejora de procesos.
- Establece objetivos y prácticas de referencia, pero permite que cada organización determine cómo implementarlos.
- Puede aplicarse a distintos ámbitos organizacionales, desde proyectos y sectores hasta organizaciones completas.
- Permite realizar evaluaciones para identificar fortalezas, debilidades y niveles de madurez o capacidad, según la representación utilizada.

CMMI no garantiza que una organización obtenga automáticamente productos de calidad. Su propósito es **mejorar la manera en que se realiza el trabajo y aumentar la capacidad de obtener buenos resultados**.

EVALUACIÓN CON CMMI

La evaluación consiste en comparar los procesos implementados y las evidencias de su ejecución con los objetivos y prácticas establecidos en el modelo.

Se analiza tanto la definición de los procesos como su aplicación real en los proyectos. Estas evaluaciones pueden ser realizadas formalmente por evaluadores capacitados, como los *Lead Appraisers* mencionados en la presentación.

CMMI no debe confundirse con una certificación de una norma ISO: en el enfoque presentado por la cátedra se realizan evaluaciones que permiten determinar el nivel de madurez o capacidad alcanzado.

CONSTELACIONES DE CMMI

En la estructura histórica presentada por la cátedra, CMMI se organiza en tres constelaciones, destinadas a diferentes ámbitos de actividad:

CMMI-DEV — DESARROLLO

Proporciona buenas prácticas para **desarrollar productos y servicios**, incluyendo la planificación, gestión, monitoreo y mejora de los procesos de desarrollo. Es la constelación en la que se concentra el estudio de las representaciones y niveles de madurez.

CMMI-ACQ — ADQUISICIÓN

Proporciona buenas prácticas para **seleccionar y administrar proveedores y adquirir productos o servicios**. Se orienta a organizaciones que necesitan gestionar adquisiciones realizadas a terceros, controlando los acuerdos, compromisos y resultados esperados.

CMMI-SVC — SERVICIOS

Proporciona buenas prácticas para **establecer, entregar y gestionar servicios**, tanto internos como externos. Estas constelaciones comparten un conjunto de prácticas fundamentales, aunque incorporan otras específicas según su ámbito de aplicación. En el esquema histórico de la presentación se identifican **16 áreas de proceso fundamentales compartidas**.

ÁREAS DE PROCESO DE CMMI-DEV

Un **área de proceso (AP)** es un conjunto de prácticas relacionadas que, implementadas conjuntamente, permiten satisfacer objetivos considerados importantes para mejorar un aspecto del trabajo de la organización.

Por ejemplo, la Administración de Requerimientos, la Planificación de Proyectos y el Aseguramiento de Calidad del Proceso y del Producto son áreas de proceso.

En **CMMI-DEV v1.3** existen **22 áreas de proceso**, organizadas en cuatro categorías:

- **Administración de Proyectos:** planificación, seguimiento y control de los proyectos.
- **Soporte:** prácticas transversales, como calidad, configuración y medición.
- **Administración de Procesos:** definición, evaluación y mejora de los procesos organizacionales.
- **Ingeniería:** actividades técnicas relacionadas con el desarrollo y la validación de productos.

Las mismas áreas de proceso pueden utilizarse en las dos representaciones del modelo. Lo que cambia es **cómo se organiza y evalúa el camino de mejora**.

REPRESENTACIONES DE CMMI-DEV

1. REPRESENTACIÓN POR ETAPAS

La representación por etapas evalúa la **madurez organizacional**, considerando un conjunto de áreas de proceso establecido para cada nivel. Proporciona un camino progresivo de mejora, en el cual **para alcanzar un nivel deben satisfacerse los objetivos correspondientes a ese nivel y a los niveles anteriores**.

Su principal ventaja es ofrecer una clasificación global que facilita la comparación entre organizaciones o unidades organizacionales evaluadas con el mismo modelo.

NIVELES DE MADUREZ

La evolución de los niveles representa una transición desde procesos reactivos e impredecibles hacia procesos administrados, definidos, medidos y continuamente mejorados.

Nivel 1 — Inicial

Los procesos son poco predecibles, frecuentemente reactivos y pueden estar insuficientemente definidos o controlados. El éxito depende en gran medida del esfuerzo individual y los proyectos presentan mayores riesgos de retrasos, sobrecostos e incumplimientos.

Nivel 2 — Administrado (Se desarrolla en profundidad)

Los procesos se gestionan principalmente **a nivel de proyecto**. Los requerimientos y compromisos se administran, y el trabajo se planifica, ejecuta, monitorea y controla.

Se incorporan prácticas de seguimiento, medición, gestión de configuración y aseguramiento de calidad.

Nivel 3 — Definido

Los procesos están **definidos y estandarizados a nivel organizacional**. Los proyectos utilizan procesos adaptados a partir de los estándares de la organización, con procedimientos y responsabilidades conocidos.

La organización dispone de una base común para desarrollar, controlar y mejorar sus proyectos.

Nivel 4 — Cuantitativamente administrado

Se utilizan **mediciones y técnicas cuantitativas** para comprender, controlar y predecir el desempeño de los procesos. Las decisiones se apoyan en datos y en objetivos cuantificables de calidad y rendimiento.

Nivel 5 — Optimizado

Se busca la **mejora continua de los procesos**, utilizando mediciones, análisis de causas y cambios organizacionales para mejorar el desempeño y responder a los objetivos del negocio.

ÁREAS DE PROCESO DEL NIVEL 2 — ADMINISTRADO

La cátedra hace especial énfasis en las **siete áreas de proceso del nivel 2**:

1. **REQM — Administración de Requerimientos:** Gestionar los requerimientos y mantener su consistencia con los productos de trabajo.
2. **PP — Planificación de Proyectos:** Establecer estimaciones, planes, compromisos y recursos necesarios.
3. **PMC — Monitoreo y Control de Proyectos:** Comparar el avance real con lo planificado y tomar acciones correctivas ante desviaciones.
4. ***SAM — Administración de Acuerdos con Proveedores:*** Establecer y gestionar acuerdos cuando se adquieren productos o servicios a proveedores.
5. **MA — Medición y Análisis:** Definir necesidades de información, obtener medidas y utilizarlas para apoyar la gestión.
6. **PPQA — Aseguramiento de Calidad del Proceso y del Producto:** Evaluar objetivamente la conformidad de los procesos y productos de trabajo con los criterios establecidos.
7. **CM — Administración de Configuración:** Controlar versiones, cambios e integridad de los productos de trabajo.

SAM merece una consideración particular: su aplicación depende de que existan adquisiciones y acuerdos con proveedores que deban gestionarse. Esto no significa que pueda eliminarse sin justificación, sino que su pertinencia debe analizarse según el contexto.

Las demás áreas del modelo se distribuyen en los niveles superiores y profundizan la ingeniería, la administración organizacional de procesos, la gestión cuantitativa y la mejora continua.

Idea clave del nivel 2: la organización dispone de la capacidad para **administrar sus proyectos de manera planificada y controlada**, reduciendo la dependencia exclusiva de esfuerzos individuales.

2. REPRESENTACIÓN CONTINUA

La representación continua se centra en **evaluar y mejorar la capacidad de áreas de proceso particulares**, en lugar de asignar inicialmente una única clasificación de madurez al conjunto de la organización.

Permite seleccionar las áreas que resulta más importante mejorar según las necesidades del negocio. Su principal ventaja es la **flexibilidad para elegir las prioridades y el orden de mejora.**

Por ejemplo, una organización podría concentrar sus esfuerzos en mejorar la Administración de Configuración, aunque otras áreas presenten capacidades diferentes.

NIVELES DE CAPACIDAD DE LA REPRESENTACIÓN CONTINUA

Según el esquema presentado por la cátedra, la capacidad de cada área de proceso se clasifica en seis niveles, del 0 al 5:

- **Nivel 0 — Incompleto:** El proceso no se ejecuta o se realiza parcialmente, sin alcanzar los objetivos establecidos.
- **Nivel 1 — Realizado:** El proceso se ejecuta y cumple sus objetivos específicos, aunque no necesariamente se encuentre gestionado de manera sistemática.
- **Nivel 2 — Administrado:** El proceso se planifica, monitorea y controla, contando con responsables y recursos definidos.
- **Nivel 3 — Definido:** El proceso está documentado y se adapta a partir de los procesos estándar de la organización.
- **Nivel 4 — Cuantitativamente administrado:** se utilizan mediciones y técnicas cuantitativas para controlar y predecir el desempeño del proceso.
- **Nivel 5 — Optimizado:** se mejora continuamente el proceso mediante el análisis de sus resultados y la incorporación de mejoras.

A diferencia de la representación por etapas, **no se asigna necesariamente un único nivel a toda la organización**. Distintas áreas de proceso pueden presentar diferentes niveles de capacidad, según las prioridades y las mejoras implementadas.

COMPARACIÓN ENTRE REPRESENTACIONES

Aspecto	Por etapas	Continua
Qué evalúa	Madurez organizacional	Capacidad de cada área de proceso
Niveles	5 niveles: del 1 al 5	6 niveles: del 0 al 5
Alcance	Conjunto de áreas de proceso agrupadas por nivel	Áreas de proceso seleccionadas individualmente
Camino de mejora	Secuencia preestablecida y acumulativa	Orden de mejora elegido según las necesidades de la organización
Resultado	Un nivel de madurez organizacional	Un perfil de capacidad de las áreas evaluadas
Principal ventaja	Facilita la comparación entre organizaciones y proporciona un camino progresivo de mejora	Permite concentrar los esfuerzos en las áreas de proceso prioritarias

En síntesis: por etapas responde principalmente **«¿qué nivel de madurez alcanzó la organización?»**, mientras que la continua responde **«¿qué capacidad alcanzó cada proceso evaluado?»**.

ROLES Y GRUPOS EN CMMI

CMMI identifica responsabilidades y actividades que deben estar cubiertas, pero **no exige que cada rol corresponda necesariamente a una persona diferente**.

La asignación depende de las características y el tamaño de la organización.

Por ejemplo, en una organización pequeña una misma persona podría asumir responsabilidades que, en una organización más grande, serían desempeñadas por diferentes grupos.

Lo importante es **que exista un responsable claramente identificado para cada función requerida**, y que puedan preservarse las condiciones de objetividad e independencia cuando correspondan.

RELACIÓN ENTRE CMMI Y LOS ENFOQUES ÁGILES

CMMI y los enfoques ágiles comparten el propósito de **mejorar la calidad, el desempeño y los resultados del desarrollo de software**, pero presentan diferencias en la manera de organizar y controlar el trabajo.

CMMI → Pone énfasis en establecer objetivos de proceso, evaluar su cumplimiento y desarrollar capacidades organizacionales que permitan obtener resultados consistentes.

Ágil → Prioriza la entrega frecuente de valor, la colaboración con el cliente, la retroalimentación y la adaptación del trabajo ante nuevos conocimientos o cambios.

VALORES Y CARACTERÍSTICAS DE CMMI FRENTE A LOS ENFOQUES ÁGILES

Aunque ambos buscan mejorar los resultados del desarrollo de software, presentan diferencias en sus prioridades y en la manera de organizar el trabajo.

Aspecto	CMMI	Enfoques ágiles
Prioridad	Mejorar los procesos para obtener resultados consistentes y productos de mayor calidad.	Responder a las necesidades del cliente mediante entregas frecuentes de valor y adaptación al cambio.
Alcance de la mejora	Enfatiza la mejora organizacional, estableciendo procesos y criterios comunes.	Enfatiza la mejora continua del equipo y del proyecto a través de la experiencia y la retroalimentación.
Comunicación y conocimiento	Favorece la documentación, los activos de proceso y la difusión del conocimiento organizacional.	Favorece la comunicación directa, la colaboración y el conocimiento compartido entre las personas.
Confianza del cliente	Se apoya en la existencia de procesos gestionados, evaluados y capaces de producir resultados predecibles.	Se apoya especialmente en el software funcionando, la colaboración y la demostración frecuente de resultados.
Enfoque de trabajo	Es un modelo principalmente **descriptivo**: establece objetivos y prácticas de referencia, pero permite decidir cómo alcanzarlos.	Los métodos y marcos ágiles pueden establecer prácticas concretas para organizar y ejecutar el trabajo, adaptándolas al contexto.
Medición y adaptación	Enfatiza la medición y el control de los procesos para aumentar su estabilidad y predictibilidad.	Enfatiza la inspección frecuente de resultados, la retroalimentación y la capacidad de responder a nuevas necesidades.

Estas diferencias representan **tendencias de cada enfoque, no características excluyentes**. CMMI también contempla la mejora continua y la adaptación de procesos, mientras que los equipos ágiles también utilizan estándares, planificación, métricas y mecanismos de control.

SIMILITUDES ENTRE AMBOS ENFOQUES

CMMI y los enfoques ágiles comparten varios aspectos:

- **Buscan mejorar el desempeño y la calidad** del software desarrollado.
- **Realizan planificación**, aunque con distintos mecanismos y horizontes.
- **Promueven la mejora continua**, utilizando la experiencia y los resultados obtenidos.
- **Establecen reglas y prácticas** que orientan el trabajo.
- **Deben adaptarse al contexto:** ninguno constituye una solución completa y universal para todos los proyectos.

Por eso, **no son necesariamente enfoques opuestos**. Pueden complementarse combinando la gestión y mejora sistemática de procesos que propone CMMI con la flexibilidad y retroalimentación frecuente de los métodos ágiles.

COMPATIBILIDAD ENTRE AMBOS ENFOQUES

Existen prácticas ágiles que contribuyen a alcanzar objetivos de CMMI:

- La planificación de iteraciones y el seguimiento del trabajo pueden contribuir a la administración del proyecto.
- Los Product Backlogs y las User Stories pueden utilizarse para gestionar requerimientos.
- Las prácticas de integración, control de versiones y automatización pueden contribuir a la administración de configuración.
- Las revisiones y retrospectivas permiten inspeccionar resultados e identificar oportunidades de mejora.
- Las métricas ágiles pueden proporcionar información para monitorear el producto y los procesos.

Sin embargo, **la existencia de estas prácticas no demuestra automáticamente el cumplimiento de CMMI**. Es necesario comprobar que satisfagan los objetivos del modelo y que exista evidencia adecuada.

PRINCIPALES TENSIONES

1. Evaluación objetiva del cumplimiento

CMMI requiere evaluar objetivamente que los procesos y productos de trabajo cumplan los criterios establecidos.

Los equipos ágiles utilizan mecanismos frecuentes de inspección y adaptación, pero estos no reemplazan necesariamente una evaluación objetiva e independiente. Por eso, puede ser necesario incorporar evaluaciones realizadas con suficiente independencia respecto del trabajo evaluado.

2. Gestión y trazabilidad de requerimientos

CMMI requiere gestionar los requerimientos y mantener su consistencia con los productos de trabajo.

En Ágil, los requerimientos pueden expresarse y evolucionar mediante User Stories y Product Backlogs. Para cumplir con los objetivos de CMMI, puede ser necesario asegurar la trazabilidad mediante esos artefactos u otros mecanismos adecuados.

Esto no implica exigir documentación extensa cuando existe otra evidencia suficiente.

3. Estandarización organizacional y adaptación

Los niveles superiores de CMMI aumentan las exigencias de definición organizacional, medición y control de procesos.

Los enfoques ágiles enfatizan la adaptación a las características de cada proyecto y el aprendizaje obtenido durante su ejecución.

La compatibilidad requiere **establecer criterios organizacionales sin eliminar la capacidad de los equipos para adaptar sus prácticas**.

4. Medición cuantitativa

CMMI, especialmente en los niveles superiores de madurez, utiliza medidas del desempeño de los procesos para mejorar su estabilidad y previsibilidad. Ágil también utiliza métricas, pero muchas son específicas del equipo y de su contexto. Por eso, no deben extrapolarse automáticamente los resultados de un equipo o proyecto a otro.

CONCLUSIÓN

CMMI y Ágil pueden complementarse, siempre que se determine cómo las prácticas ágiles permiten alcanzar los objetivos del modelo y qué mecanismos adicionales son necesarios para demostrar su cumplimiento.

CMMI puede aportar disciplina, evaluación y mejora organizacional; Ágil puede aportar flexibilidad, retroalimentación frecuente y orientación al valor.

Las diferencias no implican incompatibilidad absoluta: **ambos deben adaptarse al contexto y a las necesidades de la organización**, evitando tanto la burocracia innecesaria como la falta de controles adecuados.

7. AUDITORÍAS DE CALIDAD DE SOFTWARE

DEFINICIÓN Y PROPÓSITO

Una **auditoría de calidad de software es una evaluación objetiva e independiente de productos o procesos de software**, cuyo propósito es determinar su cumplimiento respecto de estándares, procedimientos, especificaciones y criterios previamente establecidos.

Para ello, el auditor recopila y analiza **evidencia objetiva**, como documentos, registros y productos de trabajo, y la compara con los criterios que deberían cumplirse.

Las auditorías son un instrumento del **Aseguramiento de Calidad de Software (SQA)**, porque permiten identificar incumplimientos, informar los resultados y promover acciones correctivas.

La independencia significa que el auditor debe poder evaluar el trabajo sin estar condicionado por quienes son responsables de realizarlo. **No es indispensable que pertenezca a una empresa externa**, pero sí que cuente con suficiente objetividad e independencia respecto del trabajo auditado.

BENEFICIOS Y COSTOS DE LAS AUDITORÍAS

Las auditorías permiten:

- **Obtener evaluaciones objetivas e independientes**, basadas en evidencia y criterios definidos.
- **Comprobar el cumplimiento de procesos, estándares y procedimientos**, identificando desviaciones.
- **Detectar posibles causas de insatisfacción del cliente**, antes de que generen problemas mayores.
- **Identificar fortalezas y oportunidades de mejora** en la forma de trabajar.
- **Proporcionar visibilidad a la gerencia** sobre el funcionamiento de los proyectos y los procesos de la organización.

Sin embargo, realizar auditorías requiere tiempo, preparación, personal capacitado y recursos. Por eso, deben planificarse adecuadamente, procurando que sus beneficios justifiquen el esfuerzo y costo involucrados.

TIPOS DE AUDITORÍAS DE SOFTWARE

1. AUDITORÍA DE PROYECTO

Su propósito es **verificar que el proyecto aplique correctamente el proceso de desarrollo definido y adaptado para él**, y comprobar la consistencia de los productos de trabajo a medida que evoluciona el software.

Se evalúa la ejecución real del proyecto, comparando las actividades realizadas y las evidencias disponibles con el proceso, los procedimientos y los planes establecidos. Se realizan comprobaciones concretas, como:

- Que las interfaces de hardware y software sean consistentes con los requerimientos especificados.
- Que los requerimientos funcionales de la **ERS** estén contemplados en el Plan de Verificación y Validación.
- Que el diseño documentado en el **DDS (Documento de Diseño de Software)** satisfaga los requerimientos funcionales de la ERS.
- Que el código implementado sea consistente con el diseño documentado en el DDS.

Estas auditorías se realizan conforme a lo establecido en el **PACS (Plan de Aseguramiento de Calidad de Software)**. El PACS debería identificar al responsable de realizar estas auditorías y las actividades de evaluación correspondientes

Las inspecciones de software y las revisiones de documentación pueden proporcionar evidencias útiles para estas auditorías.

Idea clave: la auditoría de proyecto evalúa **cómo se está desarrollando el producto y si los resultados de las distintas actividades son consistentes entre sí**.

2. AUDITORÍA DE CONFIGURACIÓN FUNCIONAL

Su propósito es **comprobar que el software construido implemente completa y correctamente los requerimientos funcionales especificados**. Para ello, se compara el producto desarrollado, incluyendo sus componentes ejecutables y documentación disponible, con los requerimientos definidos en la ERS. Se busca comprobar que:

- Las funcionalidades requeridas hayan sido implementadas.
- El software se corresponda con los requerimientos especificados.
- No existan funcionalidades incorporadas que contradigan o excedan injustificadamente lo establecido.

El responsable de QA también debe comprobar que la **matriz de trazabilidad de requerimientos** esté actualizada, para relacionar los requerimientos con los componentes o evidencias que permiten comprobar su implementación.

Idea clave: compara **lo construido con lo requerido**.

3. AUDITORÍA DE CONFIGURACIÓN FÍSICA

Su propósito es **verificar que los ítems de configuración, tal como fueron construidos, sean consistentes con la documentación técnica que los describe**. Para ello, se compara el código y los componentes del producto con la documentación de soporte correspondiente. Se busca comprobar que:

- La documentación técnica describa correctamente el producto desarrollado.
- Los componentes y sus versiones sean consistentes con lo documentado.
- La configuración que se entregará pueda identificarse y mantenerse correctamente.

El PACS debería establecer quién será responsable de realizar esta auditoría. El software podrá entregarse únicamente cuando se hayan corregido las desviaciones encontradas durante la auditoría de configuración física.

Idea clave: compara **lo construido con su documentación técnica**.

COMPARACIÓN ENTRE LOS TRES TIPOS

Tipo	Qué verifica	Referencia principal
Proyecto	Cumplimiento del proceso y consistencia de los productos de trabajo durante el desarrollo.	Proceso adoptado, planes, ERS, DDS y otras evidencias.
Configuración funcional	Correspondencia entre las funcionalidades implementadas y los requerimientos.	ERS y trazabilidad de requerimientos.
Configuración física	Correspondencia entre los componentes construidos y la documentación que los describe.	Código, configuración y documentación técnica.

AUDITORÍAS AL GRUPO DE CALIDAD (GAC)

Además de las auditorías anteriores, existen las **auditorías al propio Grupo de Aseguramiento de Calidad (GAC)**. Su finalidad es **evaluar si las actividades de aseguramiento de calidad se están realizando adecuadamente**, verificando el cumplimiento de los procedimientos y responsabilidades que corresponden al grupo.

Esto puede comprender la revisión de sus planes, evaluaciones realizadas, informes, seguimiento de desviaciones y otras evidencias de su trabajo. Para preservar la objetividad, la evaluación debe ser realizada por personas **independientes del grupo evaluado**, que pueden pertenecer a una instancia externa a la organización.

Distinción importante: En este caso, el objeto de evaluación es **el funcionamiento del propio grupo que realiza actividades de calidad**, no directamente la ejecución de un proyecto.

ROLES Y RESPONSABILIDADES EN UNA AUDITORÍA

En las auditorías intervienen tres roles principales:

AUDITOR

Es el responsable de **realizar la evaluación objetiva y elaborar sus resultados**. Debe contar con suficiente independencia respecto del trabajo auditado. Entre sus responsabilidades se encuentran:

- Acordar la fecha y comunicar el alcance de la auditoría.
- Recopilar y analizar evidencia objetiva, relevante y suficiente.
- Entrevistar a los auditados y comprobar el cumplimiento de los criterios establecidos.
- Registrar los hallazgos y preparar el reporte de auditoría.
- Realizar el seguimiento de los planes de acción acordados.

AUDITADO

Es la persona o el grupo responsable del proceso, proyecto o actividad que se evalúa. En una auditoría de proyecto, puede ser el líder del proyecto u otro responsable designado. Sus responsabilidades incluyen:

- Acordar la fecha y participar en la auditoría.
- Proporcionar la documentación y evidencia solicitadas.
- Responder las preguntas del auditor.
- Analizar los resultados y responder al reporte recibido.
- Proponer un plan de acción para las desviaciones identificadas.
- Ejecutar las acciones comprometidas e informar su cumplimiento.

GERENTE DE SQA

Es responsable de **organizar y gestionar las auditorías de calidad**.

Sus principales funciones son:

- Preparar el plan de auditorías.
- Estimar los costos y asignar los recursos necesarios.
- Resolver las no conformidades o desacuerdos que se produzcan entre el auditor y el auditado, cuando corresponda su intervención.

Diferencia principal: el auditor **evalúa e informa**, el auditado **proporciona evidencia y corrige las desviaciones**, y el gerente de SQA **organiza y supervisa la gestión de las auditorías**.

PROCESO DE AUDITORÍA

El proceso de auditoría comprende cuatro etapas**: preparación y planificación, ejecución, análisis y reporte de resultados, y seguimiento**.

1. PREPARACIÓN Y PLANIFICACIÓN

Se organiza la auditoría antes de realizarla. No se trata de una actividad sorpresiva, sino de una evaluación cuyo propósito y condiciones deben conocerse.

1. El auditado solicita la auditoría.
2. Auditor y auditado acuerdan la fecha, el lugar, el objetivo y el alcance.
3. Se identifican los participantes y sus responsabilidades.
4. El auditor puede solicitar documentación preliminar para conocer el proceso o producto que evaluará.
5. El auditado prepara la documentación y la información que serán necesarias durante la ejecución.

También se establecen los criterios que se utilizarán para evaluar y, cuando corresponde, las métricas que permitirán analizar el proceso de auditoría.

Resultado: una auditoría organizada, con objetivos, alcance, participantes y documentación identificados.

2. EJECUCIÓN

Se realiza la auditoría y se recopila la evidencia necesaria para determinar el cumplimiento de los criterios establecidos. Comprende dos momentos principales:

a. Reunión de apertura

El auditor presenta al equipo auditor y a los participantes, explica el procedimiento que se seguirá y comunica las condiciones de la evaluación. Se revisan el cronograma, los objetivos, el alcance y los aspectos organizativos. El auditado puede realizar preguntas y aclarar inquietudes.

b. Ejecución de la evaluación

El auditor realiza preguntas sobre cómo se ejecutan las actividades, solicita documentación y comprueba las evidencias disponibles. El auditado responde las preguntas y presenta los registros, documentos u otros productos de trabajo solicitados. El auditor utiliza un **checklist** para registrar las comprobaciones y los posibles hallazgos.

Es importante contrastar **lo que las personas declaran hacer con la evidencia de lo que efectivamente se realiza**, porque la auditoría no puede basarse únicamente en declaraciones. Al finalizar, se informa cuándo se comunicarán los resultados.

3. ANÁLISIS Y REPORTE DE RESULTADOS

Se analizan las evidencias recopiladas, se determinan los hallazgos y se comunican los resultados. Esta etapa comprende tres actividades:

a. Evaluación de los resultados

El auditor revisa el checklist, las anotaciones y la documentación obtenida. Luego contrasta las posibles desviaciones con los procedimientos y criterios aplicables, para fundamentar los resultados y elaborar un **reporte preliminar**.

b. Reunión de cierre

El auditor presenta el reporte preliminar y explica los resultados. El auditado puede solicitar aclaraciones, aportar observaciones y discutir la interpretación de determinados hallazgos. Se revisan las conclusiones y se establece cuándo se entregará el reporte definitivo.

Esto permite aclarar diferencias sin eliminar hallazgos que estén debidamente respaldados por evidencia objetiva.

c. Entrega del reporte final

El auditor incorpora las aclaraciones pertinentes de la reunión de cierre, prepara el reporte definitivo, conserva una copia y comunica formalmente los resultados al auditado.

Resultado: un informe documentado que identifica los hallazgos y las acciones que deben tratarse.

4. SEGUIMIENTO

Su propósito es **comprobar que las desviaciones identificadas sean tratadas y efectivamente resueltas**.

1. El **auditado** analiza las desviaciones y prepara un **Plan de Acción**, indicando qué hará para corregirlas y en qué fechas.
2. El **auditor** revisa y aprueba el plan acordado.
3. El **auditado** ejecuta las acciones correctivas, realiza su seguimiento e informa su cumplimiento.
4. El **auditor** verifica mediante evidencia que las acciones se hayan cumplido y que las desviaciones estén resueltas.
5. Finalmente, el auditor **cierra las desviaciones** e informa su cierre.

Por lo tanto, **el seguimiento no termina cuando el auditado promete corregir un problema, sino cuando se verifica que las acciones acordadas fueron realizadas y resultaron satisfactorias**.

HERRAMIENTAS Y TÉCNICAS UTILIZADAS EN AUDITORÍAS

CHECKLIST DE AUDITORÍA

Es una lista estructurada de preguntas o criterios que orienta la evaluación y permite registrar las comprobaciones realizadas.

Busca establecer un **mínimo común de aspectos a verificar**, independientemente de quién realice la auditoría. Sin embargo, el auditor puede formular preguntas adicionales cuando la evidencia lo requiera.

Un checklist puede incluir:

- Fecha de la auditoría.
- Nombre del auditor y de los auditados, identificando sus roles.
- Nombre y fase del proyecto auditado, cuando corresponda.
- Objetivo y alcance de la auditoría.
- Lista de preguntas o comprobaciones.

Por ejemplo, en una auditoría de proyecto pueden preguntarse si existe un plan actualizado, si las actividades tienen responsables y si se identificaron los riesgos.

En una auditoría de requerimientos puede verificarse si existe una especificación, si los requerimientos están identificados y si fueron revisados y aprobados.

TÉCNICA DE CUESTIONARIO Y CONDUCTA DEL AUDITOR

Las entrevistas permiten conocer cómo se realiza el trabajo y orientar la búsqueda de evidencias. Se recomienda:

- Comenzar con **preguntas abiertas**, como qué se hace, cómo, cuándo y quién participa.
- Formular preguntas cortas y concretas.
- Finalizar con **preguntas cerradas** para aclarar o confirmar información.

Durante la entrevista, el auditor debe escuchar sin interrumpir, tomar notas, formular preguntas, observar la comunicación y confirmar que comprendió correctamente las respuestas.

También debe considerar que los auditados pueden encontrarse nerviosos o sentir que están siendo evaluados personalmente. El objetivo es **evaluar el trabajo y sus evidencias, no juzgar a las personas**.

OTRAS TÉCNICAS

- **Muestreo:** Seleccionar una muestra representativa de productos, actividades o registros que permita evaluar el cumplimiento sin revisar necesariamente la totalidad de los elementos.
- **Revisión de registros:** Examinar documentos, planes, aprobaciones, resultados y otros registros para comprobar cómo se realizaron las actividades.
- **Herramientas automatizadas:** Apoyar la recopilación, el análisis o el registro de evidencias y resultados de auditoría.

RESULTADOS O HALLAZGOS DE UNA AUDITORÍA

Se distinguen **tres tipos de resultados**.

1. BUENAS PRÁCTICAS

Son prácticas, procedimientos o soluciones que se realizan **de manera significativamente superior a lo esperado**. Pueden representar fortalezas de la organización y servir como referencia para mejorar otros proyectos o procesos.

Por ejemplo, implementar un sistema especialmente eficaz para mantener actualizada y accesible la información de los proyectos.

2. DESVIACIONES

Son **incumplimientos respecto de los requisitos, procedimientos o criterios establecidos** que afectan o generan incertidumbre sobre la conformidad del producto o del proceso. Pueden incluir:

- Productos que no cumplen sus requerimientos.
- Falta de controles necesarios para asegurar el cumplimiento.
- Actividades que no se realizan conforme al proceso definido.
- Incumplimientos de requisitos documentados que pueden comprometer la calidad.

Las desviaciones **requieren un Plan de Acción por parte del auditado**, cuyo cumplimiento debe verificarse durante el seguimiento.

3. OBSERVACIONES

Son situaciones o prácticas que **convendría mejorar, pero que no constituyen una desviación que exija un Plan de Acción**. Pueden representar riesgos, debilidades u oportunidades de mejora.

Por ejemplo, una técnica de estimación que todavía se utiliza de acuerdo con el procedimiento establecido, pero que muestra resultados poco confiables y podría mejorarse.

Diferencia fundamental: Las desviaciones requieren acciones correctivas y seguimiento formal; las observaciones señalan aspectos a considerar, pero no exigen necesariamente un plan de acción.

REPORTE DE AUDITORÍA

Es el documento que **registra y comunica formalmente los resultados de la evaluación**. Debe incluir información como:

- Identificación y fecha de la auditoría.
- Nombre del auditor y de los auditados.
- Proyecto y fase evaluados, cuando corresponda.
- Lista de resultados o hallazgos.
- Comentarios y aclaraciones relevantes.
- Solicitud de planes de acción para las desviaciones identificadas.

El reporte permite comunicar los resultados a los responsables, proporcionar trazabilidad de los hallazgos y servir como base para su seguimiento.

MÉTRICAS DE AUDITORÍA

Las métricas permiten **evaluar el esfuerzo, desempeño y resultados de las auditorías**, para identificar oportunidades de mejora en su realización. Cada organización debe seleccionar las que resulten adecuadas. La cátedra menciona:

- **Esfuerzo por auditoría:** Recursos o tiempo de trabajo necesarios para realizarla.
- **Cantidad de desviaciones:** Número de incumplimientos identificados.
- **Duración de la auditoría:** Tiempo transcurrido durante su realización.
- **Desviaciones clasificadas por área de proceso de CMMI:** Distribución de los hallazgos según las áreas del modelo con las que se relacionan.

Estas métricas ayudan a comprender el funcionamiento del proceso de auditoría. Sin embargo, **la cantidad de desviaciones por sí sola no permite determinar la calidad de un proyecto o de una auditoría**: debe interpretarse considerando el alcance, los criterios y el contexto de la evaluación.

RELACIÓN ENTRE AUDITORÍAS Y OTROS MECANISMOS DE CALIDAD

Las auditorías, las revisiones técnicas y las retrospectivas pueden contribuir a mejorar la calidad, pero **no son equivalentes**.

- **Auditorías:** Evalúan objetivamente la conformidad de procesos o productos con criterios establecidos, requieren independencia respecto del trabajo evaluado y generan hallazgos y acciones de seguimiento.
- **Revisiones técnicas:** Examinan productos de trabajo, generalmente entre colegas, para identificar defectos tempranamente.
- **Retrospectivas ágiles:** Permiten que el equipo inspeccione su forma de trabajar y acuerde mejoras a partir de la experiencia.

Un equipo ágil puede utilizar retrospectivas y revisiones técnicas como parte de sus mecanismos de calidad, **sin que esto elimine automáticamente la necesidad de auditorías independientes cuando existan requisitos de conformidad que deban evaluarse**.

8. VERIFICACIÓN, VALIDACIÓN Y REVISIONES TÉCNICAS

VERIFICACIÓN Y VALIDACIÓN (V&V)

La **Verificación y Validación (V&V)** comprende actividades destinadas a comprobar que el software se construya correctamente y que satisfaga las necesidades para las cuales fue desarrollado.

Son actividades que deben realizarse **durante todo el ciclo de vida**, comenzando con la revisión de los requerimientos y continuando con el diseño, el código y las pruebas.

Se distinguen dos conceptos fundamentales:

- **Verificación — ¿Estamos construyendo el producto correctamente?** Comprueba que los productos de trabajo se ajusten a las especificaciones, estándares y criterios establecidos para su desarrollo.
- **Validación — ¿Estamos construyendo el producto correcto?** Comprueba que el producto responda a las necesidades y expectativas reales de los usuarios y del negocio.

Por ejemplo, un sistema puede implementar correctamente todos los requerimientos documentados y superar su verificación, pero no satisfacer las necesidades del usuario si esos requerimientos fueron definidos incorrectamente.

IMPORTANCIA DE REALIZAR V&V TEMPRANAMENTE

Las actividades de V&V deben **planificarse e incorporarse desde las primeras etapas del desarrollo**, porque detectar un problema cerca del momento en que se introduce permite reducir el retrabajo.

Si un error en los requerimientos se descubre durante su revisión, puede corregirse modificando la especificación. En cambio, si se descubre después de implementar el software, podría requerir cambios en el diseño, el código, las pruebas y la documentación.

Por eso, **el costo de corregir problemas tiende a aumentar cuanto más tarde se detectan**.

La necesidad de estas actividades se relaciona con las dificultades propias del desarrollo de software, como los problemas de comunicación, las limitaciones humanas de memoria y la complejidad del producto.

ERROR, DEFECTO Y FALLA EN LAS REVISIONES

En el contexto de las revisiones técnicas, se define **falla como un error en un producto de trabajo**, entendiendo por producto de trabajo cualquier resultado generado durante una actividad del ciclo de vida del software (requerimientos, diseño, código, documentación, etc).

Según dónde se haya originado el problema, se distingue entre:

- **Error:** se originó en el mismo producto de trabajo que está siendo inspeccionado.
- **Defecto:** se originó en un producto de trabajo anterior y se propagó al producto que ahora se inspecciona.

Por ejemplo, si durante la revisión de un diseño se identifica una decisión de diseño incorrecta, se trata de un error originado en ese documento. Si el problema proviene de un requerimiento incorrecto que fue trasladado al diseño, se considera un defecto heredado.

La presentación también clasifica los hallazgos según su gravedad:

- **Mayor:** Puede provocar un funcionamiento incorrecto o un resultado inesperado.
- **Menor:** Representa una condición no deseada que no ocasionaría directamente una falla operacional, pero puede perjudicar la claridad o el mantenimiento.
- **Cosmético:** Comprende errores de presentación, tipeo, ortografía u otros detalles menores.

Esta clasificación pertenece al contexto de las revisiones técnicas. **No debe confundirse automáticamente con la terminología de error, defecto y falla que se utiliza al estudiar Testing**, donde también se analiza la manifestación de problemas durante la ejecución.

PRINCIPIOS DE V&V

1. **Prevenir es mejor que corregir:** Evitar la introducción de problemas resulta preferible a descubrirlos tardíamente.
2. **Evitar es más efectivo que eliminar:** Conviene actuar sobre las causas de los problemas, no solamente sobre sus consecuencias.
3. **La retroalimentación favorece el aprendizaje:** Los hallazgos permiten mejorar productos y procesos.
4. **Priorizar lo rentable:** El esfuerzo de evaluación debe dirigirse a las actividades que aportan mayor beneficio.
5. **No buscar la perfección absoluta:** Deben establecerse objetivos de calidad razonables considerando costos y riesgos.
6. **Enseñar a prevenir:** Desarrollar las capacidades del equipo permite evitar problemas futuros.

Para realizar V&V existen dos aproximaciones complementarias:

- **Revisiones técnicas:** Examinan productos de trabajo sin necesidad de ejecutar el software.
- **Pruebas de software (Testing):** Comprueban el comportamiento del producto mediante su ejecución.

Ambas contribuyen a evaluar la calidad, pero se aplican de distinta manera y permiten detectar diferentes clases de problemas.

REVISIONES TÉCNICAS — PEER REVIEW

Las **revisiones técnicas son actividades de V&V estática realizadas entre colegas**, cuyo propósito principal es **detectar tempranamente defectos en los productos de trabajo**, reduciendo el retrabajo y contribuyendo a mejorar la calidad del software.

Pueden aplicarse a cualquier representación legible del producto, como requerimientos, diseños, arquitectura, código fuente, planes de proyecto o casos de prueba.

Se denominan **estáticas** porque permiten identificar problemas mediante el examen del producto de trabajo, **sin necesidad de ejecutar el programa**.

Las revisiones se apoyan en que un colega puede identificar problemas que el autor del producto no advierte, ya sea por errores de interpretación, conocimientos diferentes o familiaridad excesiva con su propio trabajo.

OBJETIVOS Y CARACTERÍSTICAS

Las revisiones técnicas buscan:

- **Detectar defectos lo más cerca posible de su origen**, evitando que se propaguen a etapas posteriores.
- Verificar el cumplimiento de requerimientos, estándares y criterios técnicos.
- Identificar oportunidades de mejora en los productos de trabajo.
- Favorecer la colaboración, el aprendizaje y el intercambio de conocimientos entre los integrantes del equipo.

Un principio fundamental es **revisar el producto, no a la persona que lo elaboró**. El objetivo de la revisión no es buscar responsables ni evaluar el desempeño individual, sino identificar problemas para mejorar el resultado del trabajo.

Asimismo, **la revisión se concentra en detectar y registrar los defectos, no en resolverlos durante la reunión**. Las correcciones se realizan posteriormente.

Las revisiones técnicas pueden utilizarse en distintos enfoques de desarrollo, incluidos los ágiles. Sin embargo, **una revisión entre pares no reemplaza automáticamente una auditoría independiente**, porque sus objetivos y condiciones de evaluación son diferentes.

VENTAJAS Y DESVENTAJAS

Ventajas:

- Permiten descubrir numerosos defectos antes de que lleguen a etapas posteriores.
- Pueden aplicarse a productos incompletos, sin esperar a que exista un software ejecutable.
- Permiten evaluar aspectos adicionales a la funcionalidad, como claridad, consistencia y mantenibilidad.
- Favorecen el aprendizaje colectivo y la difusión de conocimientos técnicos.
- Pueden reducir los costos posteriores de corrección y retrabajo.

Desventajas y costos:

- Requieren tiempo para organizar reuniones, preparar materiales y analizar los productos.
- Las inspecciones formales pueden resultar difíciles de introducir porque exigen capacitación y disciplina.
- Al comienzo implican costos adicionales, y sus beneficios pueden observarse más claramente cuando el equipo adquiere experiencia.
- Requieren recursos para preparar guías, checklists, informes, herramientas y mediciones.

Por eso, las revisiones deben planificarse de manera adecuada, procurando que el esfuerzo requerido resulte razonable frente a los beneficios esperados.

TIPOS DE REVISIONES TÉCNICAS

Se distinguen dos modalidades principales:

Aspecto	Walkthrough o recorrido	Inspección formal
Formalidad	Informal, sin un procedimiento riguroso.	Proceso definido, con roles y etapas.
Objetivo principal	Revisar el trabajo con poca sobrecarga, obtener comentarios y compartir conocimiento.	Detectar defectos de manera sistemática y efectiva.
Preparación	Poca o ninguna preparación previa.	Preparación individual antes de la reunión.
Registro y métricas	Escasos o no sistemáticos.	Registro de hallazgos, informes y mediciones.
Ventaja principal	Rapidez y flexibilidad.	Mayor control y trazabilidad del proceso de revisión.

1. WALKTHROUGH O RECORRIDO — REVISIÓN INFORMAL

Es una **técnica de análisis estático en la que el autor o responsable guía a otras personas a través de un producto de trabajo**, mientras los participantes formulan preguntas, realizan comentarios e identifican posibles defectos. Puede utilizarse para revisar código, diseños, requerimientos u otros artefactos.

Sus objetivos principales son:

- **Mínima sobrecarga:** Revisar el producto sin organizar una inspección formal.
- **Capacitación y transferencia de conocimiento:** Permitir que los participantes comprendan el trabajo realizado y aprendan de otros.
- **Rápida retroalimentación:** Identificar problemas o dudas de manera oportuna.

El recorrido no requiere necesariamente roles formalmente definidos, preparación extensa ni un sistema de medición.

Su principal ventaja es la flexibilidad, aunque **la falta de registros y controles formales limita la posibilidad de obtener métricas y analizar sistemáticamente los resultados**.

Es una práctica compatible con los enfoques ágiles, que favorecen la comunicación directa y la revisión frecuente de los productos de trabajo.

2. INSPECCIÓN — REVISIÓN FORMAL

La **inspección es una revisión técnica formal que utiliza un procedimiento definido, roles específicos, preparación previa, checklists y registro de defectos**.

Su objetivo principal es **identificar defectos de manera sistemática y efectiva**, proporcionando información útil para mejorar los productos y procesos.

Las inspecciones formales buscan:

1. Detectar errores en los productos de trabajo.
2. Verificar el cumplimiento de los requerimientos.
3. Comprobar que los productos respeten los estándares establecidos.
4. Favorecer la uniformidad del desarrollo.
5. Proporcionar mayor control y visibilidad sobre la calidad del trabajo.

Se desarrollan mediante reuniones planificadas y requieren un esfuerzo intelectual considerable, por lo que se recomienda limitar su duración.

QUÉ SON Y QUÉ NO SON LAS INSPECCIONES

Las inspecciones **son** mecanismos para detectar defectos tempranamente, obtener métricas, compartir conocimiento, promover el trabajo en equipo y mejorar la calidad del producto.

En cambio, **no deben utilizarse para resolver los defectos durante la reunión, evaluar el desempeño de las personas ni sustituir un procedimiento formal de aprobación del producto de trabajo**.

Al finalizar una inspección puede decidirse si el producto necesita correcciones o una nueva revisión. Esa decisión técnica no debe confundirse con una aprobación contractual o de negocio.

ROLES DE UNA INSPECCIÓN FORMAL

1. Autor

Es **quien creó o mantiene el producto de trabajo** que será inspeccionado.

Inicia el proceso designando al moderador y participa con él en la selección de los demás roles. Entrega el material que será revisado. Después de la inspección, realiza las correcciones necesarias e informa al moderador el esfuerzo de retrabajo y los defectos corregidos.

2. Moderador

Es el responsable de **planificar, organizar y conducir la inspección**.

Selecciona los participantes junto con el autor, distribuye el material con anticipación —48 horas antes— y coordina la reunión. Debe evitar discusiones improductivas o comportamientos inapropiados y realizar el seguimiento de los defectos identificados.

3. Inspector

Examina individualmente el producto antes de la reunión para identificar posibles defectos.

Durante la inspección, comunica los hallazgos obtenidos y registra el tiempo dedicado a su preparación. Los participantes pueden actuar como inspectores, aunque tengan otras responsabilidades dentro de la revisión.

4. Lector

Recorre o lee el producto durante la reunión, permitiendo que los participantes sigan un orden común y mantengan el foco en el contenido revisado.

5. Anotador

Registra los defectos, observaciones y decisiones obtenidos durante la reunión. Su registro sirve como base para elaborar el informe final de la inspección.

Los roles no necesariamente requieren cinco personas diferentes: algunas responsabilidades pueden ser asumidas por un mismo participante, siempre que se mantenga el funcionamiento adecuado de la inspección.

ELECCIÓN DE LOS REVISORES

Los participantes deben seleccionarse considerando el **tipo de producto que será inspeccionado y los conocimientos necesarios para evaluarlo**.

Por ejemplo:

- **Requerimientos:** Analistas, arquitectos, testers y representantes del negocio.
- **Diseño o arquitectura:** Arquitectos, diseñadores, desarrolladores y testers.
- **Código fuente:** Programadores, diseñadores y testers.
- **Plan de proyecto:** Líder de proyecto, responsables técnicos, interesados y representantes de calidad.
- **Plan de pruebas:** Testers, desarrolladores, analistas y responsables de calidad.

La finalidad es incorporar perspectivas diferentes para aumentar las posibilidades de detectar problemas.

ETAPAS DEL PROCESO DE INSPECCIÓN

El proceso convencional comprende **seis etapas**.

1. PLANIFICACIÓN

El moderador organiza la inspección, en coordinación con el autor. Se seleccionan los participantes y sus roles, se define qué producto será revisado y se establece el cronograma, lugar y duración de la reunión. También se preparan y distribuyen los materiales necesarios.

Se recomienda que **una reunión de inspección no supere las dos horas**, debido al esfuerzo intelectual que requiere.

2. VISIÓN GENERAL

El autor presenta una descripción general del producto que se inspeccionará, explicando su propósito y características principales. Esta etapa permite que los participantes comprendan el contexto antes de analizar el material en detalle. Puede omitirse cuando ese contexto ya es suficientemente conocido.

3. PREPARACIÓN

Cada inspector **analiza individualmente el producto de trabajo antes de la reunión**, utilizando la documentación, los criterios y los checklists correspondientes. El propósito es identificar posibles defectos y registrar observaciones para discutirlas posteriormente.

La preparación es fundamental porque permite dedicar la reunión a contrastar hallazgos, en lugar de comenzar a conocer el producto durante ella.

4. REUNIÓN DE INSPECCIÓN

Los participantes examinan el producto y comparten los posibles defectos identificados durante la preparación. El lector guía el recorrido del material, los inspectores presentan sus hallazgos, el anotador los registra y el moderador coordina la actividad.

Se busca **identificar defectos y descartar falsos positivos**, evitando dedicar la reunión a diseñar soluciones. Al finalizar se registra una decisión sobre el estado del producto, que puede ser:

- **Aceptarlo sin modificaciones.**
- **Aceptarlo provisionalmente**, sujeto a correcciones.
- **Rechazarlo**, requiriendo modificaciones y una nueva evaluación.

Se elabora un informe que identifica **qué se revisó, quiénes participaron, qué se descubrió y cuáles fueron las conclusiones**.

5. CORRECCIÓN

El autor analiza y corrige los defectos identificados durante la reunión. Esta etapa ocurre después de la inspección, manteniendo separadas **la detección de problemas y su resolución**. El autor también proporciona información sobre el retrabajo realizado, que puede utilizarse para las métricas del proceso.

6. SEGUIMIENTO

El moderador verifica que los defectos hayan sido tratados adecuadamente y recopila los datos necesarios para cerrar la inspección. Según la gravedad y el alcance de los problemas, pueden existir distintas acciones:

- **Defectos simples:** Se comprueba su corrección sin necesidad de repetir toda la inspección.
- **Defectos que requieren comprobación adicional:** El autor puede reunirse con el moderador para revisar las correcciones.
- **Defectos graves o numerosos:** Puede ser necesaria una **reinspección** del producto.

El objetivo es comprobar que las correcciones resultaron satisfactorias y aprovechar la información obtenida para mejorar futuras inspecciones.

HERRAMIENTAS Y MÉTRICAS DE LAS REVISIONES TÉCNICAS

CHECKLISTS Y GUÍAS DE REVISIÓN

Los **checklists** permiten establecer qué aspectos deben examinarse, disminuir la posibilidad de omitir errores conocidos y mantener criterios comunes entre los revisores.

Su contenido depende del artefacto inspeccionado. Por ejemplo, en una revisión de requerimientos pueden incluir preguntas sobre claridad, consistencia, completitud y posibilidad de verificación.

Las guías deben utilizarse como apoyo, sin impedir que los revisores identifiquen problemas no previstos.

MÉTRICAS DE INSPECCIÓN

Estas métricas permiten evaluar los resultados y el esfuerzo de las inspecciones:

- **Total de defectos encontrados:** Cantidad de defectos mayores y menores identificados durante la inspección. Los hallazgos cosméticos no se incluyen en este indicador.
- **Densidad de defectos:** Cantidad de defectos encontrados en relación con el tamaño del producto revisado. *Densidad = Total de defectos / Tamaño del producto*
- **Esfuerzo de inspección:** Esfuerzo total destinado a planificación, preparación, reunión y retrabajo. *Esfuerzo total = Planificación + Preparación + Reunión + Retrabajo*
- **Esfuerzo por defecto:** relación entre el esfuerzo invertido y la cantidad de defectos identificados. *Esfuerzo por defecto = Esfuerzo de inspección / Total de defectos encontrados*
- **Porcentaje de reinspecciones:** proporción de inspecciones que requieren realizarse nuevamente. *% Reinspecciones = (Cantidad de reinspecciones / Cantidad de inspecciones) × 100*

Estas métricas permiten conocer el costo y los resultados del proceso, identificar dificultades recurrentes y establecer mejoras.

Deben interpretarse según el tipo y tamaño del producto, la complejidad y el contexto de la revisión. **Una cantidad alta de defectos encontrados no demuestra por sí sola que la inspección sea mejor o peor**, porque también depende del estado inicial del producto.

RECOMENDACIONES PARA REVISIONES EFECTIVAS

- **Revisar el producto, no al productor**, evitando responsabilizar personalmente al autor.
- **Definir una agenda y respetarla**, limitando la duración de las reuniones.
- Exigir **preparación previa** de los participantes y distribuir el material con anticipación.
- **Limitar el número de participantes** para mantener la revisión manejable.
- **Identificar y registrar los problemas** sin convertir la reunión en una discusión extensa sobre soluciones.
- Utilizar checklists y mantener **registros escritos** de los resultados.
- Proporcionar capacitación, recursos y herramientas adecuados.
- Revisar los resultados de inspecciones anteriores para mejorar el proceso.

La idea fundamental es que **la efectividad de una inspección depende tanto de la preparación y organización como de los conocimientos técnicos de sus participantes**.

RELACIÓN ENTRE REVISIONES TÉCNICAS, AUDITORÍAS Y TESTING

Estas actividades contribuyen a la calidad del software, pero presentan diferencias en sus objetivos y funcionamiento.

Aspecto	Revisiones técnicas	Auditorías	Testing
Propósito principal	Detectar defectos tempranamente en productos de trabajo.	Evaluar objetivamente el cumplimiento de criterios establecidos.	Detectar fallas y comprobar el comportamiento del software.
Objeto de evaluación	Requerimientos, diseño, código, planes y otros artefactos.	Procesos, productos, documentación y evidencias.	Software o componentes ejecutables.
Método	Análisis estático.	Evaluación de conformidad basada en evidencias.	Pruebas dinámicas mediante ejecución.
Participantes	Generalmente colegas del equipo.	Auditores con suficiente independencia.	Desarrolladores, testers u otros responsables, según el nivel de prueba.
Resultados	Defectos identificados, informe y correcciones.	Hallazgos, informe, plan de acción y seguimiento.	Resultados de pruebas y defectos reportados.

Son actividades complementarias: las revisiones permiten detectar problemas sin esperar al software ejecutable; Testing evalúa el comportamiento durante la ejecución; y las auditorías comprueban que se respeten los procesos, estándares y criterios establecidos.

Ninguna por sí sola garantiza la ausencia de defectos ni sustituye la necesidad de gestionar la calidad durante todo el desarrollo.

9. TESTING DE SOFTWARE

DEFINICIÓN Y PROPÓSITO DEL TESTING

El **Testing de Software es un proceso que comprende actividades de planificación, preparación, ejecución y evaluación de pruebas**, destinadas a detectar defectos y obtener información sobre el cumplimiento de los requerimientos y la adecuación del software a su propósito.

La definición clásica caracteriza al Testing como una **actividad destructiva que busca encontrar defectos cuya presencia se supone de antemano**. Esto significa adoptar una actitud crítica: en lugar de intentar demostrar únicamente que el software funciona, se buscan condiciones que permitan descubrir comportamientos incorrectos o inesperados.

El Testing también puede entenderse como el **proceso de ejecutar un programa con la intención de encontrar fallas**, observando y comparando sus resultados reales con los esperados.

Sin embargo, el proceso de Testing es más amplio que la ejecución: **puede comenzar antes de escribir el código**, mediante el análisis de requerimientos, la planificación de pruebas y el diseño de casos de prueba. Por eso, comprende actividades estáticas y dinámicas, aunque la ejecución del software es su actividad más característica.

RELACIÓN ENTRE TESTING Y ASEGURAMIENTO DE CALIDAD

El Testing es una de las actividades de **control de calidad del producto**, dentro del marco más amplio del Aseguramiento de Calidad de Software (QA/SQA).

- **QA:** Trabaja sobre los procesos y las prácticas utilizadas durante todo el desarrollo, buscando prevenir problemas y asegurar el cumplimiento de criterios de calidad.
- **Testing:** Evalúa el software para descubrir defectos, comprobar su comportamiento y proporcionar información sobre su calidad.

El Testing no garantiza que el software esté libre de defectos, ni asegura por sí solo la calidad del producto o del proceso. La calidad debe construirse durante todo el desarrollo; las pruebas permiten revelar problemas y aportar información para corregirlos.

¿QUÉ SIGNIFICA UN TESTING EXITOSO?

Desde el enfoque clásico, **una prueba exitosa es aquella que logra descubrir un defecto**, porque su propósito es encontrar problemas cuya existencia se presume.

Esto no significa que una prueba que obtiene el resultado esperado carezca de valor: también aporta evidencia sobre el comportamiento comprobado, aunque no demuestra que el sistema esté libre de defectos.

Existe una **relación entre calidad previa y esfuerzo de pruebas**:

- Un producto desarrollado con buenas prácticas de QA puede presentar menos defectos, por lo que será más difícil encontrarlos durante las pruebas.
- Un producto con muchos defectos puede requerir numerosos ciclos de corrección y nuevas pruebas, incrementando el retrabajo y los costos.

Por eso, **el éxito del proyecto no debe evaluarse por la cantidad de defectos encontrados**, sino por la calidad finalmente alcanzada y la información que las pruebas aportan para tomar decisiones.

PRINCIPIOS DEL TESTING

1. EL TESTING MUESTRA LA PRESENCIA DE DEFECTOS

Las pruebas permiten demostrar que existen defectos cuando se los encuentra, pero **no pueden demostrar su ausencia absoluta**. Que todos los casos ejecutados hayan pasado significa que no se observaron fallas bajo esas condiciones, no que el producto sea perfecto.

2. EL TESTING EXHAUSTIVO ES IMPOSIBLE

Probar todas las entradas, combinaciones, secuencias y condiciones posibles suele requerir tiempo y costos inviables. Por eso, es necesario **seleccionar y priorizar las pruebas** según los riesgos, la importancia de las funcionalidades y los recursos disponibles.

3. TESTING TEMPRANO

Las actividades de prueba deben comenzar lo antes posible, incluso antes de la implementación. Revisar requerimientos y diseñar pruebas tempranamente permite descubrir ambigüedades, anticipar dificultades y **reducir el costo de corregir defectos**.

4. AGRUPAMIENTO DE DEFECTOS

Los defectos suelen concentrarse en determinadas partes del sistema, en lugar de distribuirse uniformemente. Se lo relaciona con la regla de Pareto: como aproximación orientativa, una proporción elevada de defectos puede concentrarse en una parte reducida del software.

Por eso, **las funcionalidades que muestran numerosos problemas merecen mayor atención durante las pruebas**, sin abandonar el resto del producto.

5. PARADOJA DEL PESTICIDA

Si se ejecutan siempre los mismos casos de prueba sin revisarlos, pueden dejar de descubrir defectos nuevos. Por eso, **los casos de prueba deben revisarse, actualizarse y complementarse** con nuevos escenarios, datos y técnicas a medida que evoluciona el software.

6. EL TESTING DEPENDE DEL CONTEXTO

No todos los sistemas necesitan el mismo tipo ni la misma intensidad de pruebas. Las estrategias deben adaptarse a los requerimientos, riesgos, tecnología, criticidad y condiciones del producto. Por ejemplo, probar un sistema bancario exige considerar riesgos diferentes de los que presenta una aplicación de entretenimiento.

7. FALACIA DE LA AUSENCIA DE ERRORES

Un software con pocos defectos encontrados no necesariamente es útil o adecuado para sus usuarios. Puede funcionar conforme a los requerimientos documentados y, sin embargo, no satisfacer las necesidades reales del negocio.

Por eso, el Testing debe considerar tanto el funcionamiento correcto como la adecuación del producto a su propósito.

OTRAS PAUTAS CLÁSICAS PARA REALIZAR PRUEBAS

El enfoque tradicional presentado en el material también destaca que:

- **Todo caso de prueba debe tener un resultado esperado previamente definido**, para poder reconocer cuándo el comportamiento real es incorrecto.
- **Se debe evitar depender exclusivamente de las pruebas realizadas por el propio desarrollador o su equipo**, porque pueden repetir las mismas interpretaciones equivocadas. Una perspectiva independiente ayuda a detectar problemas, aunque los desarrolladores también realizan pruebas, especialmente unitarias.
- **Deben probarse entradas válidas e inválidas**, situaciones esperadas e inesperadas, y comprobarse tanto lo que el software debe hacer como aquello que no debería permitir.
- **Es necesario inspeccionar cuidadosamente los resultados**, porque una misma ejecución puede revelar varios comportamientos incorrectos.
- **Conviene conservar casos de prueba reproducibles y reutilizables**, evitando depender exclusivamente de pruebas improvisadas.
- **No debe planificarse suponiendo que no aparecerán defectos**. Diseñar buenas pruebas exige creatividad y esfuerzo intelectual.

MITOS DEL TESTING

Existen afirmaciones incorrectas o incompletas sobre las pruebas de software:

- **«El Testing comienza cuando termina la codificación».** Falso: La planificación y el diseño de pruebas pueden comenzar desde los requerimientos.
- **«El Testing demuestra que el software funciona correctamente».** Incompleto: También busca descubrir comportamientos incorrectos y situaciones que el software no debería permitir.
- **«Testing equivale a calidad del producto».** Falso: Las pruebas evalúan características del producto, pero no generan por sí solas toda su calidad.
- **«Testing equivale a calidad del proceso».** Falso: Probar el software no demuestra que el proceso de desarrollo sea adecuado.
- **«El tester es el enemigo del programador».** Falso: Ambos buscan mejorar el producto. Los defectos deben tratarse como información sobre el software, no como una crítica personal.

¿CUÁNTO TESTING ES SUFICIENTE?

Como **el Testing exhaustivo es imposible**, debe determinarse cuánto esfuerzo resulta razonable dedicar a las pruebas.

La decisión depende principalmente de:

- **Riesgos:** Probabilidad e impacto de posibles fallas.
- **Costos:** Recursos necesarios para continuar probando frente a las consecuencias de liberar el producto con defectos.
- **Criticidad de las funcionalidades:** Importancia de cada parte del sistema para el usuario y el negocio.
- **Resultados y cobertura alcanzada:** Qué se probó, con qué resultados y qué riesgos permanecen.

Los riesgos permiten decidir **qué probar primero, dónde concentrar más esfuerzo y qué pruebas pueden postergarse**.

CRITERIOS DE ACEPTACIÓN O FINALIZACIÓN DE LAS PRUEBAS

Para determinar cuándo una fase o ciclo de Testing puede considerarse terminado se establecen criterios previamente acordados. Pueden expresarse en términos de:

- Presupuesto o esfuerzo máximo de pruebas.
- Porcentaje de casos ejecutados satisfactoriamente.
- Cobertura alcanzada sobre los requisitos o el software.
- Ausencia de defectos abiertos de determinada severidad.
- Cantidad y gravedad de los defectos que todavía permanecen.
- Riesgo residual aceptable para liberar la versión.

Puede aplicarse un criterio de **Good Enough**, aceptando ciertos defectos de baja gravedad cuando su impacto es tolerable y el costo de continuar corrigiéndolos no se justifica frente al beneficio.

Finalizar el Testing no significa que el producto no tenga defectos, sino que se alcanzaron los criterios acordados y que los riesgos pendientes se consideran aceptables para la decisión correspondiente.

Estos criterios de finalización del proceso de pruebas se relacionan con los criterios de aceptación funcional de los requerimientos, pero no son exactamente lo mismo.

ERROR, DEFECTO Y FALLA

Según la explicación utilizada por la cátedra, se distinguen por el momento en que se identifica un problema y por sus consecuencias.

- **Error:** Equivocación detectada y corregida en la misma etapa en la que fue introducida.
- **Defecto:** Equivocación que no se detectó en su etapa de origen y se propagó a una etapa posterior.
- **Falla:** Comportamiento incorrecto o mal funcionamiento que se manifiesta al ejecutar el software debido a un defecto.

Por ejemplo, si un requerimiento incorrecto se detecta durante la Ingeniería de Requerimientos, se trata de un error identificado en esa etapa. Si llega sin corregirse hasta la implementación y se descubre durante Testing, se considera un defecto propagado.

No todos los defectos producen una falla observable durante las pruebas. Algunos permanecen ocultos porque no se ejecutan las condiciones que los activan; otros afectan aspectos de presentación sin impedir el funcionamiento principal.

Esta distinción conserva el criterio de propagación enseñado en clase. En la terminología general de Testing también se utiliza «error» para una equivocación humana y «defecto» para la anomalía introducida en el producto.

SEVERIDAD Y PRIORIDAD DE LOS DEFECTOS

Son dos características distintas que se utilizan para clasificar y ordenar el tratamiento de los defectos encontrados.

Severidad: Indica **qué tan grave es el defecto desde una perspectiva técnica o funcional**. Generalmente la determina quien realiza el Testing. Se utilizan cinco niveles:

1. **Bloqueante:** Impide continuar la utilización del sistema o una actividad esencial.
2. **Crítico:** impide completar una funcionalidad importante del sistema, sin disponer de una alternativa que permita alcanzar el objetivo.
3. **Mayor:** Impide completar una funcionalidad por el procedimiento esperado, pero existe una alternativa que permite alcanzar el objetivo
4. **Menor:** Genera un comportamiento incorrecto de bajo impacto, sin impedir completar la tarea principal.
5. **Cosmético:** Afecta principalmente cuestiones de presentación, formato o distribución de elementos.

Prioridad: Indica **con qué urgencia debe corregirse el defecto**, considerando las necesidades del negocio y del cliente. La escala utilizada es: **Urgencia, Alta, Media y Baja**.

La prioridad debe establecerse teniendo en cuenta la importancia de la funcionalidad, los usuarios afectados, los compromisos y la cercanía de una entrega. Puede acordarse entre los responsables del producto, del proyecto y del equipo.

Por ejemplo, un defecto cosmético puede tener baja severidad técnica, pero alta prioridad si afecta la presentación principal de un producto cuyo valor comercial depende de su diseño visual.

CASOS DE PRUEBA

Un **caso de prueba es un conjunto de condiciones, datos y acciones definidos para comprobar un comportamiento específico del software y compararlo con un resultado esperado**.

Es uno de los artefactos centrales del Testing sistemático, porque permite **ejecutar y reproducir las pruebas**, registrar los resultados y comprobar posteriormente si un defecto fue corregido.

COMPONENTES DE UN CASO DE PRUEBA

Se distinguen tres componentes principales:

1. **Objetivo:** Característica, funcionalidad o comportamiento que se desea comprobar.
2. **Datos de entrada y ambiente:** Valores, precondiciones y configuración necesarios para ejecutar la prueba.
3. **Comportamiento esperado:** Resultado o acción que debería producir el sistema de acuerdo con los requerimientos.

Para que la prueba sea reproducible también conviene especificar **los pasos concretos de ejecución**, la versión del software y, cuando corresponda, los datos utilizados. **El resultado esperado debe definirse antes de ejecutar el caso**, para evitar considerar correcto un resultado solamente porque parece razonable.

CONDICIONES DE PRUEBA

Una **condición de prueba** identifica una respuesta o comportamiento esperado del sistema frente a determinados estímulos o datos de entrada.

Por ejemplo, «el sistema debe rechazar una contraseña incorrecta» es una condición de prueba. Los casos de prueba concretan los datos, pasos y resultados esperados necesarios para comprobarla.

Una condición puede requerir varios casos de prueba, y **cada condición identificada debe comprobarse mediante al menos un caso de prueba**.

ORIGEN Y DISEÑO DE LOS CASOS DE PRUEBA

Los casos pueden derivarse de distintas fuentes:

- Documentación y necesidades del cliente.
- Información obtenida durante el relevamiento.
- Requerimientos funcionales y no funcionales.
- Especificaciones técnicas o de programación.
- Código fuente y estructura del software.

En los enfoques ágiles también pueden basarse en **User Stories, criterios de aceptación y conversaciones con el negocio**.

Ninguna técnica de diseño de casos es completa por sí sola. Por eso, se deben combinar diferentes métodos y priorizar los escenarios relevantes según sus riesgos. El objetivo es **maximizar la capacidad de detectar defectos y cubrir situaciones importantes con una cantidad razonable de casos**, considerando las restricciones de tiempo y recursos.

PROCESO DE TESTING

El Testing se organiza **mediante un proceso definido** para planificar, preparar, ejecutar, evaluar y comunicar las pruebas. Está compuesto por cuatro actividades principales.

1. PLANIFICACIÓN Y CONTROL

Consiste en **definir los objetivos, riesgos, estrategias, recursos y criterios que orientarán las pruebas**. La planificación comienza tempranamente, a partir de los requerimientos y las necesidades del producto, sin esperar necesariamente a disponer del código.

Su principal resultado es el **Plan de Testing (Test Plan)**, que puede incluir:

- Objetivos y riesgos que se desean abordar.
- Estrategia, niveles y técnicas de prueba.
- Recursos, responsabilidades y ambientes necesarios.
- Criterios de aceptación o finalización de las pruebas.

El control acompaña a las demás actividades: se monitorean los resultados, la cobertura y el cumplimiento de los criterios, tomando decisiones cuando aparecen desviaciones.

2. IDENTIFICACIÓN Y ESPECIFICACIÓN

Se determina **qué debe probarse y cómo se diseñarán las pruebas**. Comprende:

- Revisar las bases de prueba, como requerimientos y especificaciones.
- Identificar ambigüedades y evaluar si el software puede probarse adecuadamente.
- Definir condiciones, datos y casos de prueba.
- Establecer los resultados esperados.
- Priorizar los casos según los riesgos.
- Diseñar las características del ambiente necesario para probar.

Su resultado es un conjunto de **casos de prueba especificados y priorizados**.

3. PREPARACIÓN Y EJECUCIÓN

Se preparan los datos, el ambiente y las herramientas necesarias, automatizando las pruebas cuando corresponda. Luego se ejecutan los casos sobre una **versión identificada del software**, siguiendo los pasos establecidos y comparando los resultados reales con los esperados.

Se registra qué se probó, sobre qué versión, con qué datos y cuál fue el resultado. Cuando se detecta una diferencia, se analiza y reporta el defecto, proporcionando información suficiente para reproducirlo y corregirlo.

4. EVALUACIÓN Y REPORTE

Se analizan los resultados obtenidos y se determina si se alcanzaron los criterios acordados. Comprende:

- Evaluar el cumplimiento de los criterios de aceptación y la cobertura.
- Verificar el estado de los defectos y sus correcciones.
- Comunicar los resultados a los interesados.
- Consolidar la información y los entregables de las pruebas.
- Analizar las lecciones aprendidas para mejorar el proceso.

El Testing termina cuando se alcanza el criterio de finalización acordado, no cuando se demuestra la ausencia absoluta de defectos.

CICLOS DE PRUEBA

Un **ciclo de prueba es la ejecución del conjunto de casos de prueba establecidos sobre una misma versión del sistema**.

En la explicación de clase se denomina **ciclo 0** al primero. Cuando se encuentran defectos, estos se reportan, se corrigen y se entrega una nueva versión para ejecutar nuevas comprobaciones.

Por eso, un producto puede atravesar varios ciclos de prueba antes de considerarse aceptable.

Una cantidad excesiva de ciclos puede indicar problemas de calidad, dificultades en las correcciones o una preparación insuficiente de las pruebas, generando mayor retrabajo y costo.

El nombre «ciclo 0» y la cantidad habitual de ciclos son convenciones del ejemplo de la cátedra, no reglas universales. Tampoco es obligatorio que el primer ciclo sea siempre manual: la automatización puede comenzar antes.

REPRUEBA Y PRUEBA DE REGRESIÓN

Cuando se modifica el software para corregir un defecto, deben considerarse dos necesidades diferentes.

REPRUEBA O PRUEBA DE CONFIRMACIÓN

Consiste en **volver a ejecutar el caso que detectó un defecto**, para verificar que la corrección aplicada haya solucionado ese problema.

Por ejemplo, si fallaba el inicio de sesión con una contraseña válida, se vuelve a ejecutar ese caso después de la corrección.

PRUEBA DE REGRESIÓN

Consiste en **comprobar que una modificación no haya introducido defectos en funcionalidades que antes funcionaban correctamente**. Una corrección puede afectar otros componentes, datos o comportamientos relacionados, aunque estos no hayan sido modificados directamente.

Por eso, **confirmar que se corrigió un defecto no garantiza que el resto del sistema siga funcionando**.

CICLOS CON REGRESIÓN Y SIN REGRESIÓN

La presentación compara dos formas de organizar los ciclos posteriores al inicial:

- **Sin regresión:** Se vuelven a ejecutar solamente los casos relacionados con los defectos que se intentaron corregir. Permite confirmar las correcciones rápidamente, pero puede dejar problemas nuevos sin detectar.
- **Con regresión completa:** Se ejecutan nuevamente todos los casos establecidos, incluidos los que habían pasado anteriormente, para descubrir efectos colaterales de las modificaciones.

La **regresión completa aporta mayor cobertura, pero requiere más tiempo y esfuerzo**.

También pueden seleccionarse pruebas de regresión según los componentes afectados y los riesgos. Esto permite un equilibrio entre cobertura y costo, aunque no proporciona la misma confianza que repetir todo el conjunto.

AUTOMATIZACIÓN DE LA REGRESIÓN

La regresión es especialmente adecuada para automatizarse porque implica ejecutar repetidamente casos ya definidos.

La automatización requiere una inversión inicial en scripts, datos, herramientas y mantenimiento, pero facilita ejecutar los mismos controles muchas veces con menor esfuerzo manual.

Por eso, **la automatización favorece la ejecución frecuente de pruebas de regresión**, especialmente cuando se realizan modificaciones e integraciones continuas.

No elimina por completo el costo del Testing ni hace innecesaria la revisión de los casos automatizados.

NIVELES DE PRUEBA

Los **niveles de prueba determinan el alcance o granularidad del objeto que se evalúa**, desde componentes individuales hasta el sistema completo y su aceptación por parte de los usuarios.

La **secuencia habitual avanza desde unidades pequeñas hacia conjuntos progresivamente más integrados**, aunque los niveles pueden organizarse de manera diferente según el proceso adoptado.

1. PRUEBAS UNITARIAS

Comprueban **el comportamiento de componentes individuales**, como métodos, funciones o clases, de forma independiente de otros componentes en la medida de lo posible.

Sus características principales son:

- Generalmente las realizan los desarrolladores durante la construcción.
- Se ejecutan con acceso al código y apoyo de herramientas de desarrollo, depuración y frameworks de pruebas.
- Permiten localizar problemas en unidades pequeñas.
- Los errores encontrados suelen corregirse rápidamente, sin requerir siempre un registro formal de incidentes.

En enfoques ágiles, las pruebas unitarias suelen ejecutarse frecuentemente y pueden formar parte de la **Definition of Done**, según los criterios establecidos por el equipo.

2. PRUEBAS DE INTEGRACIÓN

Comprueban **que los componentes que funcionan correctamente de manera individual también funcionen al interactuar entre sí**. Su foco principal está en las interfaces, las comunicaciones y el intercambio de información entre componentes.

La integración suele realizarse **incrementalmente**, incorporando componentes de manera progresiva para facilitar la identificación de problemas.

Existen dos estrategias principales:

- **Top-down (de arriba hacia abajo):** comienza por los componentes de mayor nivel y agrega progresivamente los de niveles inferiores.
- **Bottom-up (de abajo hacia arriba):** comienza por los componentes de menor nivel y avanza hacia los superiores.

Pueden combinarse ambas estrategias y conviene probar tempranamente los componentes críticos.

Las pruebas de integración permiten detectar problemas que no aparecen en las pruebas unitarias, como incompatibilidades de interfaces o intercambios incorrectos de datos.

3. PRUEBAS DE SISTEMA O DE VERSIÓN

Evalúan **el sistema integrado como un todo**, para determinar si funciona satisfactoriamente y cumple los requerimientos establecidos. Comprenden pruebas de características funcionales y no funcionales, como:

- Comportamiento de las funcionalidades.
- Rendimiento y tiempos de respuesta.
- Seguridad.
- Recuperación ante fallas.
- Comportamiento bajo determinadas cargas o condiciones de operación.

El ambiente de prueba debe **parecerse lo más posible al ambiente de producción**, para reducir el riesgo de problemas que no se detectaron debido a diferencias de configuración.

Es recomendable que estas pruebas sean realizadas por personas distintas de quienes desarrollaron el producto, para incorporar una perspectiva más objetiva. **No obstante, no es una exigencia universal que deban ser externos a la organización.**

Se utilizan casos de prueba reproducibles y se registran formalmente los defectos encontrados.

4. PRUEBAS DE ACEPTACIÓN DE USUARIO

Buscan determinar **si el producto satisface las necesidades y expectativas del usuario o cliente**, y si resulta aceptable para su utilización. Su propósito principal es **generar confianza y contribuir a la decisión de aceptar el producto**, más que concentrarse exclusivamente en descubrir defectos.

Deben involucrar a usuarios, clientes o representantes del negocio, aunque puedan recibir apoyo del equipo de Testing y de otros responsables. Se distinguen:

- **Pruebas alfa:** Realizadas en un ambiente controlado o de laboratorio, con participación de usuarios.
- **Pruebas beta:** Realizadas con usuarios en condiciones reales de utilización.

En los enfoques ágiles, los criterios de aceptación se definen y comprueban sobre los incrementos de producto.

En Scrum, **las pruebas de aceptación pueden realizarse durante la Sprint Review**, con participación del usuario o cliente, para comprobar que el incremento cumple sus necesidades y criterios de aceptación. Sin embargo, la Sprint Review no se limita a ejecutar pruebas: también permite inspeccionar el incremento y obtener retroalimentación para adaptar el Product Backlog.

COMPARACIÓN DE LOS NIVELES

Nivel	Qué se prueba	Objetivo principal	Responsable habitual
Unitario	Componente individual	Comprobar su funcionamiento aislado	Desarrollador
Integración	Componentes relacionados	Detectar problemas en sus interacciones	Desarrolladores o responsables técnicos/de integración
Sistema	Sistema integrado	Evaluar funcionamiento global y requerimientos	Equipo de Testing, preferentemente con cierta independencia
Aceptación	Producto desde la perspectiva del usuario	Comprobar adecuación a necesidades y decidir su aceptación	Usuario, cliente o representante del negocio

MODELO EN V

El **Modelo en V relaciona las actividades de definición y construcción del software con los niveles de prueba destinados a comprobar sus resultados**.

En el lado izquierdo de la V, el desarrollo avanza desde las necesidades generales hacia especificaciones y componentes cada vez más detallados.

En el lado derecho, las pruebas avanzan en sentido inverso: **desde los componentes individuales hacia la integración, el sistema completo y la aceptación del usuario**.

CORRESPONDENCIA ENTRE DESARROLLO Y PRUEBAS

El modelo permite identificar **contra qué especificación debe comprobarse cada nivel de prueba**. Estas correspondencias representan la relación principal entre desarrollo y Testing, aunque un nivel de prueba también puede comprobar requisitos definidos en otros niveles.

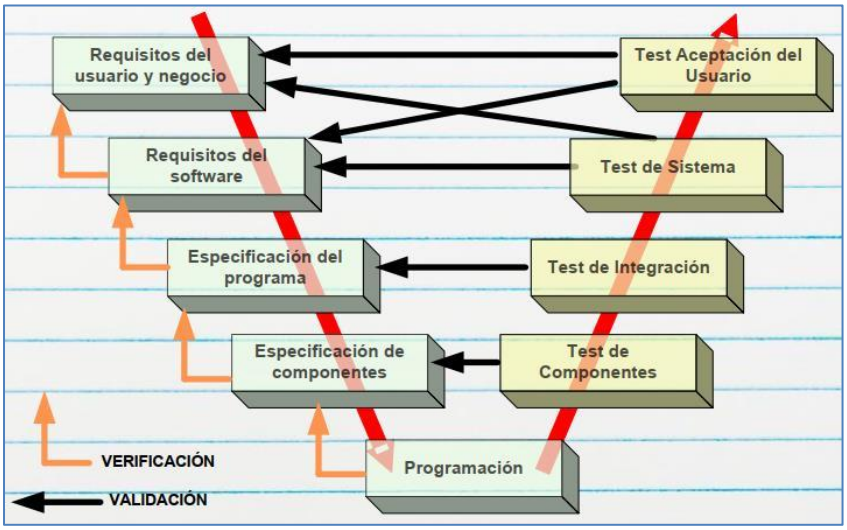
Además, el diagrama representa dos actividades complementarias:

- **Verificación:** comprobar que los productos de trabajo se construyan correctamente respecto de las especificaciones correspondientes.
- **Validación:** comprobar que el software satisfaga las necesidades y expectativas para las cuales fue desarrollado.

Las pruebas pueden **planificarse desde el momento en que se definen las especificaciones correspondientes**, sin esperar a terminar toda la codificación.

Aunque el Modelo en V representa una organización tradicional del ciclo de vida, su lógica también resulta útil en desarrollos iterativos e incrementales.

Idea central: El desarrollo avanza de lo general a lo particular; las pruebas, de lo particular a lo general. Cada nivel de prueba se relaciona con las especificaciones que permiten evaluar lo construido.



AMBIENTES DE TESTING

Un **ambiente de Testing es el conjunto de recursos y configuraciones de hardware y software necesarios para desarrollar, ejecutar o probar un producto**. Puede incluir sistemas operativos, bases de datos, servidores, redes, bibliotecas, herramientas y datos.

Durante el desarrollo se utilizan distintos ambientes, cada uno con un propósito y condiciones de acceso particulares.

1. AMBIENTE DE DESARROLLO

Es el entorno donde los desarrolladores construyen y modifican el software. Contiene las herramientas necesarias para programar, compilar y ejecutar componentes, como IDE, librerías, frameworks y bases de datos de desarrollo.

Aquí se realizan principalmente **pruebas unitarias y, frecuentemente, pruebas de integración**.

2. AMBIENTE DE PRUEBAS

Es un ambiente controlado donde el equipo de Testing evalúa una versión del producto, buscando detectar defectos sin afectar a los usuarios reales. Se realizan principalmente **pruebas de sistema**, tanto funcionales como no funcionales.

Conviene mantener una separación de responsabilidades y permisos para evitar modificaciones no controladas de la versión que se está probando. Esto no implica prohibir absolutamente cualquier acceso de los desarrolladores.

3. AMBIENTE DE PREPRODUCCIÓN

Busca **reproducir lo más fielmente posible las condiciones del ambiente productivo**, incluyendo configuraciones, servicios, integraciones y datos representativos. Permite comprobar el comportamiento del software antes de su liberación y realizar pruebas de aceptación en condiciones cercanas a las reales.

Su principal dificultad es el **costo y la complejidad de replicar el entorno productivo**, especialmente cuando involucra infraestructuras grandes o múltiples servicios.

4. AMBIENTE DE PRODUCCIÓN

Es el entorno donde el software se encuentra disponible para sus usuarios reales y se utiliza para realizar las actividades del negocio. Los cambios deben introducirse con controles adecuados para reducir los riesgos de interrupciones o pérdida de información.

En producción pueden realizarse comprobaciones controladas posteriores al despliegue y, en determinados casos, pruebas beta con usuarios reales. Sin embargo, no conviene utilizarla para ejecutar pruebas potencialmente destructivas que podrían realizarse en otros ambientes.

RELACIÓN ENTRE AMBIENTES Y NIVELES DE PRUEBA

Ambiente	Pruebas habituales
Desarrollo	Unitarias e integración
Pruebas	Sistema, funcionales y no funcionales
Preproducción	Aceptación y comprobaciones previas a la liberación
Producción	Pruebas beta y comprobaciones controladas

Esta relación es orientativa: **un nivel de prueba no está necesariamente limitado a un único ambiente**. La separación permite controlar mejor las condiciones de evaluación, proteger a los usuarios y detectar problemas antes de liberar una nueva versión.

ARTEFACTOS DEL TESTING

El proceso de Testing utiliza distintos artefactos para **planificar, ejecutar, registrar y comunicar las pruebas**.

PLAN DE PRUEBAS (TEST PLAN)

Es el documento que define cómo se organizarán y realizarán las pruebas de un producto. Puede incluir objetivos y riesgos, estrategia, recursos, herramientas, responsables, ambientes, datos necesarios y criterios de finalización. También establece cómo se gestionarán las pruebas de regresión.

Puede elaborarse a partir de los requerimientos, antes de disponer del código, y actualizarse a medida que evoluciona el proyecto.

CASOS DE PRUEBA

Describen **qué se desea comprobar, bajo qué condiciones, con qué datos y pasos, y cuál es el resultado esperado**. Deben ser claros y reproducibles para poder repetir la prueba y comparar los resultados obtenidos.

REPORTE DE INCIDENTES O DEFECTOS

Registra los problemas observados durante las pruebas y proporciona la información necesaria para investigarlos y corregirlos. Habitualmente incluye:

- Identificación del incidente y versión del software.
- Ambiente, datos y condiciones en las que ocurrió.
- Pasos para reproducir el problema.
- Resultado esperado y resultado obtenido.
- Severidad, prioridad y estado del incidente.

La reproducibilidad es una característica deseable, pero un problema difícil de reproducir no deja de ser un posible defecto: requiere mayor investigación.

INFORME FINAL DE TESTING

Resume **el trabajo realizado y los resultados obtenidos** para apoyar las decisiones sobre la calidad y liberación del producto. Puede incluir casos ejecutados y aprobados, cobertura, defectos encontrados y pendientes, cantidad de ciclos, técnicas utilizadas, riesgos residuales y cumplimiento de los criterios de finalización.

También puede aportar información estadística para mejorar futuros procesos de prueba. Su formalidad y obligatoriedad dependen de los procedimientos y acuerdos establecidos en cada proyecto.

TESTING AD HOC Y EXPLORATORIO

El **Testing ad hoc** consiste en realizar pruebas sin una planificación o estructura formal previamente definida. Puede permitir descubrir problemas rápidamente, pero la ausencia de registros dificulta reproducir los hallazgos y conocer qué se ha cubierto.

No debe confundirse con el **Testing exploratorio**, donde el tester aprende sobre el producto, diseña y ejecuta pruebas de manera simultánea. La exploración puede organizarse mediante objetivos, sesiones y registros de resultados.

EL TESTING Y EL CICLO DE VIDA

La organización de las pruebas depende del ciclo de vida de desarrollo adoptado.

En un **enfoque tradicional por fases**, aunque las pruebas pueden planificarse tempranamente, las pruebas de sistema suelen concentrarse después de la implementación e integración. Esto puede retrasar el descubrimiento de defectos y aumentar el retrabajo.

En un **enfoque ágil, iterativo e incremental**, las pruebas acompañan la construcción de cada incremento. Desarrollo y Testing trabajan de manera más integrada, permitiendo obtener retroalimentación y corregir problemas con mayor frecuencia.

En ambos casos siguen siendo necesarias la planificación, ejecución, evaluación y documentación de los resultados. La diferencia principal está en **cómo se distribuyen e integran estas actividades durante el desarrollo**, no en que un enfoque tenga Testing y el otro no.

ESTRATEGIAS DE PRUEBA

Las estrategias de prueba orientan **cómo seleccionar y diseñar casos que permitan detectar defectos y alcanzar una cobertura adecuada con recursos limitados**. Se distinguen dos enfoques principales: **Caja Negra y Caja Blanca**.

Una estrategia o técnica no debe confundirse con un nivel de prueba: los niveles indican **qué alcance del software se evalúa**, mientras que las técnicas indican **cómo se diseñan los casos de prueba**.

1. CAJA NEGRA

Las pruebas de **Caja Negra** se diseñan a partir del comportamiento esperado del software, **sin necesitar conocer su estructura interna**. Se observa la funcionalidad mediante sus entradas, condiciones y resultados, comparando las salidas obtenidas con las esperadas.

Su principal ventaja es que permite comprobar el comportamiento desde la perspectiva de los requerimientos y del usuario. Su limitación es que **no permite conocer directamente qué instrucciones o caminos internos fueron ejecutados**, por lo que pueden quedar partes del código sin cubrir.

Se distinguen técnicas **basadas en especificaciones** y **basadas en experiencia**.

TÉCNICAS BASADAS EN ESPECIFICACIONES

Utilizan requerimientos y especificaciones del sistema como referencia para diseñar las pruebas.

A. PARTICIÓN DE EQUIVALENCIAS

Consiste en **dividir el conjunto de posibles datos de entrada en clases de equivalencia**, cuyos valores deberían producir un comportamiento equivalente del sistema. En lugar de probar todos los valores de cada clase, se seleccionan **representantes** para diseñar los casos de prueba.

Se distinguen clases:

- **Válidas:** datos que satisfacen las reglas establecidas para la funcionalidad.
- **Inválidas:** datos que no cumplen esas reglas y deberían provocar un tratamiento de error o rechazo.

Su procedimiento consiste en:

1. **Identificar las condiciones externas** que intervienen en la funcionalidad, como entradas, selecciones, fechas o resultados esperados.

2. **Identificar las clases de equivalencia válidas e inválidas**, según las reglas del sistema.

3. **Diseñar casos de prueba representativos**, seleccionando valores concretos que permitan comprobar el comportamiento de cada clase.

Las clases deben cubrir todos los valores relevantes de la condición analizada y evitar superposiciones innecesarias. Pueden existir clases basadas en **rangos continuos, valores discretos, selección simple y selección múltiple**.

Esta técnica permite **reducir la cantidad de pruebas conservando una cobertura representativa**, pero no garantiza descubrir todos los defectos dentro de una clase.

B. ANÁLISIS DE VALORES LÍMITE

Es una técnica relacionada con la partición de equivalencias que selecciona **valores ubicados en los límites de las clases**, porque allí suelen producirse errores.

En lugar de elegir únicamente un valor cualquiera de cada grupo, se prueban valores inmediatamente anteriores, iguales y posteriores al límite, cuando sean aplicables. Por ejemplo, si la edad mínima permitida es 18 años, pueden utilizarse:

- **17:** inmediatamente por debajo del límite.
- **18:** exactamente en el límite.
- **19:** inmediatamente por encima del límite.

Esto permite comprobar si se utilizan correctamente las condiciones de comparación.

Diferencia principal: La partición de equivalencias selecciona representantes de clases; el análisis de valores límite concentra las pruebas en sus fronteras.

TÉCNICAS BASADAS EN EXPERIENCIA

Se apoyan en los conocimientos, la experiencia y la creatividad del tester.

A. ADIVINANZA DE DEFECTOS

Consiste en **anticipar situaciones propensas a fallar** basándose en la experiencia, los errores frecuentes y el conocimiento del producto. Por ejemplo, probar campos vacíos, datos duplicados, fechas inválidas o una operación repetida varias veces.

Es útil para complementar otras técnicas, especialmente cuando se conocen problemas habituales del sistema o de tecnologías similares.

B. TESTING EXPLORATORIO

Consiste en **aprender sobre el software mientras se diseñan y ejecutan las pruebas**.

El tester observa los resultados, formula nuevas preguntas y adapta los siguientes escenarios según lo que descubre.

Favorece el descubrimiento de situaciones no contempladas inicialmente y aprovecha especialmente la experiencia y creatividad del evaluador.

No requiere que todos los casos estén definidos de antemano, aunque puede mantener registros para conservar la trazabilidad de los hallazgos.

2. CAJA BLANCA

Las pruebas de **Caja Blanca** se diseñan mediante el análisis de la **estructura interna del software**, como su código fuente, pseudocódigo o grafos de flujo de control.

Permiten conocer qué instrucciones, decisiones, condiciones o caminos del programa deben ejecutarse para alcanzar un criterio de cobertura determinado.

Su principal ventaja es que facilitan la **medición de cobertura estructural**. Su limitación es que **recorrer correctamente la estructura existente no garantiza que el programa cumpla los requerimientos**, ni permite descubrir por sí solo funcionalidades que deberían existir, pero no fueron implementadas.

COBERTURA DE CAMINOS BÁSICOS Y COMPLEJIDAD CICLOMÁTICA

Esta técnica busca identificar un conjunto de **caminos independientes del flujo de control** que permitan recorrer las principales alternativas lógicas del programa. Para analizar el flujo se puede construir un grafo en el que los nodos representan operaciones o decisiones y las aristas representan posibles transferencias de control.

Se utiliza la **complejidad ciclomática (M)**, que indica la cantidad de caminos linealmente independientes del grafo. Puede calcularse mediante: **M = E – N + 2P**. Donde:

- **E:** cantidad de aristas del grafo.
- **N:** cantidad de nodos.
- **P:** cantidad de componentes conexos del grafo; normalmente P = 1 cuando se analiza un único grafo conectado.

Para un grafo de flujo plano y conectado también puede calcularse como: **M = cantidad de regiones interiores + 1**

En el ejemplo de la cátedra: E = 12 aristas; N = 10 nodos; P = 1 componente conexo. Entonces: **M = 12 − 10 + 2(1) = 4**. El resultado indica que deben identificarse **cuatro caminos independientes para la cobertura de caminos básicos**.

Luego se diseñan casos de prueba con datos concretos que permitan ejecutar esos caminos, siempre que sean factibles.

Importante: La complejidad ciclomática no indica la cantidad mínima universal de casos de prueba para cualquier tipo de cobertura. Su interpretación corresponde específicamente al análisis de caminos independientes. Una complejidad elevada también puede indicar mayor dificultad de comprensión, prueba y mantenimiento, aunque no demuestra por sí sola que el código tenga defectos.

COBERTURA DE SENTENCIAS

Busca **ejecutar al menos una vez cada sentencia o instrucción ejecutable** del programa. Por ejemplo, si existen dos instrucciones de asignación dentro de decisiones independientes, puede bastar un caso de prueba que permita ejecutar ambas. Su principal limitación es que **ejecutar todas las sentencias no garantiza haber recorrido todas las alternativas de decisión**.

COBERTURA DE DECISIONES

Busca **ejecutar todas las ramas o resultados posibles de las decisiones**.

En una decisión if, se debe comprobar tanto el resultado verdadero como el falso, cuando ambos sean alcanzables.

Por ejemplo, para una condición que decide si se permite una compra, deben probarse escenarios en los que la compra sea permitida y otros en los que sea rechazada.

Esta cobertura verifica las alternativas del flujo de control, pero no necesariamente todas las condiciones individuales que forman una decisión compuesta.

COBERTURA DE CONDICIONES

Busca que **cada condición lógica individual que integra una decisión tome los valores verdadero y falso** durante las pruebas. Por ejemplo, en: if (A && B) deben diseñarse pruebas que permitan evaluar tanto A como B en verdadero y falso.

Esta cobertura se concentra en las condiciones individuales y **no garantiza por sí sola que se hayan cubierto todos los resultados posibles de la decisión completa**. En expresiones con evaluación de cortocircuito, hay que tener en cuenta si una condición llega efectivamente a evaluarse.

COBERTURA DE DECISIÓN-CONDICIÓN

Combina los criterios anteriores:

- **Cada decisión** debe evaluarse en sus resultados verdadero y falso.
- **Cada condición individual** debe evaluarse en verdadero y falso.

Busca una cobertura más exigente al comprobar tanto las ramas de las decisiones como las condiciones que las componen. No implica necesariamente probar todas las combinaciones posibles entre las condiciones.

COBERTURA MÚLTIPLE

Busca **probar todas las combinaciones posibles de valores de verdad de las condiciones** que forman una decisión, considerando las combinaciones factibles. **Es una cobertura más exigente y potencialmente costosa**, especialmente cuando aumenta la cantidad de condiciones.

Por ejemplo, para una decisión con dos condiciones booleanas independientes, existen cuatro combinaciones posibles de valores de verdad. Eso permite entender inmediatamente por qué esta cobertura puede resultar costosa.

COMPARACIÓN ENTRE COBERTURAS

Cobertura	Qué busca comprobar
Sentencias	Ejecutar cada instrucción al menos una vez.
Decisiones	Ejecutar las alternativas verdadero y falso de cada decisión.
Condiciones	Evaluar cada condición individual en verdadero y falso.
Decisión-condición	Combinar la cobertura de las decisiones y de sus condiciones.
Múltiple	Evaluar las combinaciones posibles de las condiciones.
Caminos básicos	Ejecutar un conjunto de caminos independientes del flujo de control.

Estos criterios permiten medir aspectos diferentes de la estructura del código. **Alcanzar un porcentaje elevado de cobertura no garantiza la ausencia de defectos**, porque puede existir un problema en la lógica implementada o faltar funcionalidades requeridas.

COMPARACIÓN ENTRE CAJA NEGRA Y CAJA BLANCA

Aspecto	Caja Negra	Caja Blanca
Base para diseñar pruebas	Requerimientos, especificaciones y comportamiento esperado.	Código, estructura y flujo interno.
Qué evalúa principalmente	Entradas, salidas y funcionalidades.	Instrucciones, decisiones, condiciones y caminos.
Ventaja	Evalúa el comportamiento desde la perspectiva del usuario o requerimiento.	Permite medir cobertura estructural y analizar alternativas internas.
Limitación	Puede dejar partes del código sin ejecutar.	Puede pasar por alto requerimientos faltantes o comportamientos incorrectamente especificados.

Ambos enfoques son complementarios. La cátedra recomienda combinar técnicas porque ninguna resulta completa para todas las situaciones y cada una permite descubrir clases distintas de problemas.

TIPOS DE PRUEBAS

Los tipos de prueba indican **qué característica o propósito se pretende evaluar**. Pueden aplicarse en distintos niveles y utilizando diferentes técnicas de diseño.

SMOKE TEST

Es un **conjunto reducido y rápido de pruebas** que busca comprobar si una versión del software presenta fallas graves o evidentes que impidan comenzar una evaluación más profunda. Se utiliza como comprobación inicial antes de invertir esfuerzo en un ciclo completo de pruebas de sistema.

Por ejemplo, puede verificar que la aplicación inicie, permita acceder a sus funciones principales y no falle inmediatamente al realizar operaciones básicas.

Su objetivo es decidir si la versión está en condiciones de continuar siendo probada, no demostrar que todo el producto funciona correctamente.

TESTING FUNCIONAL

Evalúa **qué hace el software**, comprobando que las funcionalidades se comporten de acuerdo con lo especificado y con las necesidades del negocio. Se distinguen dos enfoques:

- **Basado en requerimientos:** Comprueba funciones o reglas particulares del sistema a partir de especificaciones, casos de uso o criterios de aceptación.
- **Basado en procesos de negocio:** Evalúa un flujo completo que combina varias funcionalidades.

Por ejemplo, en un sistema de ventas, una prueba basada en requerimientos puede comprobar el cálculo del importe de un producto. Una prueba basada en procesos de negocio puede recorrer la búsqueda, selección, facturación y registro de una venta completa.

TESTING NO FUNCIONAL

Evalúa **cómo funciona el sistema**, considerando atributos de calidad como rendimiento, fiabilidad, usabilidad y compatibilidad con diferentes entornos. Estas pruebas pueden depender especialmente de las condiciones del ambiente, los datos y la infraestructura utilizada.

1. PERFORMANCE O RENDIMIENTO

Evalúa el **tiempo de respuesta y el rendimiento del sistema** en condiciones definidas, incluyendo su comportamiento ante usuarios u operaciones concurrentes.

2. CARGA

Evalúa el comportamiento del sistema bajo **volúmenes previstos de trabajo**, como determinada cantidad de usuarios simultáneos o transacciones. Permite observar tiempos de respuesta, capacidad y utilización de recursos de hardware y comunicaciones.

3. ESTRÉS

Somete al sistema a **condiciones superiores a las esperadas** para determinar cómo se comporta al alcanzar o superar sus límites. Permite evaluar su robustez, degradación ante sobrecarga y capacidad de recuperación.

4. VOLUMEN

Comprueba el comportamiento del software al procesar o almacenar **grandes cantidades de datos**. Se diferencia de las pruebas de carga porque el foco está principalmente en el volumen de información, no necesariamente en la concurrencia de usuarios.

5. MANTENIBILIDAD

Evalúa la facilidad para **comprender, corregir, modificar y mantener** el software. Puede considerar su estructura, documentación y modularidad, mediante análisis y otras actividades de evaluación.

6. USABILIDAD

Evalúa si el software resulta **comprensible, fácil de aprender y adecuado para que el usuario realice sus tareas**.

7. PORTABILIDAD

Comprueba si el producto puede **instalarse, trasladarse o funcionar en los diferentes entornos acordados**, con las adaptaciones correspondientes.

8. FIABILIDAD

Evalúa la capacidad del sistema para **mantener un funcionamiento correcto y consistente durante un período y bajo condiciones determinadas**. Puede incluir su disponibilidad, tolerancia a fallos y capacidad de recuperación.

9. CONFIGURACIÓN E INSTALACIÓN

Configuración: comprueba que el software funcione con las combinaciones de hardware, sistema operativo, bases de datos, redes y componentes externos que debe soportar.

Instalación: verifica que el producto pueda instalarse, configurarse o actualizarse correctamente y quedar operativo.

10. PRUEBAS DE INTERFAZ DE USUARIO

Evalúan el comportamiento de la interfaz frente a las diferentes acciones y secuencias posibles del usuario. Pueden abordar aspectos **funcionales**, como la respuesta de botones y formularios, y **no funcionales**, como la facilidad de uso.

Las interfaces gráficas pueden presentar numerosas combinaciones de acciones, por lo que requieren diseñar las pruebas de manera cuidadosa.

DIFERENCIA ENTRE RENDIMIENTO, CARGA Y ESTRÉS

- **Rendimiento:** Cómo responde el sistema en condiciones definidas.
- **Carga:** Cómo responde ante la cantidad prevista de trabajo o usuarios.
- **Estrés:** Qué sucede cuando se supera la carga habitual y cómo se recupera el sistema.

Las pruebas no funcionales **son tan importantes como las funcionales**. Su complejidad o dependencia del ambiente no significa que deban omitirse o reemplazarse únicamente por pruebas de aceptación.

TEST DRIVEN DEVELOPMENT (TDD)

TDD es una técnica de desarrollo que consiste en **escribir primero una prueba para el comportamiento que se desea implementar y luego desarrollar el código necesario para satisfacerla**. Su propósito no es solamente detectar defectos, sino **utilizar las pruebas para orientar el diseño y la construcción del software**.

Se relaciona con dos prácticas:

- **Test First Development:** definir una prueba antes de implementar el comportamiento correspondiente.
- **Refactoring:** mejorar la estructura interna del código sin modificar su comportamiento observable, buscando mayor claridad, simplicidad y mantenibilidad.

CICLO BÁSICO DE TDD

TDD suele organizarse mediante el ciclo **Red–Green–Refactor**:

1. **Red:** Escribir una prueba para el comportamiento deseado y comprobar que falla porque todavía no está implementado.
2. **Green:** Escribir el código mínimo necesario para que la nueva prueba pase, comprobando también que las anteriores sigan funcionando.
3. **Refactor:** Mejorar la estructura del código sin cambiar su comportamiento y volver a ejecutar las pruebas.

Este ciclo se repite en pasos pequeños, proporcionando retroalimentación frecuente durante el desarrollo.

BENEFICIOS Y LIMITACIONES

TDD puede ayudar a aclarar el comportamiento esperado antes de programar, favorecer componentes más simples y facilitar la detección de regresiones mediante pruebas automatizadas.

Sin embargo, **no garantiza automáticamente un código correcto ni reemplaza las pruebas de integración, sistema o aceptación**. Sus beneficios dependen de la calidad de las pruebas y de las decisiones de diseño.

Es una práctica especialmente asociada al desarrollo ágil, aunque **no exige utilizar una metodología ágil determinada**.

10. TESTING EN AMBIENTES ÁGILES

CONTEXTO Y CAMBIO DE ENFOQUE

En los ambientes ágiles, el **Testing acompaña continuamente al desarrollo del software**, participando desde la comprensión de las necesidades hasta la evaluación de los incrementos construidos.

Mientras que el enfoque tradicional pone mayor énfasis en **detectar defectos y comprobar el cumplimiento de especificaciones**, el Testing Ágil busca además **prevenir la introducción de defectos**, colaborando tempranamente en la definición y construcción del producto.

Esto no significa que en Ágil se deje de buscar defectos, sino que el Testing adopta un enfoque más preventivo y colaborativo. La calidad continúa siendo responsabilidad de todos los integrantes del equipo, y las prácticas de SCM, integración continua y automatización permiten mantener la integridad del producto y obtener retroalimentación frecuente.

TESTING TRADICIONAL VS. TESTING ÁGIL

Aspecto	Testing tradicional por fases	Testing ágil
Momento	Las pruebas de sistema suelen concentrarse después de la implementación.	Las pruebas acompañan continuamente la construcción de incrementos.
Objetivo predominante	Detectar defectos y verificar la conformidad con especificaciones.	Prevenir defectos, aclarar expectativas y detectar problemas tempranamente.
Requerimientos	Mayor énfasis en especificaciones previamente definidas.	Colaboración frecuente para comprender y ajustar necesidades y expectativas.
Participación del tester	Puede concentrarse en recibir versiones y evaluarlas.	Participa desde la preparación y comprensión de las historias de usuario.
Retroalimentación	Puede llegar después de introducir el problema, aumentando el retrabajo.	Se busca obtenerla de manera temprana y frecuente.
Responsabilidad por la calidad	Puede concentrarse en un área especializada de Testing o QA.	Es responsabilidad compartida de todo el equipo.

En los desarrollos ágiles se busca evitar una **«minicascada» dentro de cada Sprint**, donde primero se implementa todo y recién al final se realizan las pruebas. En cambio, el análisis de las historias, la programación, las pruebas y la corrección de defectos deben colaborar durante toda la iteración.

El proceso de Testing sigue requiriendo planificación, diseño, ejecución y evaluación. Lo que cambia es cómo se integran y distribuyen estas actividades a lo largo del desarrollo.

MANIFIESTO DEL TESTING ÁGIL

El Manifiesto del Testing Ágil establece **cinco valores**, que expresan prioridades para orientar las actividades de prueba. Al igual que en el Manifiesto Ágil, **valorar un aspecto por encima de otro no significa eliminar el segundo**.

1. PRUEBAS DURANTE EL DESARROLLO SOBRE PRUEBAS AL FINAL

El Testing debe realizarse continuamente, desde la definición de las necesidades hasta la construcción y evaluación del producto. Esto permite detectar problemas cuando todavía resultan más sencillos de corregir, evitando acumularlos hasta el final del desarrollo.

2. PREVENIR DEFECTOS SOBRE ENCONTRAR DEFECTOS

El objetivo no es solamente descubrir problemas cuando ya existen, sino **evitar que se introduzcan**. El tester puede participar en la revisión de historias, criterios de aceptación y ejemplos antes de programar, identificando ambigüedades, contradicciones o situaciones no contempladas.

3. ENTENDER LO QUE SE ESTÁ PROBANDO SOBRE VERIFICAR LA FUNCIONALIDAD

No alcanza con comprobar mecánicamente que una función produce el resultado especificado. Es necesario **comprender el dominio del negocio, las necesidades reales del usuario y el valor que debe aportar la funcionalidad**, para identificar si el producto resuelve correctamente el problema.

4. CONSTRUIR UN MEJOR SISTEMA SOBRE ROMPER EL SISTEMA

Se supera la idea del tester como una persona que intenta destruir o romper el software. El objetivo es **colaborar con los desarrolladores y representantes del negocio para construir un producto mejor**, utilizando los defectos y las pruebas como información para aprender y mejorar.

La búsqueda crítica de defectos sigue siendo necesaria, pero está orientada a un propósito constructivo.

5. RESPONSABILIDAD DE TODO EL EQUIPO POR LA CALIDAD SOBRE EXCLUSIVA DEL TESTER

La calidad no depende únicamente de quien realiza las pruebas. **Todo el equipo debe contribuir a prevenir, detectar y corregir problemas**, cada integrante desde sus conocimientos y responsabilidades.

Esto no elimina la especialización del tester: permite aprovechar sus competencias junto con las de desarrolladores y representantes del negocio.

NUEVE PRINCIPIOS DEL TESTING ÁGIL

Los valores expresan las prioridades generales, mientras que los **principios orientan de manera más específica cómo debería trabajar un equipo**. No constituyen todavía prácticas o procedimientos concretos.

1. **El Testing se mueve hacia adelante en el proyecto:** Participa desde etapas tempranas, ayudando a definir expectativas y proporcionando retroalimentación durante el desarrollo.
2. **El Testing no es una fase:** Es una actividad integrada en el proceso, que acompaña continuamente la construcción de las funcionalidades.
3. **Todos hacen Testing:** La responsabilidad de evaluar y mejorar la calidad es compartida por el equipo, sin eliminar la necesidad de conocimientos especializados.
4. **Reducir la latencia del feedback:** Disminuir el tiempo entre realizar una acción y recibir información sobre sus resultados. Cuanto más rápido se detecta un problema, antes puede corregirse y aprovecharse el aprendizaje.
5. **Las pruebas representan expectativas:** Expresan mediante ejemplos y resultados esperados cómo debería comportarse el sistema, facilitando que negocio, desarrollo y Testing compartan una misma interpretación.
6. **Mantener el código limpio y corregir los defectos rápidamente:** Resolver problemas mientras el contexto todavía está presente y evitar que se acumulen defectos y deuda técnica.
7. **Reducir la sobrecarga de documentación de las pruebas:** Mantener documentación útil, clara y actualizada, evitando registros excesivos que no aportan valor. No significa eliminar la documentación necesaria.
8. **Las pruebas son parte del Done:** Una historia no se considera terminada únicamente porque su código fue implementado; debe cumplir los criterios de calidad y las comprobaciones establecidos en la Definition of Done.
9. **Pasar de probar al final a un desarrollo conducido por pruebas:** Utilizar pruebas y ejemplos para orientar la implementación desde el principio, mediante prácticas como TDD y ATDD.

La idea central es que las pruebas dejan de ser solamente un mecanismo de evaluación posterior y pasan a contribuir activamente a la construcción del producto.

SEIS PRÁCTICAS CONCRETAS DEL TESTING ÁGIL

Las prácticas son **acciones específicas que el equipo implementa para trabajar conforme a los valores y principios**.

1. PRUEBAS UNITARIAS Y DE INTEGRACIÓN AUTOMATIZADAS

Permiten comprobar frecuentemente componentes individuales y sus interacciones, detectando problemas cuando todavía están próximos al cambio que los originó. Su automatización facilita obtener feedback rápido e incorporar las verificaciones a la integración continua.

2. PRUEBAS DE REGRESIÓN AUTOMATIZADAS A NIVEL DE SISTEMA

Permiten volver a comprobar funcionalidades importantes cada vez que se introduce una nueva versión. Buscan descubrir defectos que puedan haberse incorporado como consecuencia de modificaciones, evitando ejecutar manualmente las mismas pruebas repetitivas.

3. PRUEBAS EXPLORATORIAS

El tester **aprende sobre el producto mientras diseña y ejecuta pruebas**, utilizando su experiencia, creatividad y pensamiento crítico para descubrir situaciones no previstas. No se reemplazan completamente mediante automatización, porque dependen de la observación, interpretación y generación de nuevas ideas.

4. TEST DRIVEN DEVELOPMENT (TDD)

Es una técnica de desarrollo en la que se **escriben pruebas antes de implementar el comportamiento correspondiente**, generalmente a nivel unitario.

Se utiliza el ciclo **Red–Green–Refactor**, ya desarrollado en la sección anterior. Su finalidad es orientar el diseño y la implementación del código mediante ciclos cortos de retroalimentación.

5. ACCEPTANCE TEST DRIVEN DEVELOPMENT (ATDD)

Es una técnica que utiliza **pruebas de aceptación definidas antes de programar** para orientar la construcción de funcionalidades de acuerdo con las necesidades del negocio. Se basa en la colaboración entre representantes del negocio, desarrolladores y testers para aclarar las expectativas mediante ejemplos concretos.

6. CONTROL DE VERSIONES DE LAS PRUEBAS JUNTO CON EL CÓDIGO

Las pruebas deben evolucionar junto con el software. Mantenerlas versionadas permite saber **qué pruebas corresponden a cada versión**, conservar su consistencia con las funcionalidades implementadas y facilitar la ejecución repetida de las verificaciones.

RELACIÓN ENTRE VALORES, PRINCIPIOS Y PRÁCTICAS

- **Valores:** Indican qué aspectos se priorizan.
- **Principios:** Orientan cómo deberían tomarse las decisiones y organizarse el trabajo.
- **Prácticas:** Establecen actividades concretas que ayudan a aplicar esos lineamientos.

Adoptar una práctica no garantiza por sí solo cumplir los principios y valores. Por ejemplo, automatizar pruebas no convierte a un equipo en ágil si sigue dejando todas las evaluaciones para el final y no existe colaboración entre sus integrantes.

ATDD — DESARROLLO CONDUCIDO POR PRUEBAS DE ACEPTACIÓN

ATDD consiste en definir colaborativamente, antes de implementar una funcionalidad, **ejemplos y pruebas de aceptación que expresen las necesidades y expectativas del negocio**.

Busca construir un entendimiento compartido de lo que debe hacer el producto, reduciendo ambigüedades y previniendo defectos derivados de interpretaciones incorrectas.

PROCESO DE ATDD

1. **Identificar la característica:** Comprender qué necesidad del negocio debe resolverse y expresarla mediante una historia de usuario.
2. **Explorar ejemplos:** Analizar las reglas de negocio, condiciones y situaciones que deben contemplarse.
3. **Definir pruebas de aceptación de alto nivel:** Establecer los comportamientos y resultados esperados que permitirán considerar aceptable la historia.
4. **Agregar y automatizar pruebas cuando corresponda:** Transformar los ejemplos en comprobaciones concretas, automatizando las adecuadas.
5. **Implementar y verificar:** Desarrollar la solución y ejecutar las pruebas para comprobar el cumplimiento de las expectativas.
6. **Explorar, corregir y aceptar:** Complementar las pruebas con otras evaluaciones, corregir los defectos detectados y comprobar la aceptación de la historia.

El proceso admite retroalimentación y nuevas pruebas: **no es una secuencia rígida que se realiza una única vez**.

DIFERENCIA ENTRE TDD Y ATDD

Aspecto	TDD	ATDD
Foco	Comportamiento de unidades o componentes.	Funcionalidad y comportamiento esperado por el negocio.
Tipo de pruebas	Principalmente pruebas unitarias.	Pruebas de aceptación.
Perspectiva	Principalmente técnica.	Necesidades y expectativas del usuario o cliente.
Colaboración	En torno al desarrollo y diseño del código.	Negocio, desarrolladores y testers.
Objetivo	Orientar la implementación y el diseño interno.	Aclarar qué debe construirse y cómo comprobar su aceptación.

Ambas técnicas son complementarias: ATDD ayuda a establecer qué comportamiento necesita el negocio, mientras que TDD ayuda a construir ese comportamiento mediante componentes correctamente diseñados y comprobados.

ROL DEL TESTER EN LOS EQUIPOS ÁGILES

En Ágil, el tester ocupa un **rol colaborativo y transversal**, participando durante todo el desarrollo y no únicamente al finalizar la implementación. Su trabajo se relaciona con diferentes integrantes del equipo.

COLABORACIÓN CON EL PRODUCT OWNER (PO)

- Ayuda a preparar y revisar las historias de usuario.
- Colabora en la definición de los criterios de aceptación.
- Contribuye a comprender las reglas y necesidades del negocio.
- Identifica dependencias, ambigüedades y situaciones no contempladas.
- Participa en la comprobación de los criterios necesarios para aceptar las historias.

COLABORACIÓN CON LOS DESARROLLADORES

- Participa en la definición de estrategias y casos de prueba.
- Colabora con la automatización de comprobaciones.
- Proporciona feedback temprano sobre las funcionalidades.
- Identifica, comunica e investiga defectos.
- Propone mejoras funcionales y de usabilidad.

COLABORACIÓN CON EL EQUIPO DE INTEGRACIÓN O SISTEMAS

Cuando existen equipos responsables de integración u operaciones, el tester puede:

- Compartir información sobre planes, casos y datos de prueba.
- Coordinar las pruebas de integración entre componentes o equipos.
- Comunicar defectos y problemas de compatibilidad.
- Colaborar para comprobar si una funcionalidad está completa y puede integrarse.

Esta colaboración no requiere que todos los proyectos ágiles tengan un equipo de sistemas separado.

RESPONSABILIDADES PROPIAS DEL TESTER

También puede encargarse de preparar y mantener ambientes y datos de prueba, diseñar y ejecutar casos, realizar pruebas exploratorias, registrar defectos, aportar métricas y colaborar en la verificación del **Done**.

El tester no es el único responsable de la calidad, pero aporta competencias especializadas en pruebas, comprensión del negocio, pensamiento crítico, comunicación y automatización.

CAMBIOS Y DESAFÍOS CULTURALES

Adoptar Testing Ágil no consiste solamente en utilizar herramientas nuevas. Requiere **cambiar hábitos, responsabilidades y relaciones de trabajo**. Se destacan diferentes aspectos:

CULTURA ORGANIZACIONAL

Se necesita una cultura que valore la colaboración, la mejora continua y la responsabilidad compartida por la calidad. Una cultura que busca culpables puede dificultar que las personas comuniquen problemas. En cambio, una cultura orientada al aprendizaje facilita detectarlos y resolverlos tempranamente.

También influyen el ritmo sostenible, las relaciones con el cliente y el grado de autonomía de los equipos.

BARRERAS AL CAMBIO

Pueden aparecer dificultades como:

- Temor a perder responsabilidades o identidad profesional.
- Falta de capacitación.
- Desconocimiento de los conceptos ágiles.
- Resistencia por experiencias anteriores.
- Diferencias culturales entre los roles del equipo.

EXPECTATIVAS DE LA GERENCIA

La organización debe comprender que el cambio exige aprendizaje e inversión. No es razonable esperar que la automatización elimine inmediatamente todos los costos de Testing ni que los nuevos mecanismos produzcan resultados perfectos desde el inicio.

INTRODUCCIÓN Y CONSOLIDACIÓN DEL CAMBIO

Conviene explicar los objetivos, escuchar las preocupaciones, capacitar a las personas y permitir su participación en las decisiones.

Los cambios culturales requieren tiempo y continuidad. **La adopción de prácticas solo resulta sostenible cuando la organización acompaña los comportamientos que esas prácticas necesitan**.

CUADRANTES DEL TESTING ÁGIL

Los **cuadrantes del Testing Ágil** organizan las actividades de prueba a partir de dos dimensiones:

- **Orientación:** pruebas enfocadas en el negocio o en la tecnología.
- **Propósito:** pruebas que **guían el desarrollo** o que **evalúan críticamente el producto**.

El resultado es una clasificación de cuatro cuadrantes, que permite identificar distintas perspectivas de evaluación.

Q1 — TECNOLOGÍA QUE GUÍA EL DESARROLLO

Incluye principalmente **pruebas unitarias y de componentes**. Se orienta a comprobar el comportamiento del código y sus unidades, proporcionando feedback rápido a los desarrolladores y contribuyendo al diseño interno.

Son pruebas especialmente adecuadas para automatizar, debido a que suelen ser pequeñas y pueden ejecutarse frecuentemente.

Q2 — NEGOCIO QUE GUÍA EL DESARROLLO

Incluye **ejemplos de negocio, pruebas de aceptación de historias, pruebas funcionales, prototipos y simulaciones**. Su propósito es aclarar qué comportamiento debe construir el equipo para satisfacer las necesidades del negocio.

Se relaciona especialmente con **ATDD** y combina comprobaciones manuales y automatizadas.

Q3 — NEGOCIO QUE EVALÚA CRÍTICAMENTE EL PRODUCTO

Incluye **Testing exploratorio, pruebas de usabilidad, recorridos de trabajo, experiencia del usuario y pruebas de aceptación con usuarios**. Busca conocer cómo se comporta el producto en situaciones reales de utilización e identificar problemas que no fueron anticipados durante la especificación.

Predomina la evaluación humana, aunque puede utilizar herramientas de apoyo.

Q4 — TECNOLOGÍA QUE EVALÚA CRÍTICAMENTE EL PRODUCTO

Incluye principalmente **pruebas de rendimiento, carga, seguridad, recuperabilidad y otros atributos de calidad**. Su propósito es comprobar características técnicas y no funcionales del sistema.

Suelen requerirse herramientas para simular usuarios, generar cargas, medir recursos o comprobar determinados comportamientos. Sin embargo, sus resultados también necesitan análisis e interpretación humana.

COMPARACIÓN DE LOS CUATRO CUADRANTES

Cuadrante	Orientación	Propósito	Ejemplos
Q1	Tecnología	Guiar el desarrollo	Unitarias y componentes
Q2	Negocio	Guiar el desarrollo	Historias, aceptación, ejemplos
Q3	Negocio	Evaluar el producto	Exploratorias, usabilidad, UAT
Q4	Tecnología	Evaluar el producto	Rendimiento, carga, seguridad

Los cuadrantes no son etapas secuenciales ni una obligación de ejecutar las pruebas en un orden determinado.

Sirven para comprobar que el equipo contempla las distintas perspectivas necesarias. Un producto puede tener excelentes pruebas unitarias y aun así presentar problemas de usabilidad, incumplir expectativas del negocio o no soportar la carga esperada.

Diferencia entre Q2 y Q3: en **Q2**, las pruebas de aceptación y los ejemplos del negocio se utilizan para **definir y guiar lo que debe construirse**. En **Q3**, las pruebas con usuarios y otras evaluaciones permiten **inspeccionar críticamente el producto construido**, identificando problemas de utilización o necesidades que no se habían contemplado.

PIRÁMIDE DEL TESTING ÁGIL

La **pirámide de Testing Ágil** representa cómo conviene distribuir el conjunto de pruebas, especialmente las automatizadas, para obtener retroalimentación rápida y mantener un costo razonable.

Propone realizar **muchas pruebas pequeñas en la base, una cantidad intermedia de pruebas de servicios o integración y menos pruebas completas a través de la interfaz**.

BASE — PRUEBAS UNITARIAS Y DE COMPONENTES

Constituyen la parte más amplia de la pirámide. Son pruebas pequeñas, rápidas y generalmente automatizadas, que permiten comprobar componentes concretos y localizar problemas con facilidad.

Al existir muchas de estas pruebas, el equipo puede detectar errores rápidamente sin necesidad de ejecutar el sistema completo.

NIVEL INTERMEDIO — SERVICIOS, API Y REGLAS DE NEGOCIO

Comprueba las interacciones entre componentes y comportamientos más amplios, como servicios web, API o reglas funcionales. Estas pruebas permiten evaluar la integración y ciertos comportamientos del negocio sin recorrer necesariamente toda la interfaz gráfica.

Generalmente requieren más recursos que las unitarias, pero menos que las pruebas completas de extremo a extremo.

PARTE SUPERIOR — INTERFAZ GRÁFICA Y FLUJOS COMPLETOS

Comprueba recorridos amplios del usuario a través del sistema, incluyendo la interacción entre sus diferentes componentes. Son importantes para detectar problemas que no aparecen cuando se prueban unidades o servicios de manera aislada.

Sin embargo, suelen ser **más lentas, más dependientes del ambiente y más costosas de mantener**, por lo que conviene utilizarlas selectivamente.

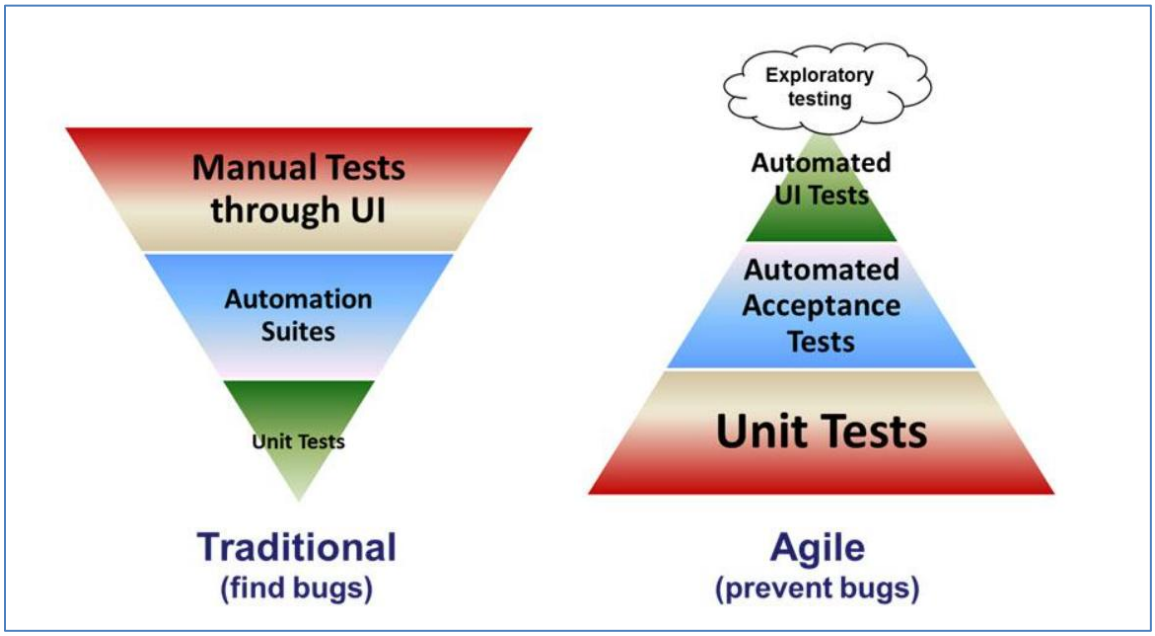
PRUEBAS MANUALES Y EXPLORATORIAS

Se las ubica por encima de la pirámide automatizada, para mostrar que **complementan las comprobaciones repetitivas**. Son especialmente útiles para investigar comportamientos inesperados, descubrir problemas de usabilidad y evaluar situaciones que requieren criterio humano.

PIRÁMIDE TRADICIONAL VS. ÁGIL

Aspecto	Testing tradicional	Testing ágil
Mayor concentración de pruebas	Interfaz y recorridos completos.	Unitarias y componentes.
Automatización	Menor proporción en los niveles bajos.	Gran cantidad de comprobaciones automatizadas desde la base.
Velocidad de feedback	Generalmente más lenta.	Generalmente más rápida.
Detección de problemas	Puede ocurrir cuando ya están integrados muchos componentes.	Se busca detectar problemas próximos al momento de su introducción.
Costo de ejecución y mantenimiento	Mayor dependencia de pruebas complejas y manuales.	Se priorizan pruebas pequeñas y repetibles, manteniendo una cantidad selectiva de pruebas completas.

Los porcentajes que aparecen en el gráfico —como una amplia mayoría de pruebas unitarias en Ágil— son **orientativos, no una regla obligatoria para todos los proyectos**. **La pirámide no indica que las pruebas de sistema o aceptación deban eliminarse**, sino que conviene equilibrarlas con una base sólida de comprobaciones rápidas y automatizadas.



AUTOMATIZACIÓN DEL TESTING ÁGIL

La **automatización es fundamental para sostener las pruebas frecuentes y la retroalimentación rápida** que buscan los equipos ágiles. Permite incorporar comprobaciones al trabajo cotidiano, ejecutar regresiones repetidamente y acompañar la integración continua.

BENEFICIOS

1. **Rápida ejecución:** Facilita obtener resultados poco después de introducir un cambio.
2. **Reusabilidad:** Una prueba automatizada puede ejecutarse repetidamente sobre nuevas versiones del producto.
3. **Mayor cobertura:** Permite ejecutar más comprobaciones de manera frecuente con los recursos disponibles.
4. **Mayor precisión:** Ejecuta de manera consistente pasos y comparaciones previamente definidas.
5. **Mayor alcance:** Permite simular cargas, operaciones o escenarios que serían difíciles de reproducir manualmente.
6. **Reducción de costos de ejecución:** Disminuye el esfuerzo manual necesario para repetir pruebas numerosas veces.

COSTOS Y LIMITACIONES

La automatización requiere **diseñar e implementar las pruebas, preparar datos y ambientes y mantener los scripts** cuando evoluciona el software. Por eso, no siempre resulta rentable automatizar una prueba que se ejecutará una sola vez o que cambia constantemente. Además:

- No todas las actividades de evaluación pueden automatizarse de manera útil.
- Una prueba automatizada puede comprobar consistentemente un comportamiento equivocado si fue mal diseñada.
- Las herramientas no reemplazan la exploración, el pensamiento crítico ni la comprensión del negocio.

La estrategia adecuada combina pruebas automatizadas y evaluación humana, aprovechando la automatización para obtener feedback rápido y reservando las capacidades de las personas para investigar, interpretar y descubrir nuevas situaciones.

RELACIÓN ENTRE MANIFIESTO, PRÁCTICAS, CUADRANTES Y PIRÁMIDE

Estos elementos describen perspectivas complementarias del Testing Ágil:

- **Manifiesto y principios:** Establecen los objetivos y valores que orientan el trabajo.
- **Prácticas concretas:** Indican actividades que permiten aplicar esos lineamientos.
- **Cuadrantes:** Ayudan a determinar qué perspectivas de evaluación deben contemplarse y qué propósito tiene cada prueba.
- **Pirámide:** Orienta cómo distribuir las comprobaciones, especialmente las automatizadas, para obtener feedback rápido y evitar un exceso de pruebas costosas.

En conjunto, permiten **integrar la calidad al desarrollo, prevenir defectos, mantener una evaluación continua del producto y entregar valor al cliente de manera frecuente y confiable**.